

POLITECNICO DI TORINO

Laurea Magistrale in Ingegneria Matematica

Numerical optimization for large scale problems

**Unconstrained Optimization – Derivative–based
Optimization**



Bagnato Francesca	346710
Foni Gianmarco	347952
Maccarone Giovanna	349134

Anno Accademico 2025–2026

Contents

1	Introduction	3
2	Modified Newton Method	3
2.1	The standard version	3
2.2	A Variant: Eigenvalue Flipping	4
3	Truncated Newton Method	4
3.1	The standard version	4
3.2	A variant: Hessian-free method	5
4	Test problems	5
4.1	Problem 16: Banded Trigonometric	5
4.2	Problem 31: Broyden tridiagonal least-squares	6
5	Parameter tuning and implementation choices	7
6	Finite Differences Implementation	8
6.1	Finite-difference implementation for Trig16	8
6.2	Finite-difference implementation for Broyden31	9
7	Numerical Results	10
7.1	Problem 16: Banded Trigonometric (Trig16)	10
7.2	Problem 31: Broyden Tridiagonal (Broyden31)	12
A	Additional Numerical Results	15
A.1	Broyden	15
A.1.1	Contour plot	15
A.1.2	Rate n=2	16
A.1.3	Rate n=1e3	17
A.1.4	Rate n=1e4	18
A.1.5	Rate n=1e5	19
A.2	Trig16	20
A.2.1	Contour plot	20
A.2.2	Rate n=2	21
A.2.3	Rate n=1e3	22
A.2.4	Rate n=1e4	23
A.2.5	Rate n=1e5	24
B	Further Implementation	24
B.1	Comparison between the flipped strategy and the β -modified approach ($\beta = 1$) in the Trig16 framework	25
B.1.1	Contour plot	26
B.1.2	Rate n=1e5	26

B.2	Truncated Hess-Free	27
C	Code	29
D	Tables	59
D.1	Trig16	59
D.2	Broy31	64
D.3	Finite Difference Error Estimation	70
D.4	Diagnostic Trig16	72

1 Introduction

This report presents a numerical study on two derivative-based methods for unconstrained optimization: the Modified Newton method and the Truncated Newton method. Both algorithms are implemented in MATLAB and integrated with a back-tracking line search strategy to ensure global convergence. The methods are tested on two benchmark problems from [1]: Problem 16 and Problem 31. The goal is to assess robustness and scalability when the dimension grows ($n = 2, 10^3, 10^4, 10^5$), considering both exact derivatives and finite-difference (FD) approximations. For each test problem we follow the assignment protocol: we use the suggested starting point $\bar{x}$, and five additional starting points sampled uniformly in the hypercube $[\bar{x}_1 - 1, \bar{x}_1 + 1] \times \dots \times [\bar{x}_n - 1, \bar{x}_n + 1]$ with a fixed random seed equal to the minimum student ID in the team. Performance is compared in terms of iterations, computational time, final gradient norm, success/failure with respect to a tolerance on $\|\nabla f(x_k)\|$, and (when required) experimental convergence rates. Finally, we extend the study to the finite-difference setting using an efficient implementation specifically tailored to the considered problem, which exploits the sparsity of the derivatives in order to reduce the number of function/gradient evaluations with respect to general-purpose FD schemes.

2 Modified Newton Method

2.1 The standard version

The first implemented method is the Modified Newton method for unconstrained optimization, which is associated to the function `modified_newton_method`. The classic Newton method computes, at each iteration, given the iterate x_k , the search direction p_k by solving the linear system $H_k p_k = -g_k$, where $g_k = \nabla f(x_k)$ and $H_k = \nabla^2 f(x_k)$. This direction is a descent direction if and only if H_k is positive definite. However, for a general nonconvex function, the Hessian may not be positive definite at every point, and in that case the Newton direction may not lead to a decrease of the objective function. The Modified Newton method addresses this issue by replacing H_k with a corrected matrix

$$B_k = H_k + E_k, \quad (1)$$

where E_k is the correction matrix; the quadratic model is consequently built using B_k instead of H_k . Clearly, we want B_k to be as close as possible to H_k , so the matrix E_k needs to be as small as possible. The matrix E_k is defined as $E_k \approx \tau_k I$, where $\tau_k \geq 0$ is chosen so that B_k is positive definite. If H_k is already positive definite, $\tau_k = 0$ and $B_k = H_k$ and the method reduces to the classical Newton method. Otherwise, τ_k is introduced until B_k becomes positive definite.

In our code, at each iteration, the Hessian H_k is computed and its diagonal entries are checked: if all of them are greater than a set tolerance (necessary condition), the matrix is considered positive definite and no correction is applied ($\tau_k = 0$). Otherwise, an initial value of τ_k is set based on the most negative diagonal entry. Starting from this value, the code tries to compute the Cholesky factorization of $B_k = H_k + \tau_k I$: if the factorization succeeds, B_k is positive definite and the correction is accepted; if it fails, the correction parameter τ_k is increased $\tau_{k+1} = \max(\beta, 2\tau_k)$ and the factorization is attempted again. Once a valid factorization R is obtained, the search direction p_k is computed by solving the system $B_k p_k = -g_k$ using the Cholesky factors R and $R^\top$: $p_k = R \setminus (R^\top \setminus (-g_k))$. Once the direction p_k is computed, the step length α_k is chosen using a backtracking procedure based on the Armijo condition

$$f(x_k + \alpha_k p_k) \leq f(x_k) + \alpha_k c_1 g_k^\top p_k, \quad (2)$$

with $c_1 \in (0, 1)$. Starting from $\alpha_k = 1$, the step length is reduced by a factor $\rho \in (0, 1)$ until (2) is satisfied, or until a maximum number of backtracking steps is reached. The new point is then updated as $x_{k+1} = x_k + \alpha_k p_k$, and the procedure is repeated until the norm of the gradient falls below a chosen tolerance, or a maximum number of iterations is reached. Finally, the point, the gradient, and the function value are updated, and all relevant quantities are stored at each iteration, so that the behaviour of the method can be analyzed afterwards.

2.2 A Variant: Eigenvalue Flipping

A second version of the Modified Newton method, implemented in `modified_newton_method_flip`, corrects a non-positive-definite Hessian by acting directly on its eigenvalues rather than adding a multiple of the identity as in (1). This variant, suggested in [3], builds the corrected matrix B_k by acting directly on the eigenvalues of H_k : every non-positive eigenvalue λ_i is replaced by $|\lambda_i|$ (or by a small value ε , if $|\lambda_i|$ is too close to zero), while the eigenvectors of H_k are left unchanged. This means that B_k has the same eigenvectors as H_k , but all its eigenvalues are positive, so B_k is guaranteed to be positive definite. This variant was applied only to the Banded Trigonometric function 16, since for this problem H_k is diagonal at every iterate, so its eigenvalues coincide with the diagonal entries.

In practice, when H_k is diagonal, as for the Banded Trigonometric function, this operation is very simple since the eigenvalues of a diagonal matrix are its diagonal entries: flipping the eigenvalues just means flipping the sign of the negative diagonal entries directly. The code also includes a general branch that computes the full eigendecomposition when H_k is not diagonal, making the method applicable to problems with a dense Hessian. Apart from this different correction strategy (Fig.1), the rest of the method is identical to the original Modified Newton method: once B_k is obtained, the search direction is computed by solving $B_k p_k = -g_k$, and the step length is chosen using the same backtracking procedure based on the Armijo condition. The function also returns an additional output, `n_flips`, which represents the number of iterations in which at least one eigenvalue had to be corrected.

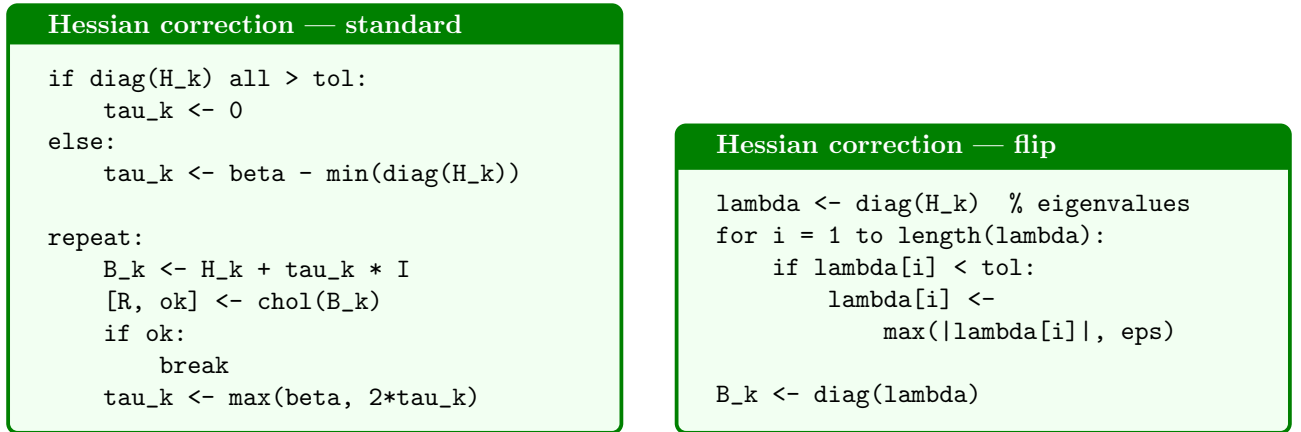


Figure 1: Comparison of the Hessian correction step: multiple of the identity added iteratively (left) vs. direct eigenvalue flip on the diagonal (right).

3 Truncated Newton Method

3.1 The standard version

The second method implemented is the Truncated Newton Method, which corresponds to the function `truncated_newton_method`. The classic Newton method computes the search direction p_k by solving the exact linear system $H_k z = c_k$, where $c_k = -\nabla f(x_k)$, $H_k = \nabla^2 f(x_k)$ and $z = p_k$. However, for large-scale problems, computing the exact solution of this system at each iteration is computationally very expensive. The Truncated Newton method addresses this issue using an iterative algorithm, the Conjugate Gradient method, to approximately solve the Newton system. In particular, it applies the Conjugate Gradient method as an inner loop. Since an exact search direction is not strictly necessary when the current point is far from the optimal solution, the CG iterations can be "truncated" to save time. In our implementation, the accuracy of the inner CG loop is governed by a dynamic tolerance η_k , which forces a less precise solution at the beginning and a more accurate one as we get closer to the minimum:

$$\eta_k = \min \left(0.5, \sqrt{\|g_k\|} \right) \|g_k\|, \quad (3)$$

where $g_k = \nabla f(x_k)$. A known limitation of the CG method is that it requires the Hessian matrix H_k to be positive definite. To guarantee that we always obtain a valid descent direction even for nonconvex functions, we monitor the curvature at each inner iteration j . Given the current conjugate direction d_j , we compute the term $d_j^\top H_k d_j$. If we detect a non-positive or near-zero curvature (specifically, $d_j^\top H_k d_j \leq 10^{-10} \|d_j\|^2$), we must terminate the CG loop. If this issue arises at the very first iteration ($j = 0$), the algorithm safely falls back to the steepest descent direction $p_k = -g_k$. If it happens at a later iteration, the method simply accepts the most recently computed valid approximation z as the search direction p_k . The inner loop successfully concludes when the norm of the residual falls below η_k . If the maximum number of inner iterations (`max_cg`) is reached without meeting this condition, the algorithm exits the loop and uses the last available approximation z .

Once a valid search direction p_k is established, the algorithm determines the step length α_k through a backtracking line search based on the Armijo condition as explained before, in the Modified Newton method. Finally, the point, the gradient, and the function value are updated, and all relevant quantities are stored at each iteration, so that the behaviour of the method can be analyzed afterwards.

3.2 A variant: Hessian-free method

A second version of the Truncated Newton method, implemented in `truncated_hess_free`, has been developed. This version is a matrix-free variant. The key difference with respect to the classical Truncated Newton method lies in how the Hessian is handled: the standard method, in fact, requires the exact Hessian matrix $\nabla^2 f(x_k)$ to be explicitly built at every iteration. In this variant, instead, the Hessian is never built: each Hessian-vector product is approximated with a centered finite difference on the gradient,

$$\nabla^2 f(x_k) p \approx \frac{\nabla f(x_k + hp) - \nabla f(x_k - hp)}{2h},$$

so that the method only needs gradient evaluations, and the Hessian matrix is neither formed nor stored. This version makes the method widely cheaper in memory and computational cost for large-scale problems, since computing and storing the full $n \times n$ Hessian is completely avoided.

Hessian-vector product via centered finite differences

```
function Hp = hess_vec_fd(x_k, p, gradf, h_fd)
    norm_p <- norm(p)

    if norm_p < eps:
        return zeros(size(p))

    h <- h_fd / norm_p
    Hp <- (gradf(x_k + h * p) - gradf(x_k - h * p)) / (2 * h)
end
```

Figure 2: Matrix-free computation of the Hessian-vector product $H(x_k)p$ using centered finite differences on the gradient.

4 Test problems

4.1 Problem 16: Banded Trigonometric

The first test function considered is the Banded Trigonometric Function, defined in the MATLAB code as `problem_trig16`. The objective function is given by

$$F(x) = \sum_{i=1}^n i [(1 - \cos x_i) + \sin x_{i-1} - \sin x_{i+1}], \quad (4)$$

with standard starting point $\bar{x} = (1, \dots, 1)^\top \in \mathbb{R}^n$ and boundary conditions $x_0 = x_{n+1} = 0$. For implementation purposes, this expression was rewritten into the equivalent form

$$F(x) = \sum_{i=1}^n i(1 - \cos x_i) + 2 \sum_{i=1}^{n-1} \sin x_i - (n-1) \sin x_n. \quad (5)$$

The gradient and the Hessian can be computed analytically. Each component of the gradient depends only on the corresponding variable x_i :

$$g_i(x) = \begin{cases} i \sin x_i + 2 \cos x_i & i < n \\ n \sin x_n - (n-1) \cos x_n & i = n \end{cases} \quad (6)$$

and the Hessian matrix's entries are

$$H_{ii}(x) = \begin{cases} i \cos x_i - 2 \sin x_i & i < n \\ n \cos x_n + (n-1) \sin x_n & i = n \end{cases} \quad (7)$$

This is the main structural property of the function: since there is no coupling between different variables in the derivatives, both the gradient and the Hessian are very cheap to evaluate, even for large values of n . The Hessian, in particular, is a diagonal matrix. In the code, this is exploited by building the Hessian directly as a sparse diagonal matrix, which avoids storing a full $n \times n$ matrix. The function is smooth, but it is not convex. Since F is built up of sine and cosine terms, it is periodic in each variable, and each diagonal entry of the Hessian changes sign depending on x_i . This means that the Hessian is not always positive definite along the optimization trajectory. In addition, the function has many stationary points, including local minima, maxima, and saddle points. For this reason, the behaviour of the optimization methods depends significantly on the starting point, and this makes the function a good test case for both the Modified Newton Method and the Truncated Newton Method, which are specifically designed to handle Hessians that may not be positive definite.

4.2 Problem 31: Broyden tridiagonal least-squares

The second test function considered is the Broyden Tridiagonal function, defined in the MATLAB code as `problem_broyden31`. Unlike the previous test function, this problem is formulated as a nonlinear least-squares problem: a residual vector $r(x) \in \mathbb{R}^n$ is defined component-wise as

$$r_k(x) = (3 - 2x_k)x_k - x_{k-1} - 2x_{k+1} + 1, \quad (8)$$

and the objective function is given by

$$F(x) = \frac{1}{2} \|r(x)\|^2 \quad (9)$$

with boundary conditions $x_0 = x_{n+1} = 0$. The starting point provided is adopted $\bar{x} = (-1, \dots, -1) \in \mathbb{R}^n$. Each residual r_i depends on x_i and on its two neighboring components x_{i-1} and x_{i+1} giving the problem its sparse tridiagonal structure. Since F is defined as a sum of squares, the gradient and the Hessian can be expressed in terms of the Jacobian $J(x)$ of the residual vector. Each row of $J(x)$ has at most three nonzero entries, since r_i depends only on x_{i-1} , x_i , and x_{i+1} :

$$J_{i,j}(x) = \begin{cases} -1 & j = i-1, \\ 3 - 4x_i & j = i, \\ -2 & j = i+1, \\ 0 & \text{otherwise,} \end{cases} \quad (10)$$

so that $J(x)$ is a sparse tridiagonal matrix. Using this Jacobian, the exact gradient of F is

$$g(x) = J(x)^T r(x), \quad (11)$$

and the exact Hessian is

$$H(x) = J(x)^\top J(x) - 4 \text{diag}(r(x)). \quad (12)$$

Since J is tridiagonal, the product $J(x)^\top J(x)$ gives a pentadiagonal matrix, and the correction term $-4 \text{diag}(r(x))$ only affects the main diagonal without altering this structure. Therefore, $H(x)$ is pentadiagonal and it is built in the code as a sparse matrix.

In addition, this function provides the handle `xstarfun`, which computes a reference global minimum x^* for the Broyden Tridiagonal problem numerically, using `lsqnonlin`, which is a MATLAB's solver for nonlinear least-squares problems, on the residual $r(x)$: starting from the point x_0 , it searches for the value of x that makes the residual $r(x)$ as close as possible to zero. Since $F(x) \geq 0 \forall x$, if the residual norm at the computed solution is numerically zero, then $F(x^*) \approx 0$, which ensures that x^* is a global minimum. This reference solution is useful to evaluate how close the point produced by the Modified Newton and Truncated Newton methods gets to the true minimum, which is not possible for the Banded Trigonometric function due to its periodicity and the resulting multiplicity of stationary points.

5 Parameter tuning and implementation choices

The algorithmic parameters were selected through preliminary numerical experiments aimed to balance robustness and computational efficiency across different dimensions. The stopping tolerance on the gradient norm was set to `tolgrad` = 10^{-6} ; this was not treated as a free parameter, but as a hard constraint. The remaining parameters, some shared between the Modified and Truncated algorithms, such as `btmax` (maximum number of backtracking iterations), `c1` (Armijo condition), `kmax` (maximum number of outer iterations), `rho` (rescaling factor in backtracking), and the problem-specific ones such as `beta` (Hessian correction term) and `max_cg` (maximum number of CG iterations for Truncated), were tuned using a combination of successive refinements and a grid search.

We started by generating a set of baseline configurations, combining a few discrete values for each parameter, and tested it with multiple runs on random initial points. If a run did not converge, we increased the parameter values step by step, following a fixed priority order (prioritizing `btmax`, then `max_cg`, then `kmax`), and tested the updated configuration again. We repeated this until every configuration had been tested, obtaining a set of admissible and robust configurations. During the second refinement stage, these admissible configurations were evaluated more rigorously by running them several times on problems of size 10^3 and 10^4 . Configurations that failed were removed. For each surviving configuration, a loss function based on the median running time over repeated starts, was defined. Finally, the configuration with the lowest loss was selected. This approach produces robust configurations that scale reasonably with the problem size. Moreover, it is constructive: parameters such as `btmax`, `max_cg`, and `kmax` act as safeguards, and only take large values when the problem actually requires them.

Table 1: Best parameter configurations selected for the Modified and Truncated Newton methods on the Trig16 and Broyden31 test problems.

Method	Trig16	Broyden31
Modified Newton	<code>btmax</code> = 30	<code>btmax</code> = 10
	<code>c1</code> = $1e-4$	<code>c1</code> = $1e-3$
	<code>kmax</code> = 200	<code>kmax</code> = 50
	<code>rho</code> = 0.5	<code>rho</code> = 0.5
	<code>beta</code> = $1e-2$	<code>beta</code> = $1e-3$
Truncated Newton	<code>btmax</code> = 40	<code>btmax</code> = 10
	<code>c1</code> = $1e-3$	<code>c1</code> = $1e-4$
	<code>kmax</code> = 500	<code>kmax</code> = 50
	<code>rho</code> = 0.5	<code>rho</code> = 0.8
	<code>max_cg</code> = 1000	<code>max_cg</code> = 100

6 Finite Differences Implementation

In this following section, we investigate how the performance of the optimization methods is affected when derivatives are approximated by finite differences (FD). The main goal is to assess robustness with respect to the choice of the FD step size and to compare a fixed increment with a component-wise scaled increment. For each differentiation we consider a standard fixed step

$$h = 10^{-k}, \quad k \in \{4, 8, 12\},$$

as well as a scaled step defined component-wise as

$$h_i = 10^{-k}|x_i|, \quad i = 1, \dots, n.$$

For both test problems (Trig16 and Broyden31) and for each dimension $n \in \{2, 10^3, 10^4, 10^5\}$, the methods are tested from six different starting points: the reference point $\bar{x}$ and five random points sampled near it. In our implementation, we use finite differences to approximate the derivatives, using central schemes.

We distinguish between two main testing scenarios: the first case, in which we use the analytical formula for the gradient while approximating the Hessian matrix and the second case, in which both the gradient and the Hessian are approximated using finite differences.

Finite difference step. The finite-difference step is computed by `steps_fd`, which returns a step vector `hvec`. Two step types are available: a constant step, denoted h , and the relative step h_i defined above. For the relative step, a safeguard is used: if $x_i = 0$ the step would be zero, and if h_i becomes smaller than 10^{-14} it could cause numerical problems, so in both cases the step is set to a minimum value instead.

6.1 Finite-difference implementation for Trig16

Case 1: Exact gradient, approximated Hessian. Owing to the separability established in Section 4.1, each gradient component g_i depends only on x_i , so perturbing all coordinates at the same time with the step vector h does not mix the derivatives together. We approximate the diagonal Hessian via a central difference of the exact gradient,

$$H_{jj} \approx \frac{g_j(x+h) - g_j(x-h)}{2h_j}, \quad j = 1, \dots, n, \quad (13)$$

using two gradient evaluations (at $x+h$ and $x-h$), while exploiting the separability of the problem to compute all diagonal entries simultaneously.

Case 2: Fully finite-difference derivatives. Approximating the gradient requires a dedicated separable decomposition, different from both forms of F used in Section 4.1. Regrouping (5) by index gives

$$F(x) = \sum_{i=1}^n \varphi_i(x_i), \quad \varphi_i(x_i) = i(1 - \cos x_i) + c_i \sin x_i, \quad c_i = \begin{cases} 2 & i < n \\ -(n-1) & i = n \end{cases}, \quad (14)$$

where each local term φ_i depends on x_i alone. Gradient and Hessian are then obtained via vectorized central differences,

$$g_i \approx \frac{\varphi_i(x_i + h_i) - \varphi_i(x_i - h_i)}{2h_i}, \quad H_{ii} \approx \frac{\varphi_i(x_i + h_i) - 2\varphi_i(x_i) + \varphi_i(x_i - h_i)}{h_i^2}. \quad (15)$$

Note that the original formulation (4) cannot be used for finite differences because each term depends on x_i , x_{i+1} , and x_{i-1} : perturbing all coordinates at the same time prevents the neighbor terms ($\sin x_{i\pm 1}$) from canceling out, so the derivatives are not computed correctly. For this reason Case 2 for Trig16 relies exclusively on the reformulation which contains x_i only.

6.2 Finite-difference implementation for Broyden31

Case 1: Exact gradient, approximated Hessian. In this first scenario, we use the exact formula for the gradient and approximate only the Hessian matrix. To approximate the Hessian, we use the centered difference formula applied to the gradient:

$$H_{:,j} \approx \frac{\nabla f(x + h_j e_j) - \nabla f(x - h_j e_j)}{2h_j}, \quad (16)$$

where e_j denotes the j -th coordinate vector and h_j is the finite-difference increment. This formula provides the j -th column of the Hessian. Since we are dealing with very large problems, computing every single entry of the Hessian would make the algorithm too slow. Instead, we exploit the fact that the Hessian is sparse and pentadiagonal. We apply a 5-coloring finite difference technique, which divides the variables into five groups according to their position, for example

$$\mathcal{C}_1 = 1, 6, 11, \dots, \quad \mathcal{C}_2 = 2, 7, 12, \dots,$$

and similarly for the remaining groups. The key observation is that Hessian contributions from variables in the same group do not overlap. Therefore, they can be perturbed simultaneously without interference. In the MATLAB implementation, for a given color i , a perturbation vector is defined as

$$d^{(c)}_j = \begin{cases} h_j & j \in \mathcal{C}_i, \\ 0 & \text{otherwise.} \end{cases}$$

The centered difference

$$\frac{g(x + d^{(c)}) - g(x - d^{(c)})}{2h_j}$$

gives us the Hessian columns for all variables in $\mathcal{C}_i$, where $g(x) := \nabla(x)$. Since the Hessian is sparse, only the nonzero entries are stored. After all five colors have been considered, the approximate Hessian is assembled as a sparse matrix and is finally symmetrized. This last operation reduces small asymmetries that can arise from numerical errors in the finite-difference calculations.

Case 2: Fully finite-difference derivatives. In the second scenario, both the gradient and the Hessian are approximated numerically. The first step is to approximate the Jacobian of the residual vector $r(x)$. Since, for the Broyden31 problem, the Jacobian is tridiagonal, we can again exploit its sparse structure by using a coloring technique. In this case, a 3-coloring strategy is sufficient. The variables are divided into three groups:

$$\mathcal{C}_1 = 1, 4, 7, \dots, \quad \mathcal{C}_2 = 2, 5, 8, \dots, \quad \mathcal{C}_3 = 3, 6, 9, \dots$$

Variables belonging to the same group affect different entries of the Jacobian and can therefore be perturbed simultaneously. For each variable, the corresponding column of the Jacobian is approximated using the centered finite-difference formula on the residuals:

$$J_{:,j} \approx \frac{r(x + h_j e_j) - r(x - h_j e_j)}{2h_j}. \quad (17)$$

In the MATLAB implementation, the perturbations associated with variables of the same color are collected in a single vector $d^{(c)}$, defined, as before, as

$$d^{(c)}_j = \begin{cases} h_j & j \in \mathcal{C}_i, \\ 0 & \text{otherwise.} \end{cases}$$

The residual is then evaluated at $x + d^{(c)}$ and $x - d^{(c)}$ and the gradient is built as

$$g_{FD}(x) \approx J(x)^T r(x).$$

For the Hessian, we use the same 5-coloring technique as in Case1. The only difference is that, instead of the exact gradient, we use the finite-difference gradient g_{FD} . Therefore, the j -th column of the Hessian is approximated as

$$H_{:,j} \approx \frac{g_{FD}(x + h_j e_j) - g_{FD}(x - h_j e_j)}{2h_j}. \quad (18)$$

7 Numerical Results

This section discusses the numerical results obtained for Trig16 and Broyden31, focusing on convergence, scalability, and the effect of finite-difference approximations.

The convergence rates reported in the tables are computed using the gradient norm as an error measure. For the convergence-rate plots, whenever possible, the error is instead measured as the distance from the exact minimizer computed using MATLAB's built-in `lsqnonlin`. The stopping criterion is fixed throughout all experiments at $\|\nabla f(x_k)\| < 10^{-6}$, with the maximum number of outer iterations determined according to the tuning strategy described in Section 5.

7.1 Problem 16: Banded Trigonometric (Trig16)

Multiple equivalent minimizers and the first Newton step. The periodic structure of Trig16 produces many equivalent local minima. This is already visible for $n = 2$ (Figure 3): depending on which basin its first step lands in, the Modified Newton method can converge to different minimizers from nearby starting points. Still, with exact derivatives, all 6 starting points converge in 3 to 7 iterations.

The different behaviour of Modified and Truncated Newton methods is mainly explained by their first steps. For Modified Newton, $B_k = \nabla^2 f(x_k) + \tau_k I$ is shifted only as much as necessary to ensure positive definiteness. When the Hessian has small or negative eigenvalues, $\lambda_{\min}(B_k)$ can remain close to β , leading to a large inverse-curvature step

$$p_0 = -B_0^{-1} \nabla f(x_0).$$

For the exact derivatives, the amplification $\|p_0\|/\|\nabla f(x_0)\|$ reaches approximately 86 (Table 50), potentially moving the iterate into the basin of a different equivalent minimizer. Truncated Newton limits this effect through the inner CG iteration, with amplification remaining of order $O(1)$ – $O(3)$ (Table 51), explaining its more local behaviour. The large first step of Modified Newton can be reduced by increasing β or by using the eigenvalue-flipping variant (`modified_newton_method_flip`, Section B.1.1).

Exact derivatives. Both methods converge from all starting points for $n = 2$ and $n = 10^3$. At $n = 10^4$, Modified Newton still converges from all starting points, whereas Truncated Newton converges in 4/6 cases. Although their convergence rates differ substantially (Table 2), Modified Newton is, as expected close to quadratic at $n = 10^3$ – 10^4 , with rates 2.46 and 2.34, while Truncated Newton instead remains around 1.5, consistently with the inexact-Newton forcing term $\eta_k = \min(0.5, \sqrt{\|\nabla f_k\|}) \|\nabla f_k\|$, which guarantees superlinear but subquadratic convergence. At $n = 10^5$, both methods deteriorate significantly: only 1 out of the 6 starting points converges within the iteration budget. This suggests that the bottleneck at large scale is the poor conditioning of the Hessian of Trig16 itself, rather than a limitation of either Newton correction strategy (Figure 4).

Finite differences. Both methods follow the same overall pattern under FD noise. In Case 1, with exact gradient and finite-difference Hessian, results stay close to those obtained with exact derivatives for $k = 4, 8$ and only degrade clearly at $k = 12$. Case 2, where both the gradient and the hessian are approximated, is much more problematic (Table 3): pooling all dimensions and step types, only 11

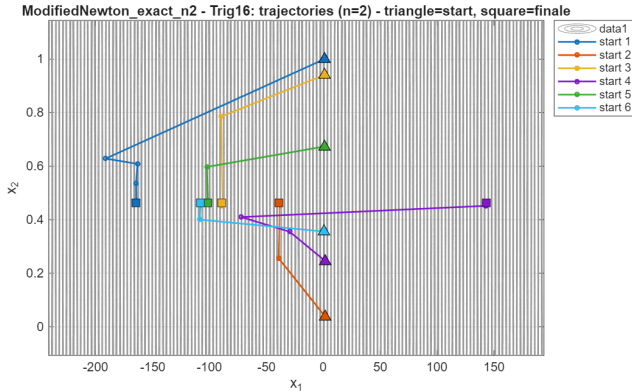
out of 36 Modified Newton and 10 out of 36 Truncated Newton runs converge for $k = 4$ and no run converges for $k = 8$ or $k = 12$. Moreover, the FD step type has a strong influence: the constant step h still allows several convergences at $n = 10^3$ – 10^4 , whereas the relative step h_i often fails immediately. This behaviour is consistent with the FD accuracy tests (Table 48), where it is clear that increasing k leads to an increase of round-off and cancellation errors. When both gradient and Hessian are affected by this noise, the methods can no longer reliably identify a descent direction on a already ill-conditioned problem.

Table 2: Trig16, exact derivatives: average experimental rate and success count (out of 6 starting points) for each dimension.

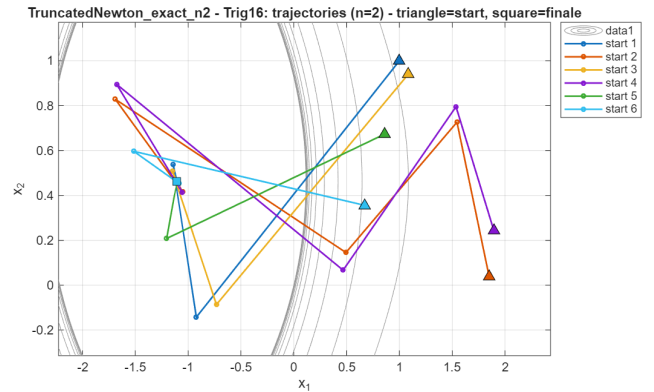
	$n = 2$	$n = 10^3$	$n = 10^4$	$n = 10^5$
Modified Newton (rate)	3.00	2.46	2.34	–
Modified Newton (succ.)	6/6	6/6	6/6	1/6
Truncated Newton (rate)	3.08	1.35	1.49	1.50
Truncated Newton (succ.)	6/6	6/6	4/6	1/6

Table 3: Trig16, FD Case 2: success count (out of 36 runs pooled over $n \in \{10^3, 10^4, 10^5\}$ and both step types h, h_i) and average rate where computable, by finite-difference order k .

	$k = 4$	$k = 8$	$k = 12$
Modified Newton (successes)	11/36	0/36	0/36
Modified Newton (rate, when available)	≈ 2.2	–	–
Truncated Newton (successes)	10/36	0/36	0/36
Truncated Newton (rate, when available)	≈ 1.4	–	–

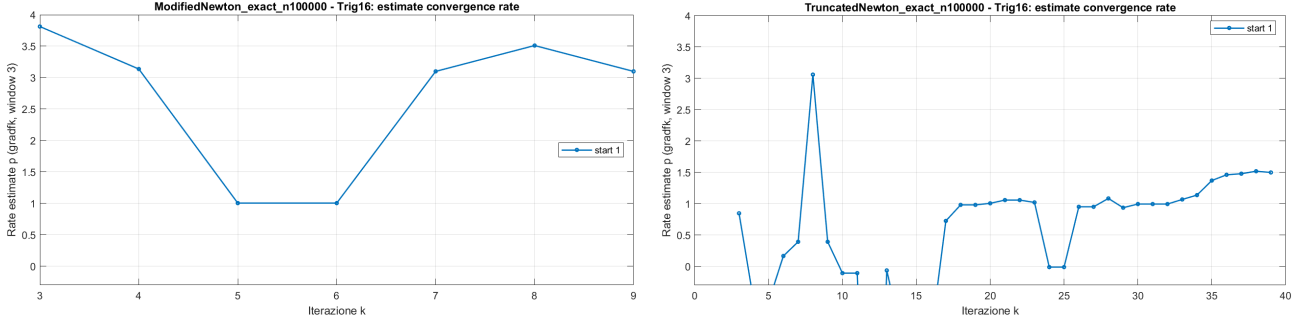


(a) Modified with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$, both types h and h_i) and for FD Case 2 with $k = 4$.



(b) Truncated with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$, both types h and h_i) and for FD Case 2 with $k = 4$.

Figure 3: Trig16 trajectories on the contour plot for $n = 2$. The two methods may converge to different equivalent minimizers because of the basin reached by the first step.



(a) Convergence rate for Modified Newton with exact derivatives. Only one starting point (xbar) converges within the iteration budget.

(b) Convergence rate for Truncated Newton with exact derivatives. Only one starting point (rand5) converges within the iteration budget.

Figure 4: Trig16 convergence rates for $n = 10^5$. Only one starting point converges for each method within the iteration budget, highlighting the severe large-scale ill-conditioning. Full per-configuration plots are reported in Appendix A.

7.2 Problem 31: Broyden Tridiagonal (Broyden31)

Exact derivatives. Both Modified Newton and Truncated Newton converge from every starting point at all tested dimensions $n \in \{2, 10^3, 10^4, 10^5\}$ (24/24 runs each), following different but equally direct paths (Figure 5). Modified Newton’s rate stays close to – but consistently below – the theoretical quadratic rate $q = 2$ (Table 4, Fig.6.). This gap can be explained by the small number of iterations per run (5 to 9 on average): with so few iterations, the measured rate reflects both the initial transient phase and the final quadratic phase, rather than the quadratic phase alone. Truncated Newton, instead, stays remarkably close to 1.50 in every tested configuration, at all dimensions. Our forcing term $\eta_k = \min(0.5, \sqrt{\|\nabla f(x_k)\|} \|\nabla f(x_k)\|)$ satisfies $\eta_k \rightarrow 0$, which guarantees superlinear convergence; since $\sqrt{\|\nabla f(x_k)\|}$ goes to zero more slowly than $\|\nabla f(x_k)\|$ itself, we expect a rate strictly between the linear and quadratic regimes, which matches the observed 1.5 (Figure 6). Truncated Newton needs more outer iterations than Modified Newton (e.g. 8 vs. 14 at $n = 10^4$), which is the usual cost of solving the system only approximately. Still, wall-clock time stays comparable at large n since Modified Neton needs a full Cholesky factorization each step. Full per-starting-point results are reported in Tables 28, 33, 38, and 43 (Appendix D).

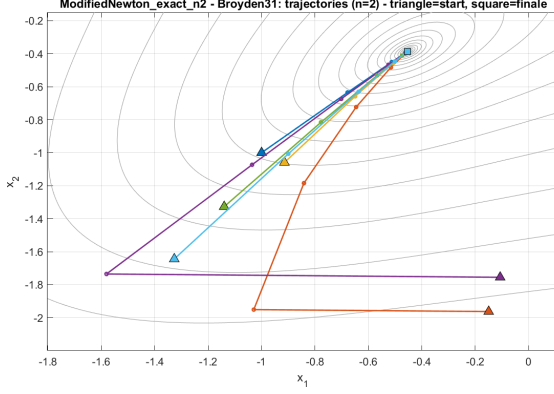
Finite differences. Case 1 (exact gradient, FD Hessian) barely changes anything: every configuration converges (36/36, both methods, all n), with rates close to the exact case. This is because Broyden31’s Hessian is tridiagonal and well-conditioned at every n , so the FD error stays small and bounded. (Tables 29, 30, 34, 35, 39, 40, 44, and 45, Appendix B). Case 2 (both derivatives approximated) is very different, and the breakdown pattern is the same for both methods: (Table 5, Figure 7; full detail in Tables 31, 32, 36, 37, 41, 42, 46, and 47, Appendix B) at $k = 4$ every run converges; at $k = 8$ both methods start failing; at $k = 12$ every run fails, for both methods, at every $n \geq 10^3$. These are not cases of slow convergence, the final gradient norm is orders of magnitude above tolerance and the stopping flag is always `maxit`: once FD noise also affects the search direction, the descent property breaks down and the outer loop simply runs out of iterations, this patten persist even with the Hessian free modification (results in B.2).

Table 4: Broyden31, exact derivatives: average experimental rate and success count (out of 6 starting points) for each dimension.

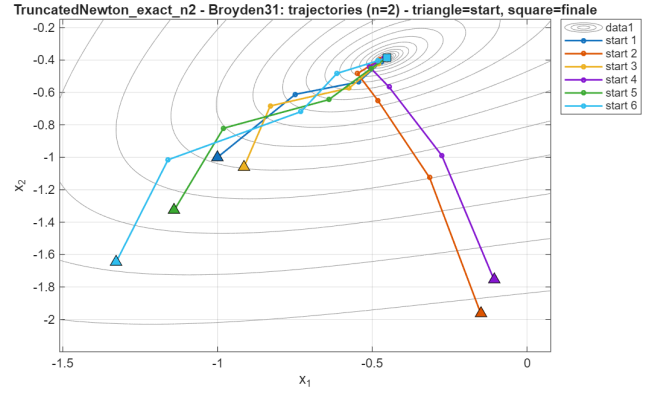
	$n = 2$	$n = 10^3$	$n = 10^4$	$n = 10^5$
Modified Newton (rate)	1.89	1.87	1.79	1.84
Modified Newton (succ.)	6/6	6/6	6/6	6/6
Truncated Newton (rate)	2.00	1.50	1.51	1.50
Truncated Newton (succ.)	6/6	6/6	6/6	6/6

Table 5: Broyden31, FD Case 2: success count (out of 36 runs tested across all $n \geq 10^3$, h and h_i) and average rate where computable, by finite-difference order k .

	$k = 4$	$k = 8$	$k = 12$
Modified Newton (successes)	36/36	21/36	0/36
Modified Newton (rate, when available)	1.795	≈ 1.42	—
Truncated Newton (successes)	36/36	30/36	0/36
Truncated Newton (rate, when available)	1.507	≈ 1.49	—

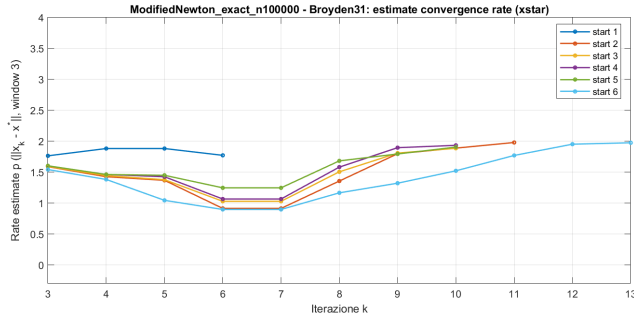


(a) Modified with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$ and both types h and h_i) and for FD Case 2 with $k = 4$.

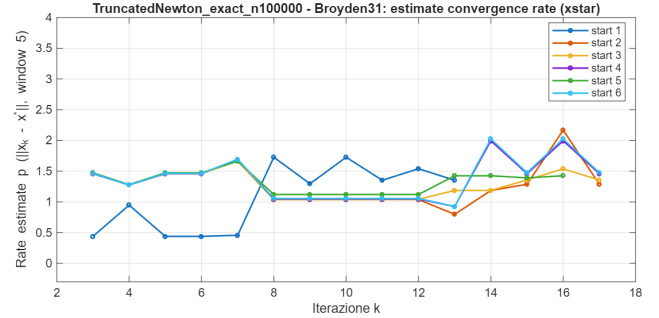


(b) Truncated with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$ and both types h and h_i) and for FD Case 2 with $k = 4$.

Figure 5: Broyden31 trajectories on the contour plot for $n = 2$.

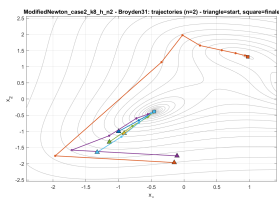


(a) Convergence rate for Modified Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and h_i) and in Case 2 with $k = 4$, type h .

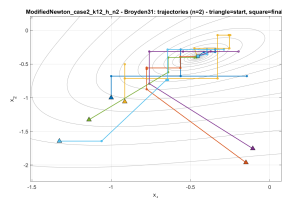


(b) Convergence rate for Truncated Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and h_i) and in Case 2 with $k = 4$.

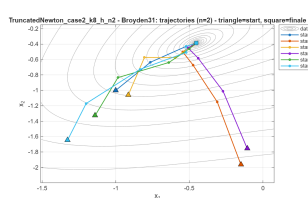
Figure 6: Broyden31 convergence rates for $n = 10^5$. The observed rates agree with the theoretical predictions: Modified Newton approaches 2, while Truncated Newton approaches 1.5. Case 2 with $k = 12$ does not converge within the prescribed number of iterations for either method.



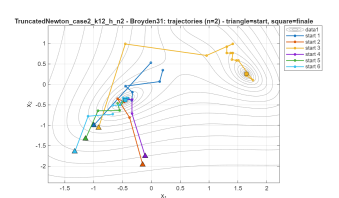
(a) MN, $k = 8$, type h .



(b) MN, $k = 12$, type h .



(c) TN, $k = 8$, type h .



(d) TN, $k = 12$, type h .

Figure 7: Contour plots for FD Case 2, $n = 2$. Both methods drift further from the smooth exact-derivative trajectories as k grows, foreshadowing the outright failures observed at larger n . See Figure in Appendix A.1.1 for the complete set of Case 2 contour plots.

Overall comparison. The two test problems show substantially different numerical behaviour. For Trig16, the main difficulty is already present with exact derivatives: severe ill-conditioning causes convergence to get worse as n increases. Broyden31, on the other hand, behaves well across all tested dimensions, with convergence rates that are almost independent of n .

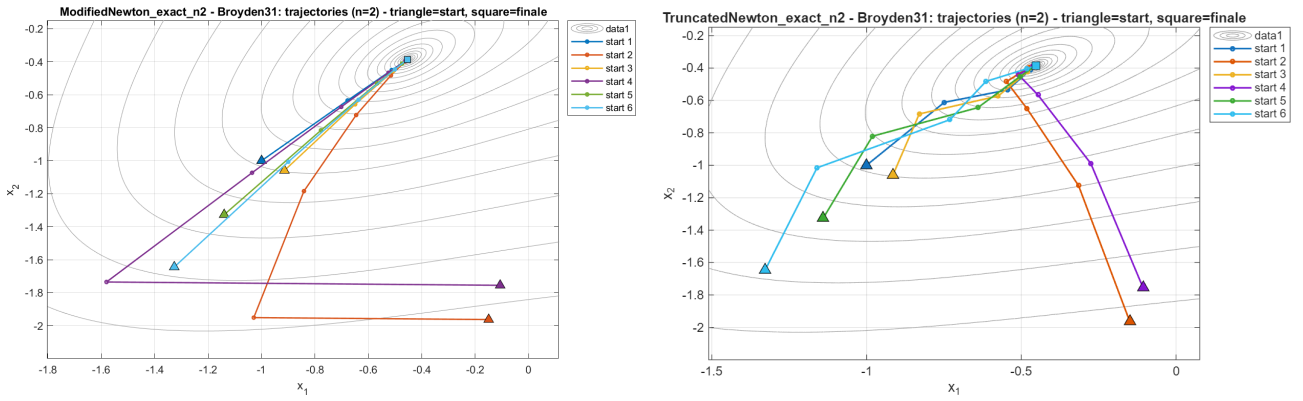
The finite-difference results further highlight this difference. For Broyden31, the main deterioration occurs in Case 2 as the FD step becomes smaller. For Trig16, instead, FD errors interact with the ill-conditioning of the problem, making the method sensitive to k , the step type, and n . Overall, conditioning is the main difficulty for Trig16, while for Broyden31 the main limitation is the accuracy of the finite-difference approximations.

A Additional Numerical Results

This appendix collects the supporting figures referenced in the main text. It includes contour plots for $n = 2$ for both Modified and Truncated Newton, followed by convergence-rate plots reported separately for each dimension, $n = 10^3, 10^4, 10^5$, for exact derivatives, FD Case 1, and FD Case 2. In many instances, the plots obtained with exact derivatives and those obtained with FD Case 1 are identical and are therefore omitted. Whenever the behaviour differs, the corresponding plot is explicitly included and identified. Whenever possible, the convergence rates are computed using the distance of the current iterate from the theoretical minimizer, obtained with MATLAB's built-in `lsqnonlin` function. In the remaining cases, the gradient norm is used as an estimate of the error, and the rate is computed over a moving window. These estimates may therefore differ slightly from the rates reported in the tables, which are always computed using the gradient norm and, in some cases, a different fitting window.

A.1 Broyden

A.1.1 Contour plot



(a) Modified with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$ and both types h and hi) and for FD Case 2 with $k = 4$.

(b) Truncated with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$ and both types h and hi) and for FD Case 2 with $k = 4$.

Figure 8: Broyden trajectories on the contour plot for $n = 2$.

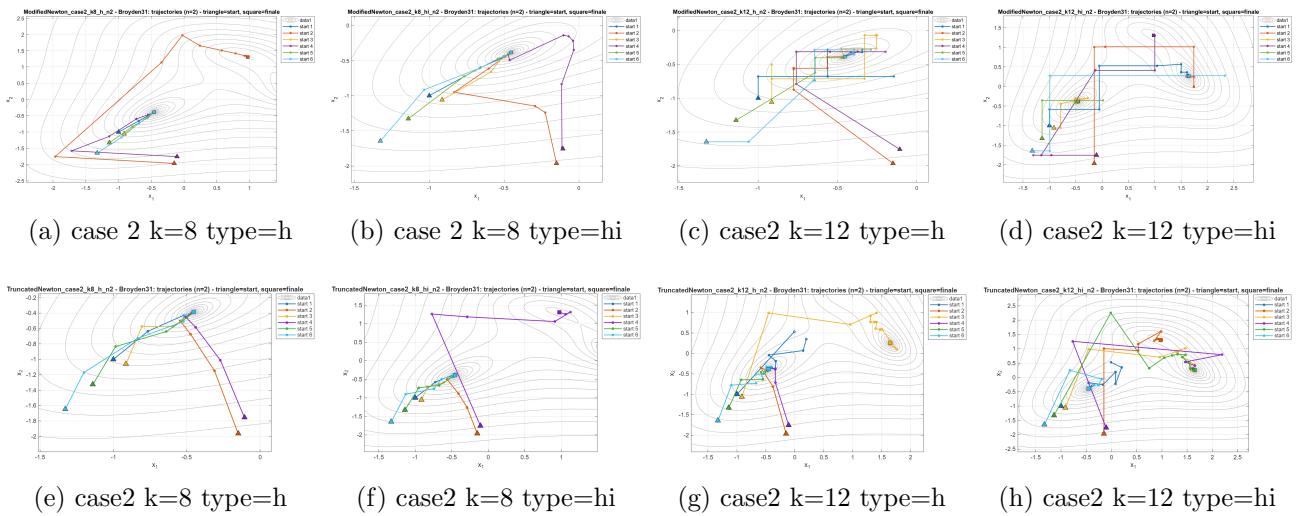
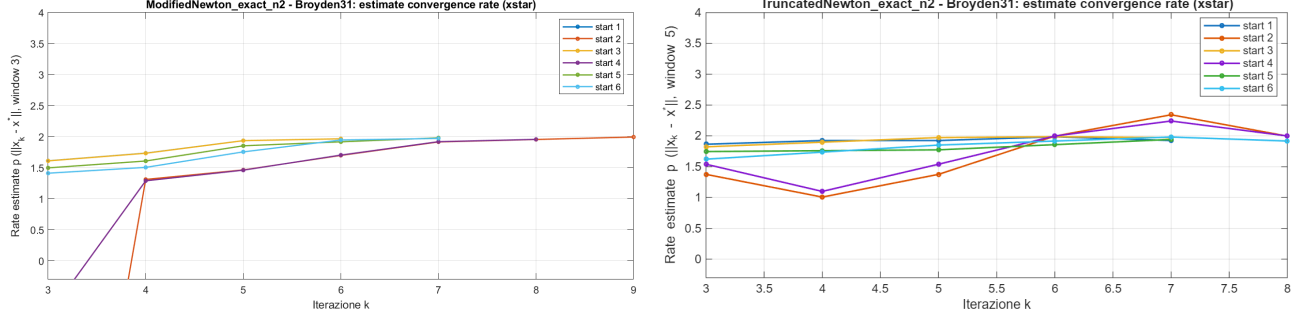


Figure 9: Contour plots for FD Case 2 for both Modified Newton (top row) and Truncated Newton (bottom row). These are the most challenging cases, where the methods may converge to stationary points which are not the actual minimizer. In the most pathological cases, the methods lose their expected quadratic convergence, as shown by the corresponding convergence-rate plots and by the sharp, nearly orthogonal turns in the contour plots.

A.1.2 Rate $n=2$



(a) Modified with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$ and both types h and hi) and for FD Case 2 with $k = 4$. (b) Truncated with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$ and both types h and hi) and for FD Case 2 with $k = 4$.

Figure 10: Broyden convergence rates for $n = 2$. Quadratic convergence is expected for Modified Newton, while superlinear convergence is expected for Truncated Newton. Due to the low dimensionality of the problem, both methods exhibit quadratic convergence in practice.

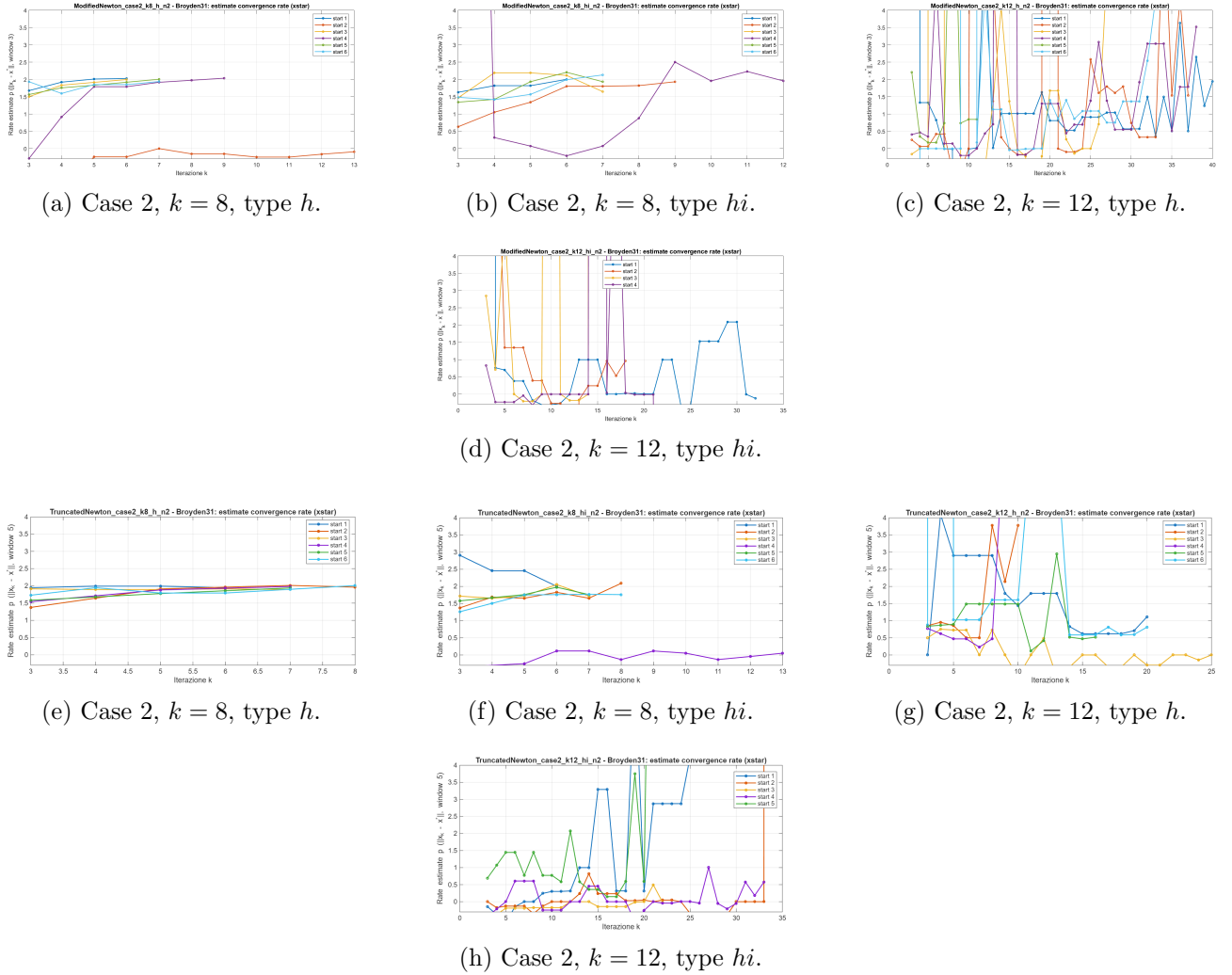
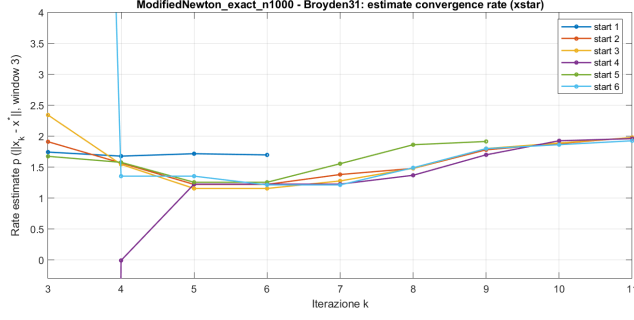
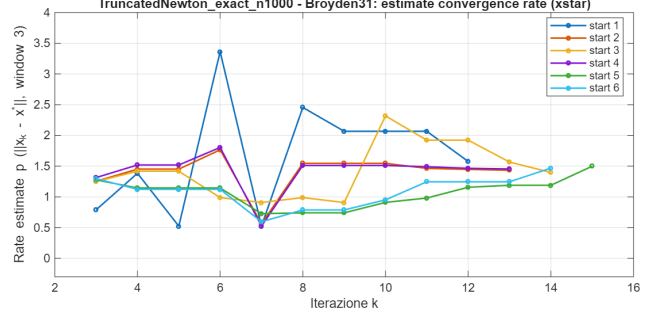


Figure 11: Convergence rates for FD Case 2. The first four plots correspond to Modified Newton and the last four to Truncated Newton. In these more challenging cases, the expected quadratic convergence is no longer consistently observed. Since the rates are computed with respect to the theoretical minimizer, runs that converge to different stationary points show pathological rate behaviour – the iterates aren't moving toward the reference solution.

A.1.3 Rate $n=1e3$

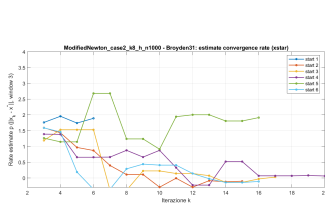


(a) Modified with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$ and both types h and h_i) and for FD Case 2 with $k = 4$.

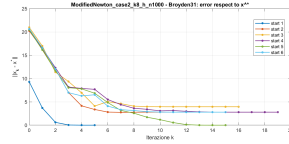


(b) Truncated with exact derivatives. The same behaviour is observed for FD Case 1 (all $k = 4, 8, 12$ and both types h and h_i) and for FD Case 2 with $k = 4$.

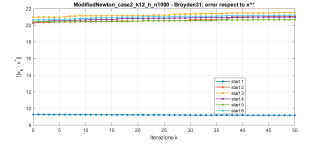
Figure 12: Broyden convergence rates for $n = 10^3$. The observed rates agree with the theoretical predictions: Modified Newton approaches 2, while Truncated Newton approaches 1.5 after an initial oscillatory transient, as expected from the chosen decay parameter.



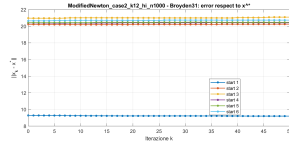
(a) Case 2, $k = 8$, type h .



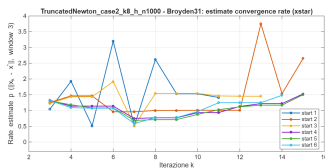
(b) Error from the minimizer, Case 2, $k = 8$, type h .



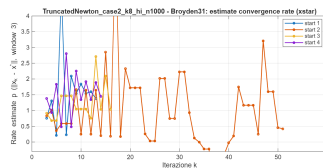
(c) Error from the minimizer, Case 2, $k = 12$, type h .



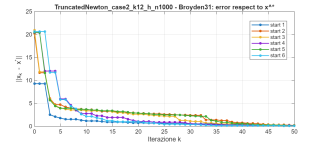
(d) Error from the minimizer, Case 2, $k = 12$, type h_i .



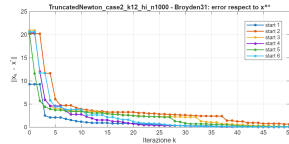
(e) Case 2, $k = 8$, type h .



(f) Case 2, $k = 8$, type h_i .



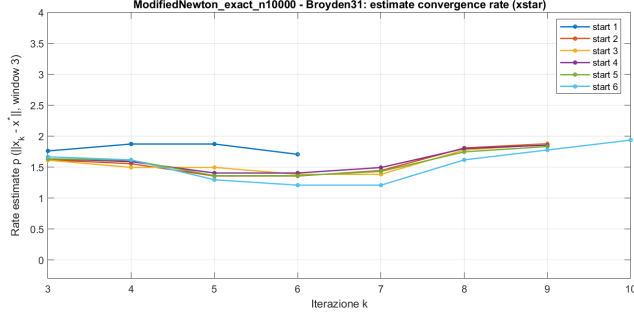
(g) Error from the minimizer, Case 2, $k = 12$, type h .



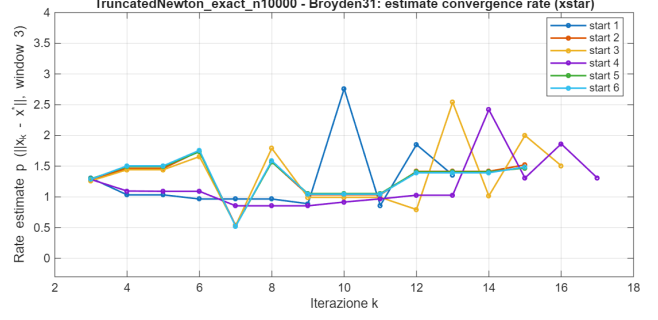
(h) Error from the minimizer, Case 2, $k = 12$, type h_i .

Figure 13: The first four plots correspond to Modified Newton and the last four to Truncated Newton. For Case 2 with $k = 12$, the error plots indicate that Modified Newton approaches stationary points different from the minimizer, while Truncated Newton exhibits a slow decay, suggesting that more iterations are required for convergence.

A.1.4 Rate $n=1e4$

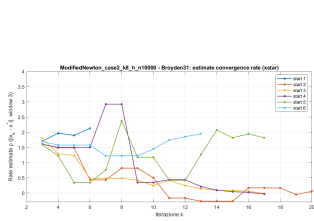


(a) Convergence rate for Modified Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$.

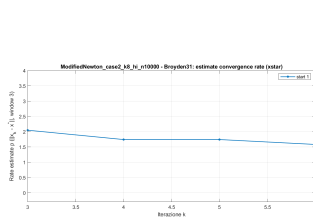


(b) Convergence rate for Truncated Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 for $k = 4$ and $k = 8$ with type h .

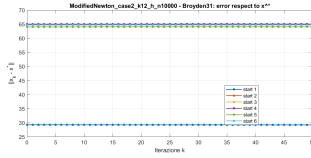
Figure 14: Broyden convergence rates for $n = 10^4$. The observed rates agree with the theoretical predictions: Modified Newton approaches 2, while Truncated Newton approaches 1.5. Case 2 with $k = 12$ does not converge within the prescribed number of iterations for either method.



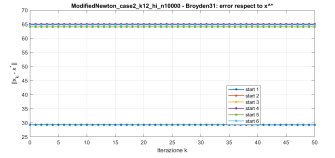
(a) Case 2, $k=8$, type= h .



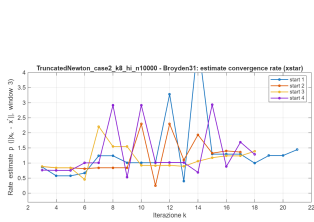
(b) Case 2, $k=8$, type= hi .



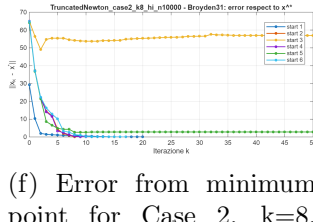
(c) Error from minimum point for Case 2, $k=12$, type= h .



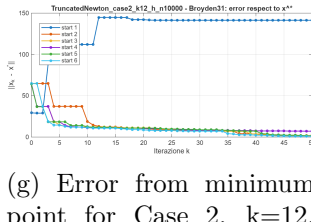
(d) Error from minimum point for Case 2, $k=12$, type= hi .



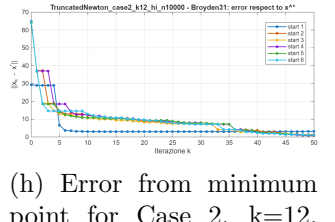
(e) Case 2, $k=8$, type= hi .



(f) Error from minimum point for Case 2, $k=8$, type= hi



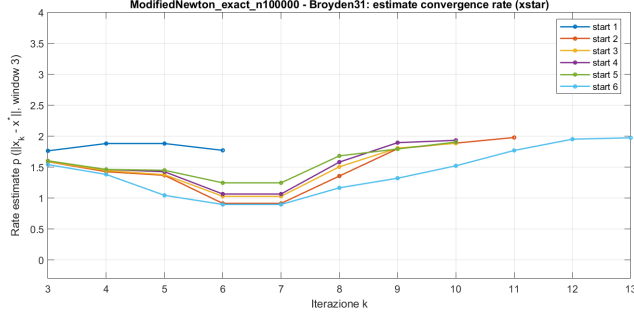
(g) Error from minimum point for Case 2, $k=12$, type= h



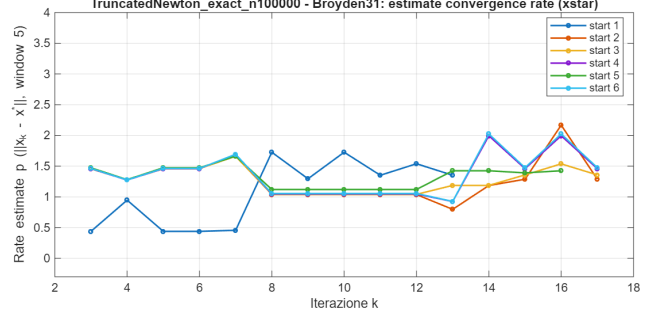
(h) Error from minimum point for Case 2, $k=12$, type= hi

Figure 15: Convergence and error-to-minimizer plots for FD Case 2 at $n = 10^4$. The first four plots correspond to Modified Newton and the last four to Truncated Newton. As seen at $n = 10^3$, Case 2 with $k = 12$, Modified Newton converges to a different stationary point from the theoretical minimizer. Truncated Newton exhibits a much slower decay of the error, reflecting the increased difficulty of the problem at this larger dimension.

A.1.5 Rate $n=1e5$

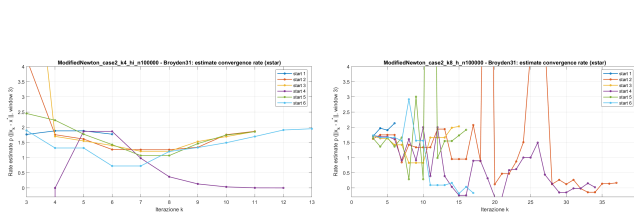


(a) Convergence rate for Modified Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$, type h .

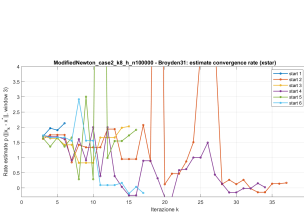


(b) Convergence rate for Truncated Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$.

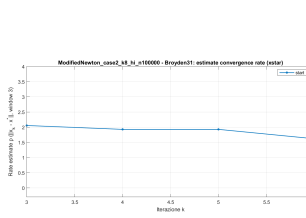
Figure 16: Broyden convergence rates for $n = 10^5$. The observed rates agree with the theoretical predictions: Modified Newton approaches 2, while Truncated Newton approaches 1.5. Case 2 with $k = 12$ does not converge within the prescribed number of iterations for either method.



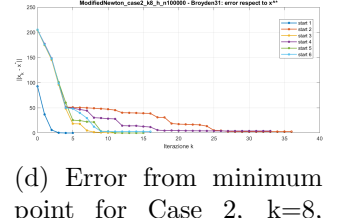
(a) Case 2, $k=4$, type= hi .



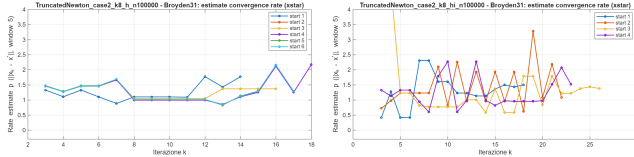
(b) Case 2, $k=8$, type= h .



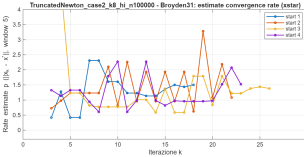
(c) Case 2, $k=8$, type= hi .



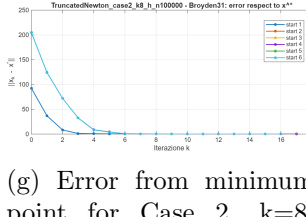
(d) Error from minimum point for Case 2, $k=8$, type= h .



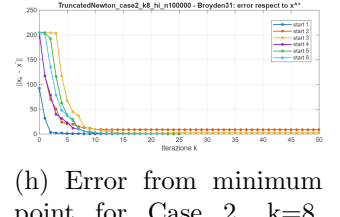
(e) Case 2, $k=8$, type= h .



(f) Case 2, $k=8$, type= hi .



(g) Error from minimum point for Case 2, $k=8$, type= h .

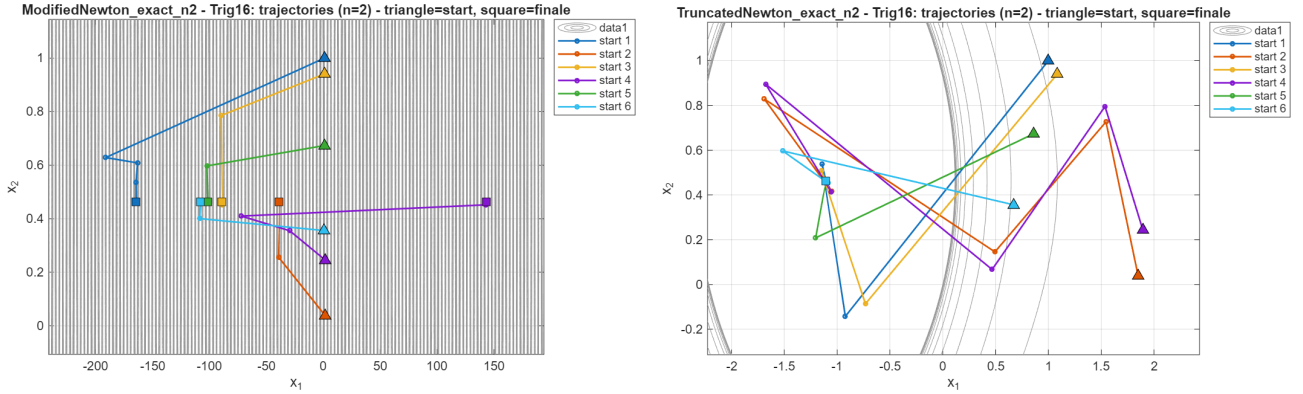


(h) Error from minimum point for Case 2, $k=8$, type= hi .

Figure 17: Convergence rate plots for FD Case 2 at $n = 10^5$. The first three plots correspond to Modified Newton and the last three to Truncated Newton. Consistently with the aggregate results reported above, only Case 2 with $k = 4$ and $k = 8$ produce valid runs at this dimension, while $k = 12$ fails to converge within the prescribed number of iterations for either method.

A.2 Trig16

A.2.1 Contour plot



(a) Modified Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$.

(b) Truncated Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$.

Figure 18: Trig16 trajectories on the contour plot for $n = 2$.

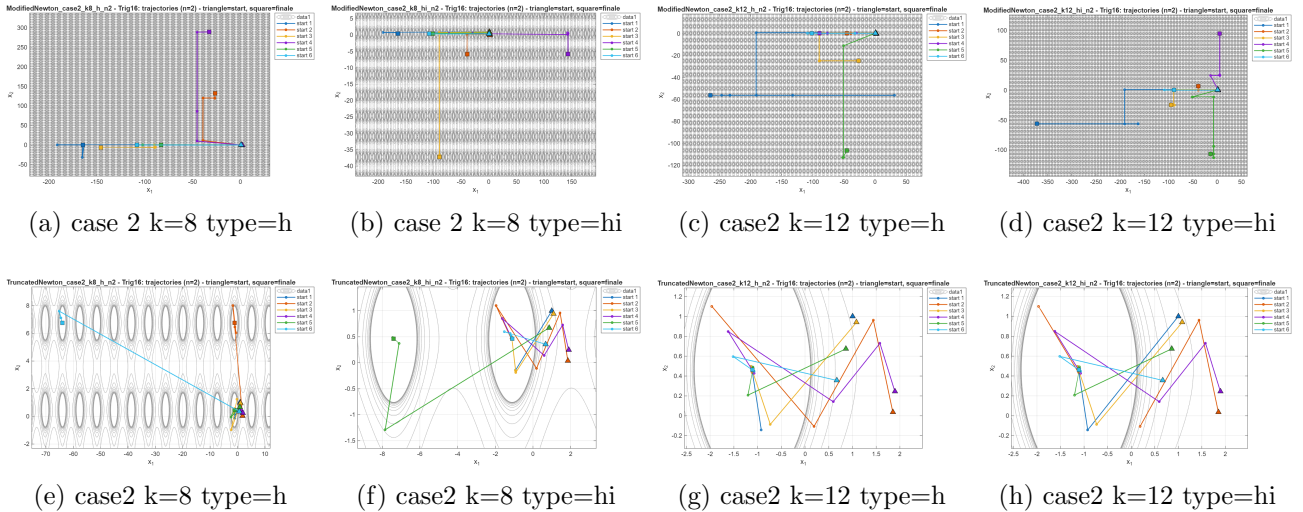
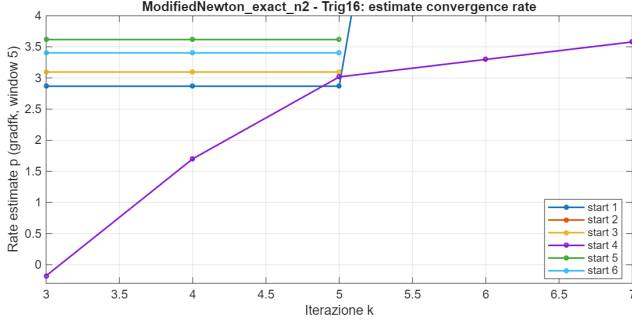
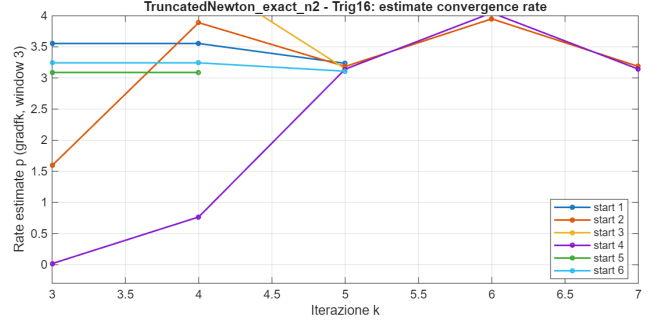


Figure 19: Contour plots for FD Case 2 for Modified Newton (top row) and Truncated Newton (bottom row). For Modified Newton, each run converges to an equivalent minimizer, highly sensitive to the initial conditions. For Case 2 with $k = 8$ and $k = 12$, both methods exhibit highly oscillatory and unexpected trajectories. Truncated Newton struggles to converge and shows severe zigzagging, due to its trajectory being strongly affected by the errors introduced by the finite-difference approximations. Although convergence is eventually achieved in the two-dimensional problem, the observed behaviour is substantially different from the expected one.

A.2.2 Rate $n=2$

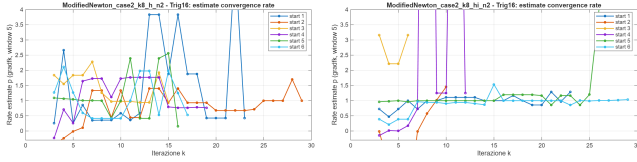


(a) Convergence rate for Modified Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$.

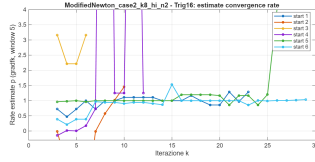


(b) Convergence rate for Truncated Newton with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$.

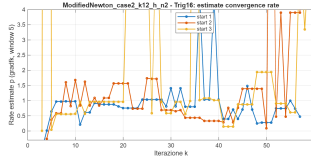
Figure 20: Trig16 convergence rates for $n = 2$. Because the algorithm converges in very few iterations, the rate estimates are highly unstable and noisy. In this low-dimensional setting, the estimated rates are highly oscillatory; because the algorithm stops so quickly, we cannot clearly separate the real convergence behavior from this numerical noise. For this reason, these plots should not be considered representative of how the algorithm actually converges.



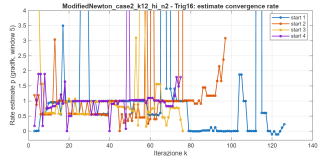
(a) Case 2, $k=8$, type= h .



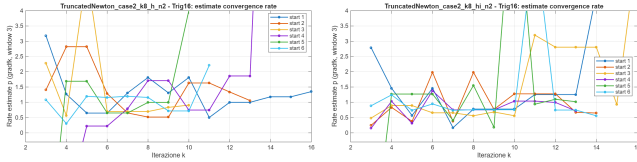
(b) Case 2, $k=8$, type= hi .



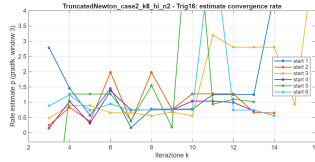
(c) Case 2, $k=12$, type= h .



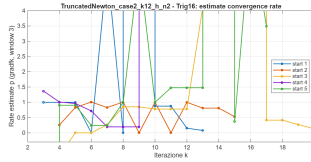
(d) Case 2, $k=12$, type= hi .



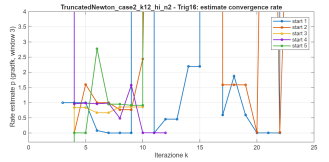
(e) Case 2, $k=8$, type= h .



(f) Case 2, $k=8$, type= hi .



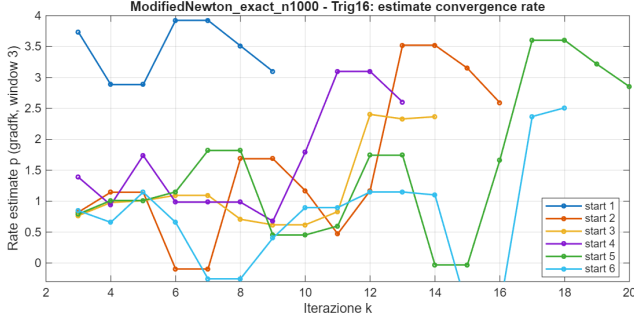
(g) Case 2, $k=12$, type= h .



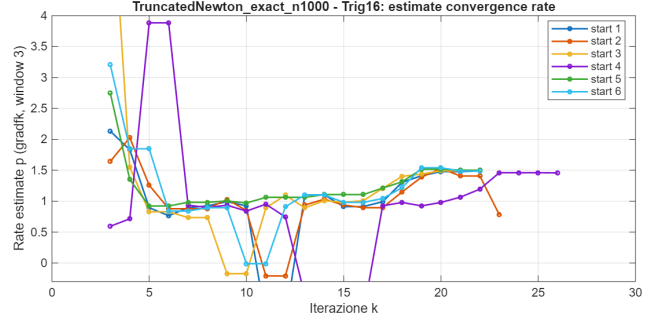
(h) Case 2, $k=12$, type= hi .

Figure 21: Convergence rates for FD Case 2 for Modified Newton (top row) and Truncated Newton (bottom row). The reported rates should be interpreted with caution: in this low-dimensional problem, the methods may still converge even when the finite-difference errors substantially affect the expected asymptotic behaviour.

A.2.3 Rate $n=1e3$

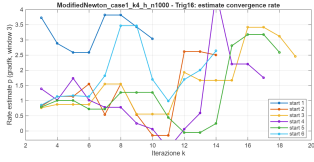


(a) Convergence rate for Modified Newton with exact derivatives

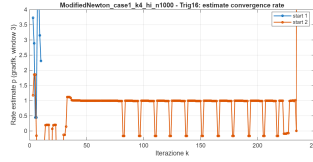


(b) Convergence rate for Truncated Newton with exact derivatives

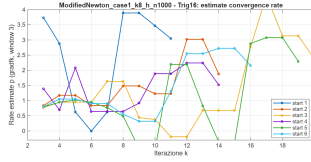
Figure 22: Trig16 convergence rates for $n = 10^3$. Different configurations produce very different convergence-rate profiles, showing that the algorithms are highly sensitive to finite-difference errors. This behavior happens because the problem becomes more ill-conditioned as the dimension grows, making the rate estimates less stable and making the overall convergence far more sensitive to the choice of finite-difference parameters.



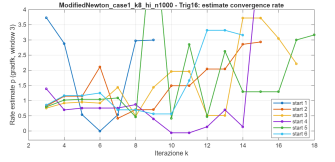
(a) Case 1, $k=4$, type=h.



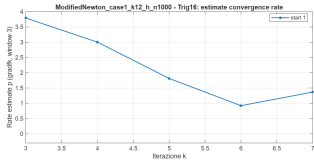
(b) Case 1, $k=4$, type=hi.



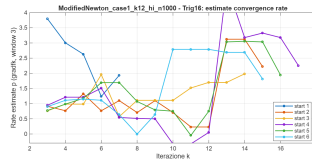
(c) Case 1, $k=8$, type=h.



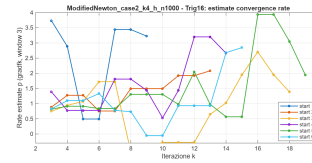
(d) Case 1, $k=8$, type=hi.



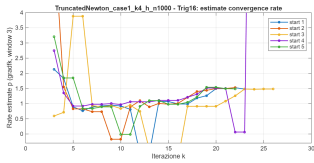
(e) Case 1, $k=12$, type=h.



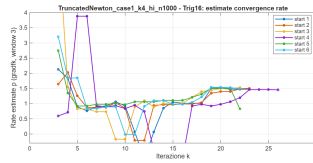
(f) Case 1, $k=12$, type=hi.



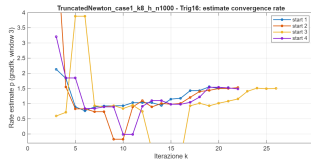
(g) Case 2, $k=4$, type=h.



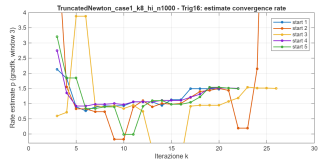
(h) Case 1, $k=4$, type=h.



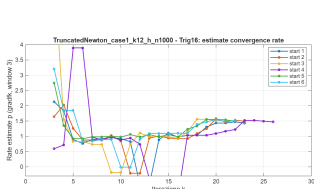
(i) Case 1, $k=4$, type=hi.



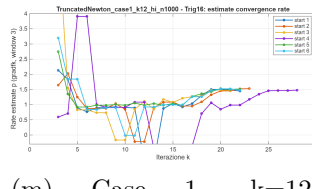
(j) Case 1, $k=8$, type=h.



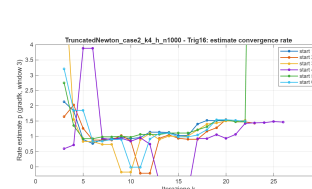
(k) Case 1, $k=8$, type=hi.



(l) Case 1, $k=12$, type=h.



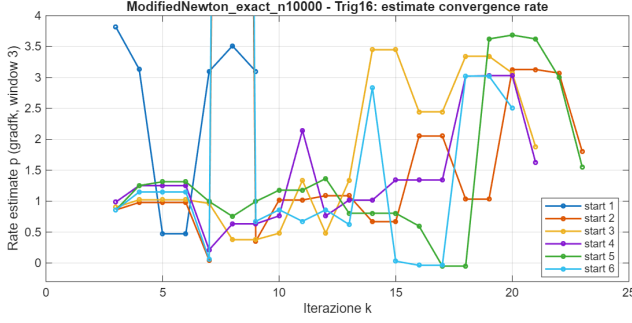
(m) Case 1, $k=12$, type=hi.



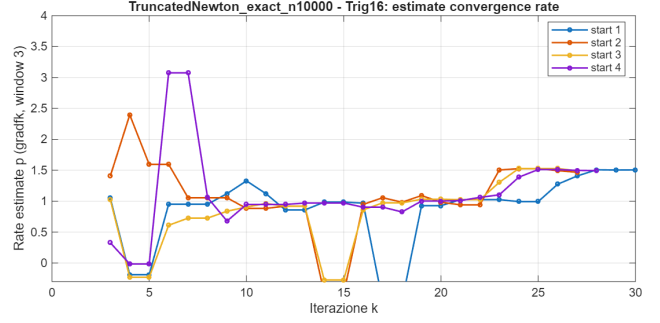
(n) Case 2, $k=4$, type=h.

Figure 23: Trig16 convergence rates for $n = 10^3$. For Modified Newton, a recurring pattern can be observed near the end of the runs: after a long phase of heavy oscillations, the rate seems to get close to quadratic convergence, but it never fully reaches it and keeps oscillating even in the final steps. Truncated Newton shows a more consistent behavior across different setups, but it spends many iterations in a linear phase before the rate finally gets close to the expected value of 1.5.

A.2.4 Rate $n=1e4$

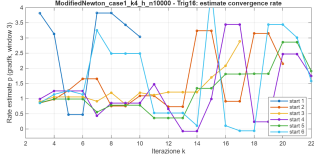


(a) Convergence rate for Modified Newton with exact derivatives

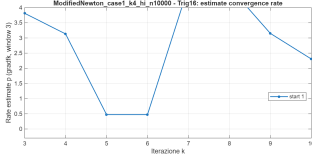


(b) Convergence rate for Truncated Newton with exact derivatives

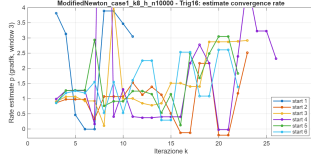
Figure 24: Trig16 convergence rates for $n = 10^4$. Both algorithms converge only up to Case 2 with $k = 4$, type h ; for the remaining configurations, they fail to converge within the allowed number of iterations. As seen before, the different configurations show very different convergence-rate profiles, proving that the methods are highly sensitive to finite-difference errors. This happens because the problem becomes increasingly ill-conditioned, which makes the convergence behaviour highly sensitive to the finite-difference parameters and prevents the algorithms from converging properly.



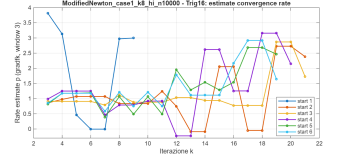
(a) Case 1, $k=4$, type= h .



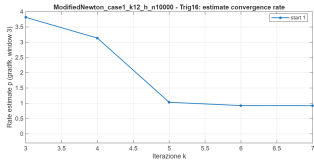
(b) Case 1, $k=4$, type= hi .



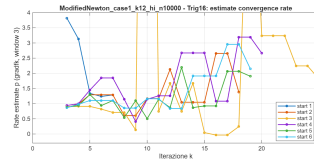
(c) Case 1, $k=8$, type= h .



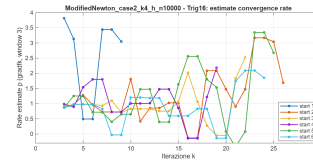
(d) Case 1, $k=8$, type= hi .



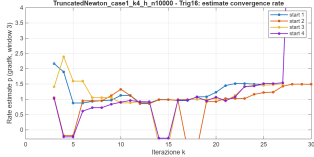
(e) Case 1, $k=12$, type= h .



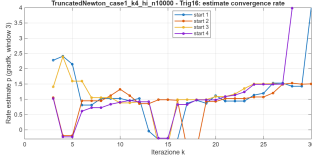
(f) Case 1, $k=12$, type= hi .



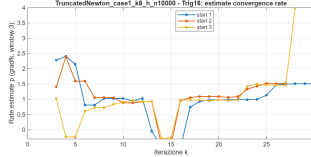
(g) Case 2, $k=4$, type= h .



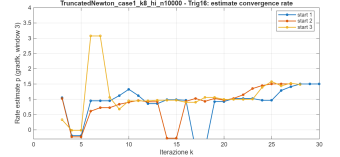
(h) Case 1, $k=4$, type= h .



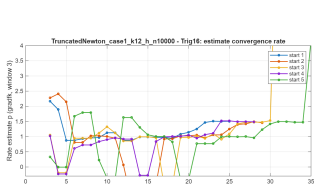
(i) Case 1, $k=4$, type= hi .



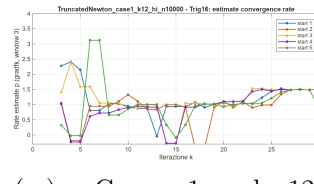
(j) Case 1, $k=8$, type= h .



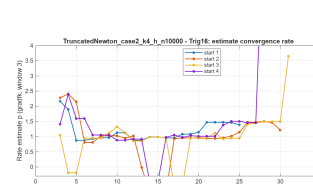
(k) Case 1, $k=8$, type= hi .



(l) Case 1, $k=12$, type= h .



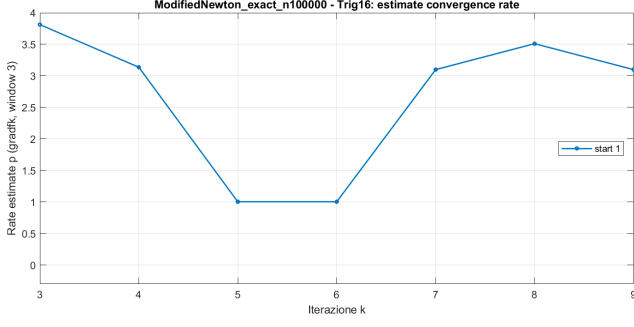
(m) Case 1, $k=12$, type= hi .



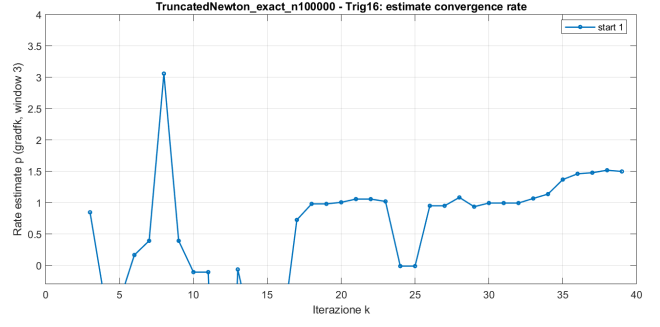
(n) Case 2, $k=4$, type= h .

Figure 25: Trig16 convergence rates for $n = 10^4$. The observed rates are strongly affected by the problem ill-conditioning, finite-difference errors, and the rate-estimation procedure. Modified Newton remains highly oscillatory, with only the last iterations showing a trend toward quadratic convergence only in the very last iterations. Similarly, Truncated Newton shows an unstable initial phase followed by a mostly linear regime, getting close to the expected value of 1.5 only at the end.

A.2.5 Rate $n=1e5$

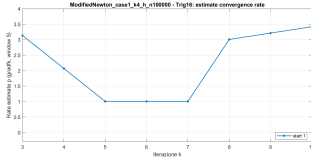


(a) Convergence rate for Modified Newton with exact derivatives

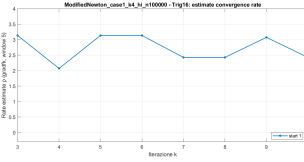


(b) Convergence rate for Truncated Newton with exact derivatives

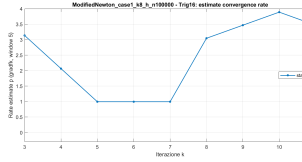
Figure 26: Trig16 convergence rates for $n = 10^5$. Convergence is extremely difficult for this problem. Even with exact derivatives, both Modified and Truncated Newton converge only in a single run starting from a favorable initial point. A few scattered runs converge under other Case 1 configurations, but most fail to converge within the allowed number of iterations, showing just how challenging this problem is.



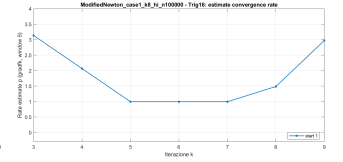
(a) Case 1, $k=4$, type=h.



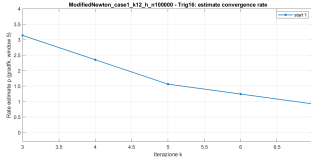
(b) Case 1, $k=4$, type=hi.



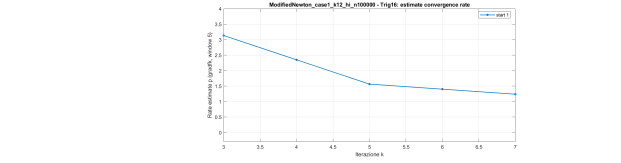
(c) Case 1, $k=8$, type=h.



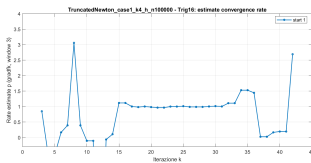
(d) Case 1, $k=8$, type=hi.



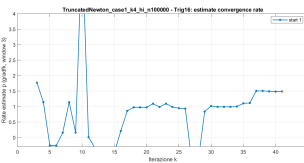
(e) Case 1, $k=12$, type=h.



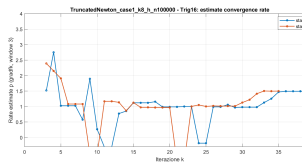
(f) Case 1, $k=12$, type=hi.



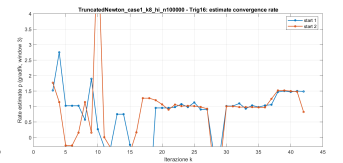
(g) Case 1, $k=4$, type=h.



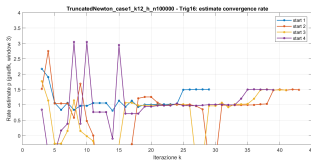
(h) Case 1, $k=4$, type=hi.



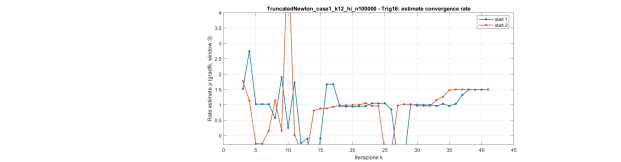
(i) Case 1, $k=8$, type=h.



(j) Case 1, $k=8$, type=hi.



(k) Case 1, $k=12$, type=h.



(l) Case 1, $k=12$, type=hi.

Figure 27: Trig16 convergence rates for $n = 10^5$. These plots are largely uninformative, as convergence is a rare event in this setting. When convergence occurs, it is typically very rapid, but only because the initial point is particularly favourable. For this reason, the reported rates should be interpreted with caution, as they do not show how the algorithms behave in general.

B Further Implementation

In this section, we present the numerical results obtained with two modifications of the `modified_newton` algorithm designed to improve its behaviour on large-scale instances of the Trig16 problem.

The first modification changes how the parameter β is used, turning it into a mechanism that acts

selectively on the unfavorable part of the eigenvalue spectrum. The second modification uses a different strategy for handling negative eigenvalues: it flips their sign directly, which enforces positive definiteness of the matrix at every iteration (cfr.2.2).

B.1 Comparison between the flipped strategy and the β -modified approach ($\beta = 1$) in the Trig16 framework

n	dm	$\beta = 0.01$			$\beta = 1$			flip		
		succ%	iter	rate	succ%	iter	rate	succ%	iter	rate
2	exact	100.0	4.2	2.96	100.0	4.5	2.94	100.0	5.7	2.97
2	case1	100.0	4.3	2.09	100.0	4.6	2.48	100.0	5.8	1.75
2	case2	86.1	50.4	1.53	94.4	23.0	1.81	83.3	56.5	1.55
10^3	exact	100.0	14.2	2.00	100.0	12.2	1.91	100.0	20.3	1.70
10^3	case1	72.2	59.8	1.29	100.0	12.3	1.86	100.0	23.8	1.72
10^3	case2	16.7	158.2	1.10	16.7	140.1	0.79	16.7	23.1	1.19
10^4	exact	100.0	17.8	1.85	100.0	16.7	2.40	100.0	25.7	2.05
10^4	case1	72.2	67.8	1.38	97.2	23.4	1.61	88.9	47.8	1.42
10^4	case2	11.1	165.5	0.91	16.7	164.2	1.07	13.9	14.2	0.75
10^5	exact	16.7	168.0	—	83.3	55.7	1.32	83.3	59.7	1.29
10^5	case1	16.7	168.0	1.06	58.3	93.1	1.27	38.9	77.2	1.24
10^5	case2	0.0	188.0	0.72	0.0	200.0	0.92	0.0	34.5	0.53

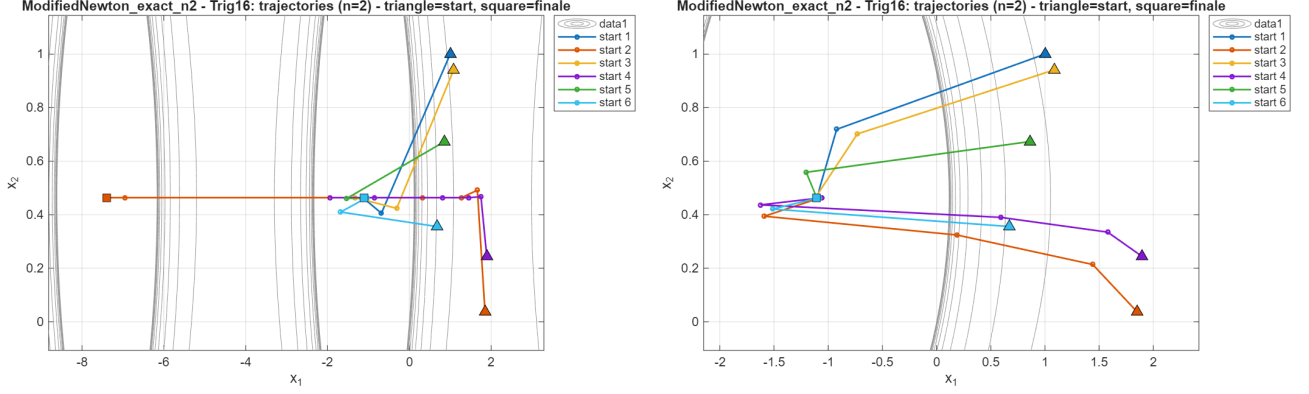
Table 6: Modified Newton on Trig16: comparison of the correction $B_k = \nabla^2 f(x_k) + \tau_k I$ under a minimal shift ($\beta = 0.01$) versus an aggressive shift ($\beta = 1$), against the eigenvalue-flipping variant. succ%: fraction of the 6 (exact) or 36 (case1/case2, pooled over $k \in \{4, 8, 12\}$ and both step types) starting-point runs that satisfy the gradient tolerance; iter: average outer iterations over converged runs; rate: average of `estimate_rate` over runs where it returns a valid value . *Note*: at $n = 10^5$, exact, $\beta = 1$ regains success where $\beta = 0.01$ collapses (83.3% vs. 16.7%).

The role of β in the correction $B_k = \nabla^2 f(x_k) + \tau_k I$ is purely a numerical safeguard: any $\beta > 0$ yields a positive-definite B_k , and the standard choice is to keep it as small as numerically safe (typically $O(10^{-3})$, $O(10^{-2})$), so that the correction perturbs the actual curvature as little as possible when the Hessian is already close to positive definite. Following this standard choice, $\beta = 0.01$, which is the value used in our main experiments, is a reasonable, literature-consistent default, chosen before observing the problematic Trig16 function.

Table 6 shows that this default represents a poor fit for Trig16: $\beta = 1$, two orders of magnitude larger than what the standard prescription would suggest, is never worse than $\beta = 0.01$ in success rate, and is much better at $n = 10^5$ (83.3% vs. 16.7% under exact derivatives). This does not mean that $\beta = 1$ is generally a better choice; it is a symptom of Trig16’s specific ill-conditioning. A minimal β -correction keeps $\lambda_{\min}(B_k)$ close to zero, compared to the rest of the spectrum. This is exactly the condition that causes the first Newton step to be amplified (Section X): the ”gentle” correction recommended by the standard choice is exactly what fails to control this amplification here. A larger $\beta = 1$ avoids this problem, because it dominates the correction over the problematic part of the spectrum, but it also loses most of the real curvature information in the process.

So one should not read this result as ” $\beta = 1$ is the right choice”. Instead, it shows that a single, uniform shift is simply the wrong tool for a Hessian whose eigenvalues are both very negative and very large at the same time: a scalar τ_k cannot control the smallest eigenvalue without a trade-off: if it is too small, it has no real effect (like $\beta = 0.01$ here); if it is too large, it loses curvature information (like $\beta = 1$). This is why we also test the eigenvalue-flipping variant in Table 6: it corrects each eigenvalue on its own, instead of applying the same shift to all of them. This variant is not always the best choice either, but by construction it avoids this specific problem.

B.1.1 Contour plot



(a) Modified Flipped with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$.

(b) Modified $\beta=1$ with exact derivatives. The same behaviour is observed for all runs in Case 1 ($k = 4, 8, 12$, both types h and hi) and in Case 2 with $k = 4$.

Figure 28: Trig16 trajectories on the contour plot for $n = 2$.

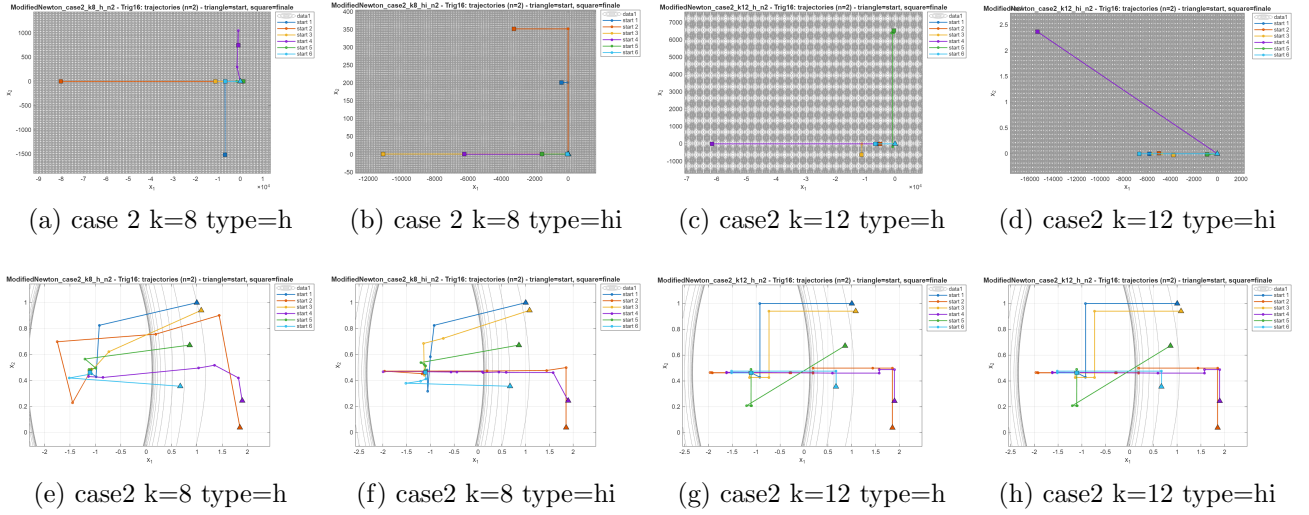
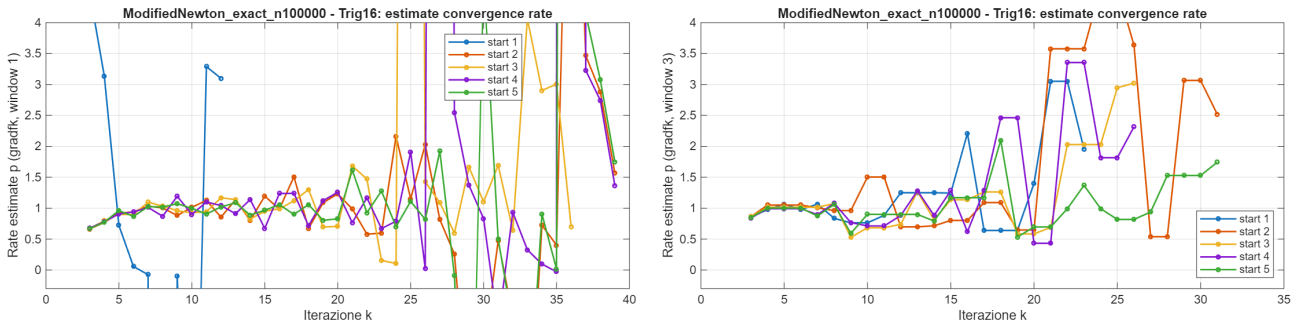


Figure 29: Contour plots for FD Case 2 for Modified Flipped (top row) and Modified $\beta=1$ (bottom row).

B.1.2 Rate $n=1e5$



(a) Convergence rate for Modified Flipped with exact derivatives

(b) Convergence rate for Modified $\beta=1$ with exact derivatives

Figure 30: Trig16 convergence rates for $n = 10^5$ with exact derivatives, comparing the Modified Flipped variant (left) against Modified Newton with $\beta = 1$ (right). Both variants recover convergence far more often than the standard $\beta = 0.01$ correction, in line with Table 6, though the underlying difficulty of the problem at this dimension still limits the number of successful runs.

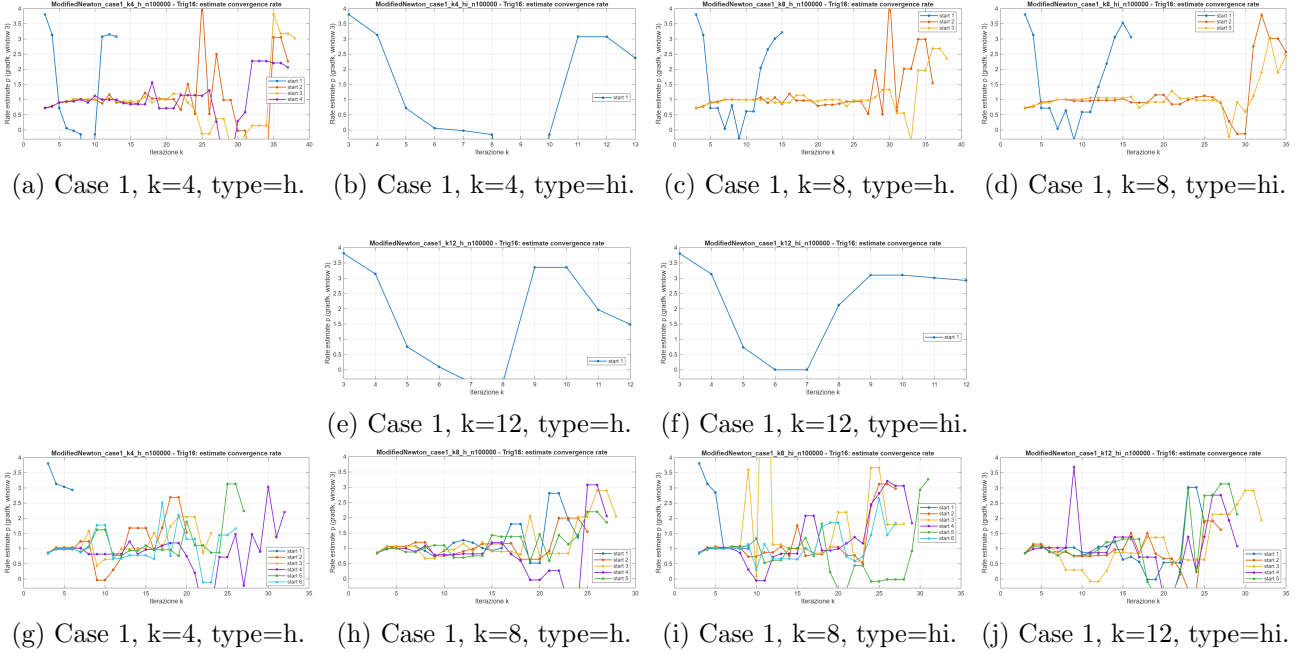


Figure 31: Trigl6 convergence rates for $n = 10^5$. Top two rows Modified Flipped, Bottom row Modified $\beta=1$.

B.2 Truncated Hess-Free

k	matrix-free (truncated_hess_free)				explicit-Hessian FD			
	succ%	iter	rate	time (s)	succ%	iter	rate	time (s)
$n = 10^4$, Case 1								
4	100.0	14.2	1.507	0.04	100.0	14	1.507	1.32
8	100.0	14.2	1.507	0.04	100.0	14	1.507	1.31
12	100.0	14.2	1.378	0.03	100.0	14	1.507	1.02
$n = 10^4$, Case 2								
4	100.0	14.2	1.507	4.10	100.0	14	1.507	8.66
8	33.3	32.5	0.920	76.14	100.0	14	1.515	8.73
12	0.0	—	—	16.89	0.0	—	—	30.10
$n = 10^5$, Case 1								
4	100.0	15.2	1.502	0.46	100.0	15	1.502	111.43
8	100.0	15.2	1.503	0.51	100.0	15	1.502	110.11
12	100.0	15.8	1.283	0.58	100.0	15	1.502	109.09
$n = 10^5$, Case 2								
4	100.0	15.2	1.502	401.49	100.0	15	1.502	716.88
8	50.0 [†]	33.0 [†]	1.260 [†]	19698.87 [†]	100.0	16	1.483	1066.90
12	— [‡]	—	—	— [‡]	0.0	—	—	2473.55

Table 7: Truncated Newton on Broyden31(constant step h): matrix-free Hessian-vector product vs. explicit FD Hessian, at $n = 10^4$ and $n = 10^5$. At $k = 4$, both variants converge for all starting points with approximately matched accuracy (rate, gradient norm), but the matrix-free product is 2–1500 $\times$ cheaper depending on (n, case) . At $k \geq 8$, in Case2, finite difference noise influences the CG curvature test, causing the matrix-free speed advantage to fall. *NOTE*:for $n=1e5$, the matrix-free variant has missing data due to long runtimes: only 2 of 6 starting points finished at $k = 8$ (one run was cut off after 10.5 hours), and $k = 12$ is missing entirely.

Scaling with n . When the curvature test works properly (Case 1), the matrix-free method gets faster as problem size grows—jumping from 30 $\times$ faster at $n = 10^4$ to 240 $\times$ faster at $n = 10^5$. However, in Case 2 at $k = 8$, it breaks down: explicit hessian’s runtime goes from 8.7s to 1,067s, matrix-free’s

runtime explodes from 76s to over 20,000s. Why does matrix-free fail? We suppose that it it because finite-difference noise gets recalculated at every inner CG step, confusing the algorithm's stopping test and forcing it into maximum iterations while in the standard version the noise is calculated only once per outer step, keeping the stopping test reliable and avoiding useless iterations.

C Code

```
1 function [f,gradf,hessf,xbar,xstarfun] = problem_trig16()
2 % Problem 16: Banded Trigonometric
3 %
4 %  $F(x) = \sum_{i=1}^n i * (1 - \cos(x_i))$ 
5 %  $+ 2 * \sum_{i=1}^{n-1} \sin(x_i)$ 
6 %  $- (n - 1) * \sin(x_n)$ 
7 %
8 % OUTPUT:
9 % f : F(x)
10 % gradf : gradient of F
11 % hessf : Hessian matrix of F (diagonal)
12 % xbar : starting point
13 % xstarfun : handle xstarfun(n) -> closest minimum point to x*
14 %
15 % The function is periodic in each component (a sum of sines and cosines),
16 % so it has many stationary points: minimums, saddles, and maximums.
17 % Consequently, it might converges to different stationary points, depending
18 % on
19 % the starting point.
20 f = @Ffun;
21 gradf = @gfun;
22 hessf = @Hfun;
23
24 xbar = @(n) ones(n,1);
25
26 % Computes the scalar objective function value
27 function Fx = Ffun(x)
28     n = length(x);
29     i = (1:n)';
30     Fx = sum( i .* (1 - cos(x)) ) ...
31         + 2 * sum( sin(x(1:n-1)) ) ...
32         - (n-1) * sin(x(n));
33 end
34
35 % Computes the analytical gradient vector
36 function g = gfun(x)
37     n = length(x);
38     i = (1:n)';
39     g = zeros(n,1);
40     g(1:n-1) = i(1:n-1) .* sin(x(1:n-1)) + 2*cos(x(1:n-1));
41     g(n) = n*sin(x(n)) - (n-1)*cos(x(n));
42 end
43
44 % Computes the exact Hessian matrix.
45 % The matrix is diagonal, so it is built as a sparse matrix for
46 % efficiency.
47 function H = Hfun(x)
48     n = length(x);
49     i = (1:n)';
50     d = zeros(n,1);
51     d(1:n-1) = i(1:n-1) .* cos(x(1:n-1)) - 2*sin(x(1:n-1));
52     d(n) = n*cos(x(n)) + (n-1)*sin(x(n));
53     H = spdiags(d, 0, n, n);
54 end
55 end
```

Listing 1: Problem 16: Banded Trigonometric

```

1 function [f,gradf,hessf,xbar,rfun,xstarfun] = problem_broyden31()
2 % Problem 31: Broyden tridiagonal least-squares problem
3 %
4 % OUTPUT:
5 %   f           : F(x) = 0.5*||r(x)||^2
6 %   gradf        : gradient of F
7 %   hessf         : Hessian of F
8 %   xbar          : starting point
9 %   rfun          : residuals vector
10 %   xstarfun      : handle xstarfun(n) -> global minimum x*
11 %
12 % The solution x* is computed using LSQNONLIN by solving
13 %           min_x 0.5*||r(x)||^2
14 % by explicitly providing the sparse Jacobian of the residual.
15 % Since F(x) >= 0 for all x, if ||r(x*)|| is approximately 0, then F(x*)
16 % is approximately 0 and therefore x* is a global minimum.
17 % The result is cached: repeated calls with the same n
18 % do not re-run lsqnonlin.
19
20 f      = @Ffun;
21 gradf  = @gfun;
22 hessf  = @Hfun;
23 rfun   = @rvec;
24
25 xbar = @(n) -ones(n,1);
26 xstarfun = @xstar_cached;
27
28 % Computes the residual vector r(x)
29 function r = rvec(x)
30     n = length(x);
31     xm1 = [0; x(1:n-1)];
32     xp1 = [x(2:n); 0];
33     r = (3 - 2*x).*x - xm1 - 2*xp1 + 1;
34 end
35
36 % Computes the scalar objective function value F(x) = 0.5 * ||r(x)||^2
37 function Fx = Ffun(x)
38     r = rvec(x);
39     Fx = 0.5 * (r' * r);
40 end
41
42 % Computes the analytical gradient using the sparse Jacobian of the
43 % residual
44 function g = gfun(x)
45     n = length(x);
46     r = rvec(x);
47     d = 3 - 4*x;
48     J = spdiags( ...
49         [-ones(n,1), d, -2*ones(n,1)], ...
50         [-1,0,1], n, n);
51     g = J' * r;
52 end
53
54 % Computes the exact Hessian matrix using the sparse Jacobian
55 function H = Hfun(x)
56     n = length(x);
57     r = rvec(x);
58     d = 3 - 4*x;
59     J = spdiags( ...
60         [-ones(n,1), d, -2*ones(n,1)], ...

```

```

60         [-1,0,1], n, n);
61     H = J' * J - 4 * spdiags(r,0,n,n);
62 end
63
64
65 % Computes the global solution using LSQNONLIN
66 function xs = xstar_cached(n)
67     persistent cache_n cache_x
68
69     % Return the cached result if n has not changed
70     if ~isempty(cache_n) && cache_n == n
71         xs = cache_x;
72         return;
73     end
74
75     disp('Broyden31: computing global solution...');
76
77     % Starting point
78     x0 = xbar(n);
79
80     % Options for LSQNONLIN
81     opts = optimoptions('lsqnonlin', ...
82         'Algorithm', 'trust-region-reflective', ...
83         'SpecifyObjectiveGradient', true, ...
84         'Display', 'iter', ...
85         'FunctionTolerance', 1e-14, ...
86         'StepTolerance', 1e-14, ...
87         'OptimalityTolerance', 1e-12, ...
88         'MaxIterations', 2000, ...
89         'MaxFunctionEvaluations', 1e6);
90
91     % Solve nonlinear least squares problem
92     [xs,resnorm,residual,exitflag,output] = ...
93         lsqnonlin(@residual_J,x0,[],[],opts);
94
95     % Solution verification metrics
96     Gstar = 0.5 * resnorm;
97     rinf = norm(residual,inf);
98     r2    = norm(residual,2);
99
100    % Global minimum check
101    %  $G(x) = 0.5 * ||r(x)||^2 \geq 0$  for all  $x$ .
102    % If the residual is numerically zero, then  $G(x^*) = 0$  and  $x^*$  is a
        global minimum.
103    if rinf < 1e-10
104        disp('Reference solution found. Residual is numerically zero, x*
            is a global minimum.');
```

```

118 % The Jacobian is tridiagonal and sparse.
119 function [r, J] = residual_J(x)
120     n = length(x);
121
122     % Compute residual vector
123     xm1 = [0; x(1:n-1)];
124     xp1 = [x(2:n); 0];
125     r = (3 - 2*x).*x - xm1 - 2*xp1 + 1;
126
127     % Compute sparse Jacobian matrix
128     d = 3 - 4*x;
129     J = spdiags( ...
130         [-ones(n,1), d, -2*ones(n,1)], ...
131         [-1, 0, 1], n, n);
132 end
133
134 end

```

Listing 2: Problem 31: Broyden Tridiagonal Least Squares

```

1 function hvec = steps_fd(x, k, type)
2 % STEPS_FD Computes the finite difference step size vector.
3 %
4 % INPUTS:
5 %   x      : current point
6 %   k      : exponent used to define the step size
7 %   type   : 1 -> constant step, h_i = 10^(-k)
8 %           2 -> relative step, h_i = 10^(-k) * |x_i|
9 %
10 % OUTPUT:
11 %   hvec : vector containing the step sizes
12
13 x = x(:);
14 h = 10^(-k);
15
16 if type == 1
17     % Constant step size for all components
18     hvec = h * ones(length(x), 1);
19 else
20     % Relative step size
21     hvec = h * abs(x);
22     % Use h when the corresponding component of x is zero
23     hvec(hvec == 0) = h;
24     % Avoid excessively small step sizes
25     hvec(hvec < 1e-14) = 1e-14;
26 end
27
28 end

```

Listing 3: Function for computing the finite difference step size vector

```

1 function H = trig_hess_fd_case1(gradf, x, k, mode)
2 % TRIG_HESS_FD_CASE1 Finite-difference Hessian for Problem 16 (Case 1).
3 % Approximates the diagonal Hessian via a centered finite difference of
4 % the gradient (exact or FD).
5 %
6 %
7 % INPUT ARGUMENTS:
8 %   gradf : function handle for the gradient (exact or approximated)
9 %   x      : current point
10 %   k      : power for the increment (e.g., 4, 8, 12)

```

```

11 % mode : 1 for constant step h, 2 for relative step hi
12 %
13 % OUTPUT ARGUMENT:
14 % H : approximated diagonal Hessian (sparse)
15
16 x = x(:);
17 n = length(x);
18
19 h = steps_fd(x, k, mode);
20
21 gp = gradf(x + h);
22 gm = gradf(x - h);
23
24 d = (gp - gm) ./ (2*h); % d_i approximates dg_i/dx_i = H_ii
25
26 H = spdiags(d, 0, n, n);
27 end

```

Listing 4: Finite-difference Hessian for Problem 16 (Case 1)

```

1 function [g, H] = trig_fd_case2(x, k, mode)
2 % TRIG_FD_CASE2 Finite-difference gradient and Hessian for Problem 16
3 % (Case 2: both gradient and Hessian approximated).
4 %
5 % INPUT ARGUMENTS:
6 % x : current point
7 % k : power for the increment (e.g., 4, 8, 12)
8 % mode : 1 for constant step h, 2 for relative step hi
9 %
10 % OUTPUT ARGUMENTS:
11 % g : approximated gradient
12 % H : approximated diagonal Hessian (sparse)
13
14 x = x(:);
15 n = length(x);
16 h = steps_fd(x, k, mode);
17
18 phi0 = trig16_local_terms(x, n);
19 phip = trig16_local_terms(x + h, n);
20 phim = trig16_local_terms(x - h, n);
21
22 g = (phip - phim) ./ (2*h); % g_i = phi_i'(x_i)
23 d = (phip - 2*phi0 + phim) ./ (h.^2); % d_i = phi_i''(x_i)
24
25 H = spdiags(d, 0, n, n);
26 end
27
28 function phi = trig16_local_terms(x, n)
29 % Returns the vector [phi_1(x_1), ..., phi_n(x_n)], the single-variable
30 % decomposition of F(x) used by TRIG_FD_CASE2.
31 i = (1:n)';
32 c = 2*ones(n,1);
33 c(n) = -(n-1);
34 phi = i.*(1-cos(x)) + c.*sin(x);
35 end

```

Listing 5: Finite-difference gradient and Hessian for Problem 16 (Case 2)

```

1 function g = trig_fd_case2_grad_only(x, k, mode)
2 % TRIG_FD_CASE2_GRAD_ONLY Wrapper extracting only the gradient from
3 % TRIG_FD_CASE2, for use as a single-output function handle gradf(x).

```

```

4     [g, ~] = trig_fd_case2(x, k, mode);
5 end

```

Listing 6: Gradient-only wrapper for Problem 16 Case 2

```

1 function H = trig_fd_case2_hess_only(x, k, mode)
2 % TRIG_FD_CASE2_HESS_ONLY Wrapper extracting only the Hessian from
3 % TRIG_FD_CASE2, for use as a single-output function handle hessf(x).
4     [~, H] = trig_fd_case2(x, k, mode);
5 end

```

Listing 7: Hessian-only wrapper for Problem 16 Case 2

```

1 function g = grad_fd_broyden31(x, k, type, rfun)
2 % GRAD_FD_BROYDEN31 Computes the FD gradient for Problem 31.
3 % It exploits the tridiagonal structure of the Jacobian, computing
4 % it with centered finite differences (3-coloring approach).
5 %
6 % INPUTS:
7 %   x      : current point
8 %   k      : power for the increment (e.g., 4, 8, 12)
9 %   type   : 1 for constant step, 2 for relative step
10 %   rfun   : handle to the residual function rvec(x)
11 %
12 % OUTPUT:
13 %   g      : approximated gradient
14
15 n = length(x);
16
17 % Compute the step size
18 hvec = steps_fd(x, k, type);
19
20 % Preallocate sparse Jacobian (tridiagonal -> max 3*n non-zeros)
21 J = spalloc(n, n, 3*n);
22
23 % 3-coloring approach -> variables spaced by 3 do not overlap
24 for group = 1:3
25     d = zeros(n, 1);
26     indices = group:3:n;
27     d(indices) = hvec(indices); % Apply perturbation only to the current
        group
28
29     % Evaluate residuals at the perturbed x
30     r_plus = rfun(x + d);
31     r_minus = rfun(x-d);
32
33     % Compute the finite difference for the residuals
34     delta_r = r_plus - r_minus;
35
36     % Building the Jacobian
37     for j = indices
38         row_start = max(1, j-1);
39         row_end = min(n, j+1);
40
41         J(row_start:row_end, j) = delta_r(row_start:row_end) / (2*hvec(j));
42     end
43 end
44
45 % Compute the final approximated gradient: g = J' * r
46 r_x = rfun(x);
47 g = J' * r_x;

```

48 `end`

Listing 8: Finite-difference gradient for Problem 31

```
1 function H = hess_fd_broyden31(gfun, x, k, type)
2 % HESS_FD_BROYDEN31 computes the finite difference Hessian for problem 31.
3 % It exploits the pentadiagonal structure of the Hessian, applying a
4 % 5-colouring technique with centered finite differences.
5 %
6 % INPUTS:
7 %   x      : current point
8 %   k      : power for the increment (e.g., 4, 8, 12)
9 %   type   : 1 for constant step, 2 for relative step
10 %   gfun   : handle to the gradient function (exact or approximated)
11 %
12 % OUTPUT:
13 %   H      : approximated Hessian matrix
14
15 n = length(x);
16
17 % Compute step sizes
18 hvec = steps_fd(x, k, type);
19
20 % Preallocate sparse Hessian (pentadiagonal -> max 5*n non-zeros)
21 H = spalloc(n, n, 5*n);
22
23 % 5-coloring approach: variables spaced by 5 do not overlap
24 for group = 1:5
25     d = zeros(n, 1);
26     indices = group:5:n;
27
28     % Apply perturbation only to the current group
29     d(indices) = hvec(indices);
30
31     % Evaluate gradient at the perturbed x
32     g_plus = gfun(x + d);
33     g_minus = gfun(x - d);
34
35     % Compute the finite difference for the gradient
36     delta_g = g_plus - g_minus;
37
38     % Building the Hessian matrix
39     for j = indices
40         % The non-zero elements are only from row j-2 to j+2
41         row_start = max(1, j-2);
42         row_end = min(n, j+2);
43
44         H(row_start:row_end, j) = ...
45             delta_g(row_start:row_end) / (2*hvec(j));
46     end
47 end
48
49 % Taking the symmetric to correct numerical errors
50 H = (H + H') / 2;
51 end
```

Listing 9: Finite-difference Hessian for Problem 31

```
1 function H = hess_grad_fd_broyden31(x, k, type, rfun)
2 % HESS_GRAD_FD_BROYDEN31 Computes the FD Hessian using the FD gradient.
3 % It calculates the Hessian for Problem 31 applying a 5-coloring technique,
```



```

4 % using a 3-coloring technique on the gradient.
5 %
6 % INPUTS:
7 %   x      : current point
8 %   k      : power for the increment (e.g., 4, 8, 12)
9 %   type   : 1 for constant step, 2 for relative step
10 %   rfun   : handle to the exact residual function rvec(x)
11 %
12 % OUTPUT:
13 %   H      : fully approximated Hessian matrix
14
15 n = length(x);
16
17 % Handle function for the approximated gradient
18 gfun_fd = @(y) grad_fd_broyden31(y, k, type, rfun);
19
20 % Compute step sizes
21 hvec = steps_fd(x, k, type);
22
23 % Preallocate sparse Hessian
24 H = spalloc(n, n, 5*n);
25
26 % 5-coloring approach for the Hessian
27 for group = 1:5
28     d = zeros(n, 1);
29     indices = group:5:n;
30
31     % Apply perturbation to the current group
32     d(indices) = hvec(indices);
33
34     % Evaluate FD gradient at the perturbed points
35     g_plus = gfun_fd(x + d);
36     g_minus = gfun_fd(x - d);
37
38     % Compute the centered finite difference for the gradient
39     delta_g = g_plus - g_minus;
40
41     % Building the Hessian matrix
42     for j = indices
43         row_start = max(1, j-2);
44         row_end = min(n, j+2);
45
46         H(row_start:row_end, j) = ...
47             delta_g(row_start:row_end) / (2*hvec(j));
48     end
49 end
50
51 % Taking the symmetric to correct numerical errors
52 H = (H + H') / 2;
53 end

```

Listing 10: Finite-difference Hessian using the finite-difference gradient for Problem 31

```

1 function [rate_seq, k_centers] = estimate_rate_polyfit(err, window, min_err)
2 % ESTIMATE_RATE_POLYFIT Estimates the convergence rate p such that e_{k+1}
   ~ C * e_k^p using linear regression (polyfit) on a sliding window of
   consecutive points.
3 % In log-log scale: log(e_{k+1}) = log(C) + p * log(e_k).
4 % The slope of the fitted line is the estimated rate p for that window.
5 %
6 % INPUT

```

```

7 %   err      : sequence of errors ((e.g. ||x_k - x*|| or
8 %             ||grad f(x_k)||)
9 %   window   : number of points in each regression window
10 %   min_err  : threshold below which an error value is treated as
11 %              numerical noise and excluded from the fit (1e-12)
12 %
13 % OUTPUT
14 %   rate_seq  : estimated rate for each window
15 %   k_centers : iteration index at the center of each window,
16 %              useful to plot rate_seq against iteration number
17
18 if nargin < 3, min_err = 1e-12; end
19
20 err = err(:)';
21 K = numel(err);
22
23 % mask of "valid" points (above the noise threshold)
24 valid = err > min_err;
25
26 rate_seq = [];
27 k_centers = [];
28
29 if window < 3
30     error('Window must be at least 3 for a meaningful regression');
31 end
32
33 for k0 = 1:(K - window + 1)
34     idx = k0 : k0+window-1;
35     if ~all(valid(idx)), continue; end % skip windows that touch noise
36
37     x_win = log(err(idx(1:end-1))); % log(e_k)
38     y_win = log(err(idx(2:end))); % log(e_{k+1})
39
40     if numel(x_win) < 2, continue; end
41
42     pfit = polyfit(x_win, y_win, 1); % pfit(1) = slope = estimated rate
43     rate_seq(end+1) = pfit(1); %#ok<AGROW>
44     k_centers(end+1) = idx(1) + floor(window/2); %#ok<AGROW>
45 end
46
47 end

```

Listing 11: Sliding-window estimator for the convergence rate.

```

1 function [xk,fk,gradfk_norm,k,xseq,btseq,alphas,gradfk_seq,fk_seq,tau_new,
2         pks] = modified_newton_method(x0,f,gradf,hessf,kmax,tolgrad,c1,rho,btmax,
3         beta)
4
5 % MODIFIED_NEWTON_METHOD Modified Newton method
6 % This functions is aimed to solve large scale numerical optimization
7 % problems using the modified Newton method with backtracking techniques.
8 %
9 % INPUT ARGUMENTS:
10 %   x0          : initial point (column vector)
11 %   f           : function handle for objective function f(x)
12 %   gradf       : function handle for the gradient gradf(x)
13 %   hessf       : function handle for the Hessian hessf(x)
14 %   kmax        : maximum number of iterations
15 %   tolgrad     : tolerance on the gradient norm for the stopping
16 %   criterion
17 %   c1, rho, btmax : parameters for Armijo/backtracking (0 < c1 < 1, 0

```

```

15     < rho < 1)
16 %         beta          : minimum initial increment for the Hessian
17 %         correction (beta > 0)
18 %
19 %     OUTPUT ARGUMENTS:
20 %         xk, fk, gradfk_norm : final point, objective value and gradient
21 %         norm at the solution
22 %         k                  : number of iterations performed
23 %         xseq               : sequence of points generated [x0, x1, ..., xk]
24 %         ] (n x (k+1))
25 %         btseq              : number of backtracking steps at each
26 %         iteration
27 %         tau_new            : matrix storing the tau values used at each
28 %         iteration
29 %         alphas             : step lengths
30 %         pks                : search directions
31
32 % INPUT CHECKS
33 if ~isnumeric(x0) || ~isvector(x0)
34     error('x0 must be a numeric column vector.');

```

```

70 if ~isscalar(kmax) || kmax <= 0 || floor(kmax) ~= kmax
71     error('kmax must be a positive integer.');
```

```

72 end
73
74 if ~isscalar(tolgrad) || tolgrad <= 0
75     error('tolgrad must be a positive scalar.');
```

```

76 end
77
78 if ~isscalar(c1) || c1 <= 0 || c1 >= 1
79     error('c1 must satisfy 0 < c1 < 1.');
```

```

80 end
81
82 if ~isscalar(rho) || rho <= 0 || rho >= 1
83     error('rho must satisfy 0 < rho < 1.');
```

```

84 end
85
86 if ~isscalar(btmax) || btmax <= 0 || floor(btmax) ~= btmax
87     error('btmax must be a positive integer.');
```

```

88 end
89
90 if ~isscalar(beta) || beta <= 0
91     error('beta must be a positive scalar.');
```

```

92 end
93
94 gradfk_norm = norm(gradfk);
95 if gradfk_norm < tolgrad
96     warning('Initial point already satisfies the stopping criterion.');
```

```

97 end
98
99
100 % Function handle for the armijo condition.
101 farmijo = @(fk, alpha, c1_gradfk_pk) ...
102     fk + alpha * c1_gradfk_pk;
103
104
105 % Inner parameters.
106 tol = 1e-6; % Tolerance to check the positiveness of Hk diagonal.
107 maxit = 200; % Max number of iteration to check the positiveness of Hk
    diagonal.
108
109 % Preallocations.
110 xseq = zeros(length(x0), kmax);
111 btseq = zeros(1, kmax);
112 alphas = zeros(1, kmax);
113 gradfk_seq = zeros(1, kmax);
114 fk_seq = zeros(1, kmax);
115 pks = zeros(length(x0), kmax);
116 tau_new = zeros(maxit+1, kmax);
117
118
119 k = 0;
120 while k < kmax && gradfk_norm >= tolgrad
121
122     % Compute the hessian matrix.
123     Hk = hessf(xk);
124
125     % Check if the hessian matrix is positive definite.
126     diagHk = full(diag(Hk));
127     isPositive = all(diagHk > tol);
128
129     if isPositive == true

```

```

130     tau_k = 0; % No need to add a correction term
131 else
132     tau_k = beta-min(diagHk);
133 end
134
135 % Vector to store the history of the correction term per iteration.
136 tauk = zeros(maxit+1,1);
137 tauk(1)= tau_k;
138
139 % Once the parameter tau is found then the correction of the hessian
140 % can be built.
141 % In order to check if the corrected matrix is positive definite, we
142 % attempt to perform the incomplete Choleski factorization:
143 % if Bk is not positive definite, then we get an error and consequently
144 % we must try to improve it.
145 for j = 1:maxit
146
147     if n >= 1e4
148         Bk = Hk+tauk(j)*speye(n);
149     else
150         Bk = Hk+tauk(j)*eye(n);
151     end
152
153     % Cholesky factorization.
154     [R,flag] = chol(Bk);
155
156     % Check if the correction is good enough (Is Bk positive
157     % definite?).
158     if flag == 0
159         break
160     else
161         % If the Bk is not positive definite, then we need to
162         % correct it.
163         tauk(j+1) = max(beta, 2*tauk(j));
164     end
165 end
166
167 % Once we obtain a matrix Bk which is positive definite we solve the
168 % following system with a direct solver.
169 pk = R\'(R\'(-gradfk));
170
171 % BACKTRACKING
172
173 alpha = 1; % Reset the value of alpha (parameter used for line-search).
174 xnew = xk + alpha * pk; % Computing the candidate solution at step k.
175 fnew = f(xnew);
176 c1_gradfk_pk = c1 * gradfk' * pk;
177 bt = 0;
178
179 while bt < btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
180
181     alpha = rho * alpha; % Step reduction.
182     xnew = xk + alpha * pk;
183     fnew = f(xnew);
184     bt = bt + 1;
185 end
186
187
188 % Check if the maximum number of backtracking iterations is reached

```

```

189     if bt == btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
190         k = k+1;
191         xseq(:, k)      = xk;
192         btseq(k)        = bt;
193         tau_new(:, k)   = tauk;
194         alphas(k)        = alpha;
195         pks(:, k)        = pk;
196         gradfk_seq(k)    = gradfk_norm;
197         fk_seq(k)        = fk;
198         break;
199     end
200
201     % Update variables.
202     xk = xnew;
203     fk = fnew;
204     gradfk = gradf(xk);
205     gradfk_norm = norm(gradfk);
206
207
208     k = k + 1;
209
210     % Storing.
211     xseq(:, k)      = xk;
212     btseq(k)        = bt;
213     tau_new(:, k)   = tauk;
214     alphas(k)        = alpha;
215     pks(:, k)        = pk;
216     gradfk_seq(k)    = gradfk_norm;
217     fk_seq(k)        = fk;
218
219 end %while loop on k
220
221
222 % Trimming the final structures.
223 xseq      = xseq(:, 1:k);
224 btseq     = btseq(1:k);
225 alphas    = alphas(1:k);
226 pks       = pks(:, 1:k);
227 gradfk_seq = gradfk_seq(1:k);
228 fk_seq    = fk_seq(1:k);
229 xseq = [x0, xseq];
230
231
232 end

```

Listing 12: Modified Newton method with backtracking

```

1 function [xk,fk,gradfk_norm,k,xseq,btseq,alphas,gradfk_seq,fk_seq,n_flips] =
   modified_newton_method_flip(x0,f,gradf,hessf,kmax,tolgrad,c1,rho,btmax)
2
3 % MODIFIED_NEWTON_METHOD_FLIP Modified Newton method using eigenvalue
   flipping to correct the Hessian matrix.
4 % Negative eigenvalues are replaced by their absolute values (|lambda_i|).
5 % If Hessian is diagonal, flip diagonal entries directly without calling eig
   ().
6 % Otherwise, compute full spectral decomposition (eig) for any dimension n.
7 %
8 %
9 %
10 % ADDITIONAL OUTPUT with respect to standard modified:
11 %     n_flips : number of iterations in which at least one eigenvalue was

```

```

    flipped
12
13 if ~isnumeric(x0) || ~isvector(x0)
14     error('x0 must be a numeric column vector.');
```

15 end

16

17 xk = x0(:);

18 n = length(xk);

19

20 fk = f(xk);

21 gradfk = gradf(xk);

22 gradfk_norm = norm(gradfk);

23

24 % Function handle for the armijo condition .

25 farmijo = @(fk, alpha, c1_gradfk_pk) fk + alpha * c1_gradfk_pk;

26

27 eps_floor = 1e-8;

28

29 % Preallocations.

30 xseq = zeros(n, kmax);

31 btseq = zeros(1, kmax);

32 alphas = zeros(1, kmax);

33 gradfk_seq = zeros(1, kmax);

34 fk_seq = zeros(1, kmax);

35 n_flips = 0;

36 toldiag = 1e-6;

37

38 k = 0;

39 while k < kmax && gradfk_norm >= tolgrad

40

41 % Compute the hessian matrix .

42 Hk = hessf(xk);

43

44 % Check if the matrix is diagonal

45 is_diag = nnz(Hk - spdiags(spdiags(Hk,0), 0, n, n)) == 0;

46

47 % If the matrix is diagonal, then just flipp the diagonal elements

48 if is_diag

49 d = full(diag(Hk));

50 applied = any(d <= toldiag);

51 d_mod = d;

52 d_mod(d <= toldiag) = max(abs(d(d <= toldiag)), eps_floor);

53 Bk = spdiags(d_mod, 0, n, n);

54 else

55 % If the matrix is not diagonal, then comput a full decomposition

56 [Q, D] = eig(full(Hk));

57 lambda = diag(D);

58 applied = any(lambda <= 0);

59 lambda_mod = lambda;

60 lambda_mod(lambda <= 0) = max(abs(lambda(lambda <= 0)), eps_floor);

61 Bk = Q * diag(lambda_mod) * Q';

62 end

63

64 %Updating the number of flips

65 if applied

66 n_flips = n_flips + 1;

67 end

68

69 % Once we obtain a matrix Bk which is positive definite we solve the

70 following system with a direct solver .

pk = -Bk \ gradfk;

```

71
72
73 % BACKTRACKING
74
75 alpha = 1; % Reset the value of alpha ( parameter used for line - search
    ) .
76 xnew = xk + alpha * pk; % Computing the candidate solution at step k .
77 fnew = f(xnew);
78 c1_gradfk_pk = c1 * gradfk' * pk;
79 bt = 0;
80
81 while bt < btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
82     alpha = rho * alpha; % Step reduction
83     xnew = xk + alpha * pk;
84     fnew = f(xnew);
85     bt = bt + 1;
86 end
87
88 % Check if the maximum number of backtracking iteration is reached
89 if bt == btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
90     k = k+1;
91     xseq(:, k)      = xk;
92     btseq(k)        = bt;
93     alphas(k)        = alpha;
94     gradfk_seq(k)    = gradfk_norm;
95     fk_seq(k)        = fk;
96     break;
97 end
98
99 % Update variables
100 xk = xnew;
101 fk = fnew;
102 gradfk = gradf(xk);
103 gradfk_norm = norm(gradfk);
104
105 k = k + 1;
106
107 % Storing
108 xseq(:, k)      = xk;
109 btseq(k)        = bt;
110 alphas(k)        = alpha;
111 gradfk_seq(k)    = gradfk_norm;
112 fk_seq(k)        = fk;
113
114 end % while loop on k
115
116 % Trimming the final structures.
117 xseq      = xseq(:, 1:k);
118 btseq     = btseq(1:k);
119 alphas    = alphas(1:k);
120 gradfk_seq = gradfk_seq(1:k);
121 fk_seq    = fk_seq(1:k);
122 xseq = [x0, xseq];
123
124 end

```

Listing 13: Modified Newton method with backtracking, with flipping strategy to correct the Hessian

```

1 function [xk,fk,gradfk_norm,k,xseq,btseq,pks,inner_iters] =
    truncated_newton_method(x0,f,gradf,hessf,kmax,tolgrad,c1,rho,btmax,max_cg
    )

```



```

2
3 % TRUNCATED_NEWTON_METHOD Truncated Newton method (Newton-CG)
4 %
5 % [xk, fk, gradfk_norm, k, xseq, btseq, pks, inner_iters] = ...
6 %     TRUNCATED_NEWTON_METHOD(x0, f, gradf, hessf, kmax, tolgrad, ...
7 %     c1, rho, btmax, max_cg)
8 %
9 % This function solves unconstrained numerical optimization problems
10 % using a Truncated Newton approach.
11 %
12 % INPUT ARGUMENTS:
13 %     x0          : Initial guess (column vector)
14 %     f           : Function handle for the objective function f(x)
15 %     gradf       : Function handle for the gradient gradf(x)
16 %     hessf       : Function handle for the Hessian matrix H(x)
17 %     kmax        : Maximum number of outer iterations
18 %     tolgrad     : Stopping tolerance on the norm of the gradient
19 %     c1          : Armijo condition parameter (0 < c1 < 1)
20 %     rho         : Backtracking reduction factor (0 < rho < 1)
21 %     btmax       : Maximum number of backtracking steps per iteration
22 %     max_cg      : Maximum number of inner (CG) iterations
23 %
24 % OUTPUT ARGUMENTS:
25 %     xk          : Final point reached by the algorithm
26 %     fk          : Function value at xk
27 %     gradfk_norm : Norm of the gradient at xk
28 %     k           : Total number of iterations performed
29 %     xseq        : Sequence of points generated
30 %     btseq       : Number of backtracking steps per iteration
31 %     pks         : Sequence of search directions
32 %     inner_iters : Number of CG iterations performed per step
33 %
34 % INPUT CHECKS
35
36
37 if ~isnumeric(x0) || ~isvector(x0)
38     error('x0 must be a numeric column vector.');

```

```

63 if ~isnumeric(gradfk) || ~isequal(size(gradfk),[n,1])
64     error('gradf(x) must return a column vector of the same size as x.');
```

65 end

```

66
67 if ~isa(hessf,'function_handle')
68     error('hessf must be a function handle.');
```

69 end

```

70 try
71     H0 = hessf(xk);
72 catch
73     error('hessf(x0) cannot be evaluated.');
```

74 end

```

75 if ~isnumeric(H0) || ~isequal(size(H0),[n,n])
76     error('hessf(x) must return an n-by-n matrix.');
```

77 end

```

78
79 if ~isscalar(kmax) || kmax <= 0 || floor(kmax) ~= kmax
80     error('kmax must be a positive integer.');
```

81 end

```

82
83 if ~isscalar(tolgrad) || tolgrad <= 0
84     error('tolgrad must be a positive scalar.');
```

85 end

```

86
87 if ~isscalar(c1) || c1 <= 0 || c1 >= 1
88     error('c1 must satisfy 0 < c1 < 1.');
```

89 end

```

90
91 if ~isscalar(rho) || rho <= 0 || rho >= 1
92     error('rho must satisfy 0 < rho < 1.');
```

93 end

```

94
95 if ~isscalar(btmax) || btmax <= 0 || floor(btmax) ~= btmax
96     error('btmax must be a positive integer.');
```

97 end

```

98
99 if ~isscalar(max_cg) || max_cg <= 0
100     error('max_cg must be a positive scalar.');
```

101 end

```

102
103 gradfk_norm = norm(gradfk);
104 if gradfk_norm < tolgrad
105     warning('Initial point already satisfies the stopping criterion.');
```

106 end

```

107
108
109 % Function handle for the Armijo condition.
110 farmijo = @(fk, alpha, c1_gradfk_pk) ...
111     fk + alpha * c1_gradfk_pk;
112
113 % Preallocations.
114 xseq      = zeros(length(x0), kmax+1);
115 btseq     = zeros(1, kmax);
116 alphas    = zeros(1, kmax);
117 pks       = zeros(length(x0), kmax);
118 inner_iters = zeros(1, kmax);
119
120
121 k = 0;
122 xseq(:,1) = xk;
```

```

124 while k < kmax && gradfk_norm >= tolgrad
125
126     eta_k = min(0.5, sqrt(gradfk_norm)) * gradfk_norm;
127
128     % The system to solve is  $H_k * p_k = -\text{gradfk}$ .
129     z = zeros(length(x0),1);
130     Hk = hessf(xk);
131     rk = gradfk;
132     dk = -rk;
133
134     r_old = rk' * rk;
135
136     % Boolean variable used to understand whether the inner loop got to
137     % convergence, it has to go back to false at each iteration k.
138     stop_inner = false;
139     j = 0;
140     while ~stop_inner && j < max_cg % Inner loop for solving the system with
141     % CG.
142
143         Hdk = Hk * dk;
144         curv = dk' * Hdk;
145
146         % If the curvature is non-positive (or near zero), the CG loop is
147         % terminated: fall back to steepest descent at the first iteration,
148         % or accept the last valid approximation otherwise.
149         if curv <= 1e-10 * norm(dk)^2
150             if j == 0
151                 p_tn = -gradfk;
152             else
153                 p_tn = z;
154             end
155             break
156         else
157             alpha_j = r_old / curv;
158             z = z + alpha_j * dk;
159             rk = rk + alpha_j * Hdk;
160
161             % Check convergence.
162             if norm(rk) <= eta_k
163                 p_tn = z;
164                 break;
165             end
166
167             % Updates for the next iteration.
168             rk_new = rk' * rk;
169             beta_j = rk_new / r_old;
170             dk = -rk + beta_j * dk;
171             r_old = rk_new;
172
173             j = j + 1;
174         end
175     end
176
177     j_cg = j;
178
179     if j == max_cg
180         disp(['MAX_CG (Truncated Newton): maximum number of ', ...
181             'CG iterations reached.']);

```

```

181         p_tn = z; % CG done without meeting the curvature or tolerance
           condition: use the last valid iterate
182     end
183
184     pk = p_tn;
185
186     if norm(pk) <= 1e-12
187         disp('Truncated Newton: null direction, stop.');
```

188

```

189         if gradfk_norm < tolgrad
190             return;
191         else
192             warning(['Truncated Newton: pk near zero but ', ...
193                     'gradient not converged.']);
194         end
195     end
196 end
197
198 % BACKTRACKING.
199
200
201 alpha = 1; % Reset the value of alpha (parameter used for linesearch).
202 xnew = xk + alpha * pk; % Computing the candidate solution at step k.
203 fnew = f(xnew);
204 c1_gradfk_pk = c1 * gradfk' * pk;
205
206 bt = 0;
207
208 while bt < btmax && ...
209     fnew > farmijo(fk, alpha, c1_gradfk_pk)
210
211     alpha = rho * alpha; % Step reduction
212     xnew = xk + alpha * pk;
213     fnew = f(xnew);
214     bt = bt + 1;
215
216 end
217
218 if bt == btmax && ...
219     fnew > farmijo(fk, alpha, c1_gradfk_pk)
220
221     disp(['Backtracking (Truncated Newton): maximum number ', ...
222           'of iterations reached.']);
223
224     k = k + 1;
225     xseq(:, k+1) = xk;
226     btseq(k) = bt;
227     pks(:, k) = pk;
228     alphas(k) = alpha;
229     inner_iters(k) = j_cg;
230
231     break;
232 end
233
234 % Update variables.
235 xk = xnew;
236 fk = fnew;
237 gradfk = gradf(xk);
238 gradfk_norm = norm(gradfk);
239
240 k = k + 1;

```

```

241
242     % Store iteration data.
243     xseq(:, k+1) = xk;
244     btseq(k) = bt;
245     pks(:, k) = pk;
246     alphas(k) = alpha;
247     inner_iters(k) = j_cg;
248
249 end % Outer loop.
250
251 % Trimming the final structures.
252 xseq = xseq(:, 1:k+1);
253 btseq = btseq(1:k);
254 pks = pks(:, 1:k);
255 inner_iters = inner_iters(:, 1:k);
256
257 end

```

Listing 14: Truncated Newton method with Conjugate Gradient and backtracking

```

1 function [xk,fk,gradfk_norm,k,xseq,btseq,pks,inner_iters] =
   truncated_hess_free(x0,f,gradf,kmax,tolgrad,c1,rho,btmax,max_cg,h_fd)
2
3 % TRUNCATED_NEWTON_METHOD Truncated Newton method, matrix-free.
4 %
5 % This function solves unconstrained numerical optimization problems
6 % using a Truncated Newton approach.
7 % The Hessian is never built explicitly: at each outer iteration the
8 % Newton system  $H(x_k)p = -\text{gradf}(x_k)$  is solved approximately with the
9 % Conjugate Gradient (CG) method, and every Hessian-vector product
10 %  $H(x_k)p$  required by CG is approximated with a centered finite
11 % difference on the gradient.
12 %
13 % INPUT ARGUMENTS:
14 %     x0           : Initial point (column vector)
15 %     f            : Function handle for the objective function  $f(x)$ 
16 %     gradf        : Function handle for the gradient  $\text{gradf}(x)$ 
17 %     kmax         : Maximum number of outer iterations
18 %     tolgrad      : Stopping tolerance on the gradient norm for the
   stopping criterion
19 %     c1, rho, btmax : parameters for Armijo/backtracking ( $0 < c1 < 1$ ,  $0 < \rho < 1$ )
20 %     max_cg       : Maximum number of inner (CG) iterations
21 %     h_fd         : Finite-difference step for the Hessian-vector
   product
22 %
23 % OUTPUT ARGUMENTS:
24 %     xk, fk, gradfk_norm : final point, objective value and gradient
   norm at the solution
25 %     k               : Total number of iterations performed
26 %     xseq            : Sequence of points generated ( $n \times (k+1)$ )
27 %     btseq           : Number of backtracking steps per iteration (1
   x k)
28 %     pks             : Sequence of search directions
29 %     inner_iters     : Number of CG iterations performed per step
30
31 % INPUT CHECKS
32
33 if ~isnumeric(x0) || ~isvector(x0)
34     error('x0 must be a numeric column vector.');
```

```

36 xk = x0(:); % force column vector
37 n = length(xk);
38
39 if ~isa(f,'function_handle')
40     error('f must be a function handle.');
```

```

41 end
42 try
43     fk = f(xk);
44 catch
45     error('f(x0) cannot be evaluated.');
```

```

46 end
47 if ~isscalar(fk) || ~isreal(fk)
48     error('f(x) must return a real scalar.');
```

```

49 end
50
51 if ~isa(gradf,'function_handle')
52     error('gradf must be a function handle.');
```

```

53 end
54 try
55     gradfk = gradf(xk);
56 catch
57     error('gradf(x0) cannot be evaluated.');
```

```

58 end
59 if ~isnumeric(gradfk) || ~isequal(size(gradfk),[n,1])
60     error('gradf(x) must return a column vector of the same size as x.');
```

```

61 end
62
63 if ~isscalar(kmax) || kmax <= 0 || floor(kmax) ~= kmax
64     error('kmax must be a positive integer.');
```

```

65 end
66
67 if ~isscalar(tolgrad) || tolgrad <= 0
68     error('tolgrad must be a positive scalar.');
```

```

69 end
70
71 if ~isscalar(c1) || c1 <= 0 || c1 >= 1
72     error('c1 must satisfy 0 < c1 < 1.');
```

```

73 end
74
75 if ~isscalar(rho) || rho <= 0 || rho >= 1
76     error('rho must satisfy 0 < rho < 1.');
```

```

77 end
78
79 if ~isscalar(btmax) || btmax <= 0 || floor(btmax) ~= btmax
80     error('btmax must be a positive integer.');
```

```

81 end
82
83 if ~isscalar(max_cg) || max_cg <= 0
84     error('max_cg must be a positive scalar.');
```

```

85 end
86
87 if nargin < 10 || isempty(h_fd)
88     h_fd = eps^(1/3);
89 end
90
91 gradfk_norm = norm(gradfk);
92 if gradfk_norm < tolgrad
93     warning('Initial point already satisfies the stopping criterion.');
```

```

94 end
95
96

```

```

97 % Function handle for the armijo condition.
98 farmijo = @(fk, alpha, c1_gradfk_pk) ...
99     fk + alpha * c1_gradfk_pk;
100
101 % Preallocations
102 xseq      = zeros(length(x0), kmax+1);
103 btseq     = zeros(1, kmax);
104 alphas    = zeros(1, kmax);
105 pks       = zeros(length(x0), kmax);
106 inner_iters = zeros(1, kmax);
107
108
109 k = 0;
110 xseq(:,1) = xk;
111
112 while k < kmax && gradfk_norm >= tolgrad
113
114     eta_k = min(0.5, sqrt(gradfk_norm))*gradfk_norm;
115
116     % The system we need to solve is Hess(fk)*pk = -graf(fk) <-> Hk*z=ck
117     % Hv(p) returns H(xk)*p via a centered finite difference (matrix-free).
118
119     Hv = @(p) hess_vec_fd(xk, p, gradf, h_fd);
120     z = zeros(length(x0),1);
121     rk =gradfk;
122     dk = -rk;
123
124     r_old = rk'*rk;
125
126     % Boolean variable used to understand whether the inner loop got to
127     % convergence,
128     % it has to go back to false at each iteration k.
129     stop_inner = false;
130
131     j = 0;
132     while ~stop_inner && j < max_cg % Inner loop for solving the system with
133     CG.
134
135         Hdk = Hv(dk);
136         curv = dk'*Hdk;
137
138         % If the curvature is non-positive (or near zero), the CG loop is
139         % terminated: fall back to steepest descent at the first iteration,
140         % or accept the last valid approximation otherwise.
141         if curv <= 1e-10 * norm(dk)^2
142             if j == 0
143                 p_tn = -gradfk;
144             else
145                 p_tn = z;
146             end
147             break
148         else
149             alpha_j = r_old/curv;
150             z = z + alpha_j * dk;
151             rk = rk + alpha_j * Hdk;
152
153             % Check convergence
154             if norm(rk) <= eta_k
155                 p_tn = z;
156                 %stop_inner = true;

```

```

154         break;
155     end
156
157     % Updates for the next iteration.
158     rk_new = rk'*rk;
159     beta_j = rk_new/r_old;
160     dk = -rk + beta_j * dk;
161     r_old = rk_new;
162
163     j = j+1;
164
165     end
166
167 end
168
169 j_cg = j;
170
171 if j == max_cg
172     disp('MAX_CG (Truncated Newton - Hess free): maximum number of cg
173         iterations reached. ');
174     p_tn = z; % CG done without meeting the curvature or tolerance
175               condition: use the last valid iterate
176 end
177 % Assign the computed step/search direction
178 pk = p_tn;
179
180 % Check if the search direction is null
181 if norm(pk) <= 1e-12
182     disp('Truncated Newton Hess free: null direction, stop. ');
183     if gradfk_norm < tolgrad
184         return;
185     else
186         warning('Truncated Newton: pk near zero but gradient not
187             converged. ');
188     end
189 end
190
191 % BACKTRACKING.
192
193 alpha = 1; % Reset the value of alpha (parameter used for linesearch)
194 xnew = xk + alpha * pk; % Computing the candidate solution at step k.
195 fnew = f(xnew);
196 c1_gradfk_pk = c1 * gradfk' * pk;
197
198 bt = 0;
199 while bt < btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
200     alpha = rho * alpha; % Step reduction
201     xnew = xk + alpha * pk;
202     fnew = f(xnew);
203     bt = bt + 1;
204 end
205
206 if bt == btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
207     disp('Backtracking (Truncated Newton): maximum number of iterations
208         reached. ');
209     k = k+1;
210     xseq(:, k+1) = xk;
211     btseq(k) = bt;
212     pks(:, k) = pk;

```



```

211         alphas(k) = alpha;
212         inner_iters(k) = j_cg;
213
214         break;
215     end
216
217     xk = xnew;
218     fk = fnew;
219     gradfk = gradf(xk);
220     gradfk_norm = norm(gradfk);
221     k = k + 1;
222
223     % Storing iteration data
224     xseq(:, k+1) = xk;
225     btseq(k) = bt;
226     pks(:, k) = pk;
227     alphas(k) = alpha;
228     inner_iters(k) = j_cg;
229
230
231 end %while loop on k
232
233 % Trimming the final structures.
234 xseq = xseq(:, 1:k+1);
235 btseq = btseq(1:k);
236 pks = pks(:, 1:k);
237 inner_iters = inner_iters(:, 1:k);
238 end %function end
239
240 function Hp = hess_vec_fd(xk, p, gradf, h_fd)
241 % HESS_VEC_FD Approximates the Hessian-vector product  $H(x_k)*p$  using a
242 % centered finite difference on the gradient, without ever building  $H$ 
243 % explicitly:
244 %
245 %  $Hp = ( \text{gradf}(x_k + h*p) - \text{gradf}(x_k - h*p) ) / (2*h)$ 
246
247 normp = norm(p);
248
249 if normp < eps
250     Hp = zeros(size(p));
251     return;
252 end
253
254 h = h_fd/ normp;
255 Hp = (gradf(xk + h*p) - gradf(xk - h*p)) / (2*h);
256 end

```

Listing 15: Truncated Newton method with Conjugate Gradient and backtracking, Hessian free implementation

```

1 clear; clc; close all
2 % MAIN SCRIPT FOR BROYDEN31
3 % Add current folder and subfolders to MATLAB path
4 project_root = fileparts(mfilename('fullpath'));
5 addpath(genpath(project_root));
6
7
8 % Problem Loading
9 [f, grad_exact, hess_exact, xbarfun, rfun, xstarfun] = problem_broyden31();
10
11 % Fixed Parameters

```



```

71         case "case2"
72             % Full FD: FD gradient and FD Hessian
73             gradf = @(x) grad_fd_broyden31(x, kk, type, rfun);
74             hessf = @(x) hess_grad_fd_broyden31(x, kk, type, rfun);
75         end
76
77     % Loop over optimization methods
78     for method = ["modified", "truncated"]
79         if method == "modified"
80             prm = params_modified;
81         else
82             prm = params_truncated;
83         end
84
85     % Loop over the 6 starting points (1 standard + 5 random)
86     for s = 1:6
87         x0 = X0(:, s);
88         tic;
89         run_failed = false;
90         try
91             % Run selected optimization algorithm
92             if method == "modified"
93                 [~, fk, gn, it, xseq] = modified_newton_method(
94                     x0, f, ...
95                     gradf, hessf, prm.kmax, tolgrad, prm.c1, prm
96                     .rho, ...
97                     prm.btmax, prm.beta);
98             else
99                 [~, fk, gn, it, xseq, ~, inner] =
100                     truncated_newton_method(x0, f, ...
101                     gradf, hessf, prm.kmax, tolgrad, prm.c1, prm
102                     .rho, ...
103                     prm.btmax, prm.max_cg);
104             end
105         catch ME
106             fk = NaN; gn = Inf; it = 0; xseq = [];
107             run_failed = true;
108             warning('Failed (%s, %s, n=%d, k=%g, type=%d): %s',
109                 ...
110                 char(method), char(dm), n, kk, type, ME.message)
111             ;
112         end
113
114     t = toc; % Save execution time
115
116     % Determine success and reason for stopping
117     succ = ~run_failed && gn < tolgrad;
118     if run_failed
119         stop_reason = "error";
120     elseif succ
121         stop_reason = "converged";
122     elseif it >= prm.kmax
123         stop_reason = "maxit";
124     else
125         stop_reason = "backtracking_or_other";
126     end
127
128     % Compute convergence rate only if successful
129     if succ
130         gnorm_last5 = zeros(1,5);
131         K = size(xseq,2);
132         % Get gradient norm for the last 5 iterations

```

```

126         for i = 1:5
127             gnorm_last5(i) = norm(grad_exact(xseq(:, K-5+i))
128                                     );
129         end
130         % Estimate convergence rate using polynomial fit
131         [rate_seq, ~] = estimate_rate_polyfit(gnorm_last5,
132                                             5);
133         rate = rate_seq;
134     else
135         rate = NaN;
136     end
137     % Save results
138     idx = idx + 1;
139     results(idx).n = n;
140     results(idx).deriv_mode = dm;
141     results(idx).k = kk;
142     results(idx).type = type;
143     results(idx).method = method;
144     results(idx).start = s;
145     results(idx).iter = it;
146     results(idx).gradnorm = gn;
147     results(idx).success = succ;
148     results(idx).stop_reason = stop_reason;
149     results(idx).rate = rate;
150     results(idx).time = t;
151     results(idx).xseq = xseq;
152     %results(idx).inner_iters = inner;
153
154     % Formatting for printing
155     if isnan(kk)
156         kstr = "n/a";
157         typestr = "n/a";
158     else
159         kstr = string(kk);
160         if type == 1, typestr = "h"; else, typestr = "hi";
161         end
162     end
163
164     fprintf('%-9s|%-6s|n=%-6d|k=%-4s|%-3s|start=%d
165             |iter=%3d|gn=%.2e|succ=%d|reason=%-22s|rate
166             =%.2f|t=%.2fs\n', ...
167             method, dm, n, kstr, typestr, s, it, gn, succ,
168             stop_reason, rate, t);
169
170     end
171
172     end
173
174     end
175
176     end
177
178     % Save Results
179     save('broyden31_fd_results.mat', 'results');
180     disp('Done. Results saved in broyden31_fd_results.mat');
181
182     % Summary of Failed Runs
183     non_conv = ~[results.success];
184     if any(non_conv)
185         reasons = string({results(non_conv).stop_reason});
186         fprintf('\n---Run NON convergent: %d / %d---\n', nnz(non_conv), numel
187                 (results));
188         for r = unique(reasons)

```

```

180         fprintf('  %d %d %d\n', r, sum(reasons == r));
181     end
182 end

```

Listing 16: Main script for Broyden 31 problem.

```

1 % MAIN SCRIPT FOR TRIG16
2 clear; clc; close all;
3
4 % Add current folder and subfolders to MATLAB pathproject_root = fileparts(
    mfilename('fullpath')); % cartella dove sta questo script
5 addpath(genpath(project_root));
6
7 % Problem Loading
8 [f, grad_exact, hess_exact, xbarfun, xstarfun] = problem_trig16();
9
10 % Fixed Parameters
11 seed = 346710;
12 rng(seed);
13 tolgrad = 1e-6;
14
15 n_list = [2, 1e3, 1e4, 1e5]; % Problem dimensions
16 k_list = [4, 8, 12]; % Finite difference parameter k values
17 fdtypes = [1, 2]; % Step type: 1= constant step (h), 2 =
    relative step
18
19 % Tuned Parameters for Modified and Truncated Newton
20 params_modified.kmax = 200; params_modified.c1 = 1e-4;
21 params_modified.rho = 0.5; params_modified.btmax = 30;
22 params_modified.beta = 1e-2;
23
24 params_truncated.kmax = 500; params_truncated.c1 = 1e-3;
25 params_truncated.rho = 0.5; params_truncated.btmax = 40;
26 params_truncated.max_cg = 1000;
27
28 % Main Loop
29 results = struct([]);
30 idx = 0;
31
32 for n = n_list
33     fprintf(' \n===== n = %d =====\n', n);
34
35     % Generate starting points
36     xb = xbarfun(n); % Standard starting point
37
38     % Generate 5 random points
39     X0 = [xb, xb + (2*rand(n,5) - 1)];
40
41     % Test cases: Exact, FD Hessian only, Full FD
42     for dm = ["exact", "case1", "case2"]
43
44         % Determine which k and type values to loop over
45         if dm == "exact"
46             k_loop = [NaN]; % no FD loop needed
47             type_loop = [0]; % no type loop needed
48         else
49             k_loop = k_list; % Loop over k = [4, 8, 12]
50             type_loop = fdtypes; % Loop over type = [1, 2]
51         end
52
53         % Loop over k values

```

```

54     for kk = k_loop
55         for type = type_loop
56
57             % Select gradient and Hessian evaluation methods
58             switch dm
59                 case "exact"
60                     % Exact gradient and exact hessian
61                     gradf = grad_exact;
62                     hessf = hess_exact;
63                 case "case1"
64                     % Exact gradient, FD Hessian
65                     gradf = grad_exact;
66                     hessf = @(x) trig_hess_fd_case1(grad_exact, x, kk, type)
67                     ;
68                 case "case2"
69                     % Full FD: FD gradient and FD Hessian
70                     gradf = @(x) trig_fd_case2_grad_only(x, kk, type);
71                     hessf = @(x) trig_fd_case2_hess_only(x, kk, type);
72             end
73
74             % Loop over optimization methods
75             for method = ["modified", "truncated"]
76                 if method == "modified"
77                     prm = params_modified;
78                 else
79                     prm = params_truncated;
80                 end
81
82                 % Loop over the 6 starting points (1 standard + 5 random)
83                 for s = 1:6
84                     x0 = X0(:, s);
85                     tic;
86                     run_failed = false;
87                     try
88                         % Run selected optimization algorithms
89                         if method == "truncated"
90                             [~, fk, gn, it, xseq,~,~,inner] =
91                                 truncated_newton_method(x0, f, ...
92                                     gradf, hessf, prm.kmax, tolgrad, prm.c1, prm
93                                         .rho, ...
94                                         prm.btmax, prm.max_cg);
95                         else
96                             [~, fk, gn, it, xseq] = modified_newton_method(
97                                 x0, f, ...
98                                 gradf, hessf, prm.kmax, tolgrad, prm.c1, prm
99                                     .rho, ...
100                                     prm.btmax, prm.beta);
101                         end
102                     catch ME
103                         fk = NaN; gn = Inf; it = 0; xseq = [];
104                         run_failed = true;
105                         warning('Failed_(%s,%s,n=%d,k=%g,type=%d):_s',
106                             ...
107                             char(method), char(dm), n, kk, type, ME.message)
108                         ;
109                     end
110
111                     t = toc; % Save execution time
112
113                     % Determine success and reason for stopping
114                     succ = ~run_failed && gn < tolgrad;
115                     if run_failed

```

```

108         stop_reason = "error";
109     elseif succ
110         stop_reason = "converged";
111     elseif it >= prm.kmax
112         stop_reason = "maxit";
113     else
114         stop_reason = "backtracking_or_other";
115     end
116
117     % Compute convergence rate only if successful
118     if succ
119         gnorm_last4 = zeros(1,4);
120         K = size(xseq,2);
121         % Get gradient norm for the last 4 iterations
122         for i = 1:4
123             gnorm_last4(i) = norm(grad_exact(xseq(:, K-4+i))
124                                     );
125         end
126         % Estimate convergence rate using polynomial fit
127         [rate_seq, ~] = estimate_rate_polyfit(gnorm_last4,
128             4);
129         rate = rate_seq;
130     else
131         rate = NaN;
132     end
133
134     % Save results
135     idx = idx + 1;
136     results(idx).n = n;
137     results(idx).deriv_mode = dm;
138     results(idx).k = kk;
139     results(idx).type = type;
140     results(idx).method = method;
141     results(idx).start = s;
142     results(idx).iter = it;
143     results(idx).gradnorm = gn;
144     results(idx).success = succ;
145     results(idx).stop_reason = stop_reason;
146     results(idx).rate = rate;
147     results(idx).time = t;
148     results(idx).xseq = xseq;
149
150     % Formatting for printing
151     if isnan(kk)
152         kstr = "n/a";
153         typestr = "n/a";
154     else
155         kstr = string(kk);
156         if type == 1, typestr = "h"; else, typestr = "hi";
157     end
158
159     fprintf('%-9s|%-6s|n=%-6d|k=%-4s|%-3s|start=%d|iter=%3d|gn=%.2e|succ=%d|reason=%-22s|rate=%.2f|t=%.2fs\n', ...
160         method, dm, n, kstr, typestr, s, it, gn, succ, stop_reason, rate, t);
161
162     end
163
164     end
165
166     end

```

```

163 end
164 end
165
166 % Save Results
167 save('trig16_fd_results_MODIFIED.mat', 'results');
168 disp('Done. Results saved in trig16_fd_results_MODIFIED.mat');
169
170 % Summary of Failed Runs
171 non_conv = ~[results.success];
172 if any(non_conv)
173     reasons = string({results(non_conv).stop_reason});
174     fprintf('\n--- Run NON convergent i: %d / %d ---\n', nnz(non_conv), numel
175             (results));
176     for r = unique(reasons)
177         fprintf('  %s: %d\n', r, sum(reasons == r));
178     end
179 end

```

Listing 17: Main script for Trig16 problem.

D Tables

D.1 Trig16

Consulting these tables one should note that, across several case-2 runs, we obtained experimentally estimated rates well above the theoretical maximum of Newton’s method (e.g. 6.04 and 10.92 for two starting points at $k = 8$). These are not evidence of super-cubic convergence: they are a known artefact of the point-wise 3-point rate estimator which becomes unstable when consecutive gradient norms are nearly equal in ratio, close to convergence. We report them for completeness but they should not be read as genuine convergence-order estimates.

Table 8: Trig16 – $n=2$, exact: Modified Newton vs Truncated Newton with exact gradient and hessian.

start.pt ID	Modified Newton							Truncated Newton						
	grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)		
xbar	$1.38e-07$	3/200	yes	-	3.000	0.00	$9.93e-13$	4/500	yes	-	NaN	0.00		
rand1	$5.64e-14$	7/200	yes	-	NaN	0.00	$9.31e-14$	6/500	yes	-	NaN	0.00		
rand2	$2.48e-16$	4/200	yes	-	NaN	0.00	$2.29e-14$	4/500	yes	-	NaN	0.00		
rand3	$6.63e-14$	6/200	yes	-	NaN	0.00	$1.04e-13$	6/500	yes	-	NaN	0.00		
rand4	$2.54e-11$	3/200	yes	-	3.000	0.00	$7.92e-08$	3/500	yes	-	3.080	0.00		
rand5	$2.22e-16$	4/200	yes	-	NaN	0.00	$2.48e-16$	4/500	yes	-	NaN	0.00		
Avg (successes)	$2.30e-08$	4/200	-	-	3.000	0.00	$1.32e-08$	4/500	-	-	3.080	0.00		

Table 9: Trig16 – $n=2$, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton							Truncated Newton						
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)		
xbar	4	$1.38e-07$	3/200	yes	-	3.000	0.00	$1.48e-12$	4/500	yes	-	3.020	0.00		
rand1	4	$2.14e-13$	7/200	yes	-	NaN	0.00	$3.28e-13$	6/500	yes	-	NaN	0.00		
rand2	4	$8.22e-15$	4/200	yes	-	NaN	0.00	$1.70e-13$	4/500	yes	-	NaN	0.00		
rand3	4	$2.33e-13$	6/200	yes	-	NaN	0.00	$3.47e-13$	6/500	yes	-	NaN	0.00		
rand4	4	$2.69e-11$	3/200	yes	-	2.990	0.00	$7.92e-08$	3/500	yes	-	3.080	0.00		
rand5	4	$1.75e-14$	4/200	yes	-	NaN	0.00	$1.07e-14$	4/500	yes	-	NaN	0.00		
Avg (successes)	4	$2.30e-08$	4/200	-	-	2.995	0.00	$1.32e-08$	4/500	-	-	3.050	0.00		
xbar	8	$1.38e-07$	3/200	yes	-	3.000	0.00	$1.54e-12$	4/500	yes	-	3.020	0.00		
rand1	8	$7.72e-13$	7/200	yes	-	NaN	0.00	$4.48e-13$	6/500	yes	-	NaN	0.00		
rand2	8	$4.10e-14$	4/200	yes	-	NaN	0.00	$4.07e-13$	4/500	yes	-	NaN	0.00		
rand3	8	$8.12e-13$	6/200	yes	-	NaN	0.00	$4.59e-13$	6/500	yes	-	NaN	0.00		
rand4	8	$2.95e-11$	3/200	yes	-	2.970	0.00	$7.92e-08$	3/500	yes	-	3.080	0.00		
rand5	8	$8.76e-14$	4/200	yes	-	NaN	0.00	$2.12e-14$	4/500	yes	-	NaN	0.00		
Avg (successes)	8	$2.30e-08$	4/200	-	-	2.985	0.00	$1.32e-08$	4/500	-	-	3.050	0.00		
xbar	12	$4.91e-07$	3/200	yes	-	2.730	0.00	$6.62e-09$	4/500	yes	-	1.950	0.00		
rand1	12	$7.56e-09$	7/200	yes	-	1.640	0.00	$3.28e-09$	6/500	yes	-	1.790	0.00		
rand2	12	$9.89e-11$	4/200	yes	-	1.310	0.00	$3.30e-09$	4/500	yes	-	1.700	0.00		
rand3	12	$5.62e-09$	6/200	yes	-	1.680	0.00	$5.39e-09$	6/500	yes	-	1.750	0.00		
rand4	12	$6.13e-08$	3/200	yes	-	1.910	0.00	$5.27e-08$	3/500	yes	-	3.160	0.00		
rand5	12	$4.20e-10$	4/200	yes	-	1.400	0.00	$1.49e-11$	4/500	yes	-	1.570	0.00		
Avg (successes)	12	$9.43e-08$	4/200	-	-	1.778	0.00	$1.19e-08$	4/500	-	-	1.987	0.00		

Table 10: Trig16 – n=2, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.38e-07	3/200	yes	-	3.000	0.00	1.26e-12	4/500	yes	-	3.040	0.00
rand1	4	2.50e-13	7/200	yes	-	NaN	0.00	2.63e-13	6/500	yes	-	NaN	0.00
rand2	4	1.02e-14	4/200	yes	-	NaN	0.00	1.29e-13	4/500	yes	-	NaN	0.00
rand3	4	2.70e-13	6/200	yes	-	NaN	0.00	2.80e-13	6/500	yes	-	NaN	0.00
rand4	4	2.65e-11	3/200	yes	-	2.990	0.00	7.92e-08	3/500	yes	-	3.080	0.00
rand5	4	2.11e-14	4/200	yes	-	NaN	0.00	9.04e-15	4/500	yes	-	NaN	0.00
Avg (successes)	4	2.30e-08	4/200	-	-	2.995	0.00	1.32e-08	4/500	-	-	3.060	0.00
xbar	8	1.38e-07	3/200	yes	-	3.000	0.00	5.73e-13	4/500	yes	-	NaN	0.00
rand1	8	3.67e-13	7/200	yes	-	NaN	0.00	5.47e-13	6/500	yes	-	NaN	0.00
rand2	8	5.14e-14	4/200	yes	-	NaN	0.00	1.02e-13	4/500	yes	-	NaN	0.00
rand3	8	2.43e-13	6/200	yes	-	NaN	0.00	4.56e-13	6/500	yes	-	NaN	0.00
rand4	8	3.24e-11	3/200	yes	-	2.960	0.00	7.91e-08	3/500	yes	-	3.080	0.00
rand5	8	7.97e-14	4/200	yes	-	NaN	0.00	2.44e-14	4/500	yes	-	NaN	0.00
Avg (successes)	8	2.30e-08	4/200	-	-	2.980	0.00	1.32e-08	4/500	-	-	3.080	0.00
xbar	12	1.67e-07	3/200	yes	-	2.960	0.00	8.35e-09	4/500	yes	-	1.910	0.00
rand1	12	4.56e-09	7/200	yes	-	1.780	0.00	4.23e-09	6/500	yes	-	1.880	0.00
rand2	12	1.90e-10	4/200	yes	-	1.510	0.00	1.79e-09	4/500	yes	-	1.790	0.00
rand3	12	4.52e-09	6/200	yes	-	1.710	0.00	7.83e-09	6/500	yes	-	1.740	0.00
rand4	12	3.67e-08	3/200	yes	-	1.990	0.00	4.69e-07	3/500	yes	-	2.700	0.00
rand5	12	1.41e-10	4/200	yes	-	1.620	0.00	2.73e-10	4/500	yes	-	1.520	0.00
Avg (successes)	12	3.55e-08	4/200	-	-	1.928	0.00	8.19e-08	4/500	-	-	1.923	0.00

Table 11: Trig16 – n=2, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.38e-07	3/200	yes	-	3.000	0.00	2.29e-12	4/500	yes	-	2.960	0.01
rand1	4	1.14e-12	7/200	yes	-	2.660	0.00	1.24e-12	6/500	yes	-	NaN	0.00
rand2	4	5.55e-13	4/200	yes	-	NaN	0.00	8.33e-13	4/500	yes	-	2.600	0.00
rand3	4	2.24e-12	6/200	yes	-	2.640	0.00	1.14e-12	6/500	yes	-	NaN	0.00
rand4	4	3.35e-11	3/200	yes	-	2.960	0.00	7.91e-08	3/500	yes	-	3.080	0.00
rand5	4	1.14e-12	4/200	yes	-	NaN	0.00	1.39e-12	4/500	yes	-	1.980	0.00
Avg (successes)	4	2.30e-08	4/200	-	-	2.815	0.00	1.32e-08	4/500	-	-	2.655	0.00
xbar	8	4.52e-07	15/200	yes	-	0.500	0.00	7.38e-07	15/500	yes	-	0.850	0.00
rand1	8	1.24e-08	19/200	yes	-	6.040	0.00	6.11e-07	12/500	yes	-	1.000	0.00
rand2	8	5.86e-07	16/200	yes	-	3.400	0.00	2.33e-07	9/500	yes	-	0.900	0.00
rand3	8	3.01e-07	12/200	yes	-	0.300	0.00	1.34e-07	14/500	yes	-	0.560	0.00
rand4	8	3.38e-08	17/200	yes	-	4.190	0.00	1.33e-07	9/500	yes	-	4.000	0.00
rand5	8	6.89e-07	14/200	yes	-	10.920	0.00	6.05e-07	10/500	yes	-	1.890	0.00
Avg (successes)	8	3.46e-07	16/200	-	-	4.225	0.00	4.09e-07	12/500	-	-	1.533	0.00
xbar	12	6.94e-04	200/200	no	maxit	NaN	0.01	0.00e+00	12/500	yes	-	0.140	0.00
rand1	12	0.00e+00	92/200	yes	-	0.870	0.00	1.11e-04	500/500	no	maxit	NaN	0.01
rand2	12	1.11e-04	200/200	no	maxit	NaN	0.01	0.00e+00	14/500	yes	-	0.570	0.00
rand3	12	0.00e+00	94/200	yes	-	0.950	0.01	0.00e+00	19/500	yes	-	0.000	0.00
rand4	12	0.00e+00	74/200	yes	-	0.770	0.00	0.00e+00	11/500	yes	-	17.260	0.00
rand5	12	0.00e+00	71/200	yes	-	0.350	0.00	0.00e+00	16/500	yes	-	1.710	0.00
Avg (successes)	12	0.00e+00	83/200	-	-	0.735	0.00	0.00e+00	14/500	-	-	3.936	0.00

Table 12: Trig16 – n=2, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.37e-07	3/200	yes	-	3.000	0.00	7.71e-12	4/500	yes	-	2.810	0.00
rand1	4	1.56e-12	7/200	yes	-	2.630	0.00	1.17e-12	6/500	yes	-	2.640	0.00
rand2	4	1.00e-12	4/200	yes	-	NaN	0.00	4.24e-12	4/500	yes	-	2.460	0.00
rand3	4	2.09e-12	6/200	yes	-	2.610	0.00	1.80e-12	6/500	yes	-	2.700	0.00
rand4	4	1.97e-11	3/200	yes	-	3.030	0.00	7.87e-08	3/500	yes	-	3.080	0.00
rand5	4	1.00e-12	4/200	yes	-	NaN	0.00	1.20e-12	4/500	yes	-	1.990	0.00
Avg (successes)	4	2.28e-08	4/200	-	-	2.817	0.00	1.31e-08	4/500	-	-	2.613	0.00
xbar	8	3.18e-07	34/200	yes	-	3.950	0.00	3.90e-07	13/500	yes	-	1.480	0.00
rand1	8	7.08e-07	31/200	yes	-	3.610	0.00	5.42e-07	13/500	yes	-	0.650	0.00
rand2	8	7.50e-07	35/200	yes	-	3.010	0.00	2.76e-07	15/500	yes	-	1.420	0.00
rand3	8	8.26e-07	35/200	yes	-	0.890	0.00	7.34e-07	12/500	yes	-	0.720	0.00
rand4	8	1.88e-07	25/200	yes	-	5.540	0.00	2.16e-07	12/500	yes	-	1.020	0.00
rand5	8	9.05e-07	30/200	yes	-	0.990	0.00	5.81e-07	13/500	yes	-	0.590	0.00
Avg (successes)	8	6.16e-07	32/200	-	-	2.998	0.00	4.56e-07	13/500	-	-	0.980	0.00
xbar	12	0.00e+00	74/200	yes	-	0.640	0.01	0.00e+00	23/500	yes	-	0.250	0.00
rand1	12	1.20e-04	96/200	no	bt/maxit	NaN	0.01	0.00e+00	22/500	yes	-	0.540	0.00
rand2	12	0.00e+00	82/200	yes	-	2.060	0.00	0.00e+00	10/500	yes	-	0.170	0.00
rand3	12	0.00e+00	72/200	yes	-	0.310	0.00	1.00e-04	500/500	no	maxit	NaN	0.02
rand4	12	3.01e-04	200/200	no	maxit	NaN	0.01	0.00e+00	11/500	yes	-	0.010	0.00
rand5	12	9.03e-04	200/200	no	maxit	NaN	0.01	0.00e+00	10/500	yes	-	0.000	0.00
Avg (successes)	12	0.00e+00	76/200	-	-	1.003	0.00	0.00e+00	15/500	-	-	0.194	0.00

Table 13: Trig16 – n=10³, exact: Modified Newton vs Truncated Newton with exact gradient and hessian.

start.pt ID	Modified Newton						Truncated Newton					
	grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4.81e-09	4/200	yes	-	3.030	0.06	2.74e-07	21/500	yes	-	1.490	0.00
rand1	5.81e-07	12/200	yes	-	2.540	0.19	7.61e-07	22/500	yes	-	0.640	0.00
rand2	1.82e-11	13/200	yes	-	1.600	0.21	1.36e-08	21/500	yes	-	1.500	0.00
rand3	1.60e-11	16/200	yes	-	2.240	0.26	4.02e-07	25/500	yes	-	1.460	0.00
rand4	4.46e-11	14/200	yes	-	2.260	0.24	2.10e-09	21/500	yes	-	1.500	0.00
rand5	1.68e-08	16/200	yes	-	3.080	0.27	1.07e-07	21/500	yes	-	1.490	0.00
Avg (successes)	1.00e-07	12/200	-	-	2.458	0.20	2.60e-07	22/500	-	-	1.347	0.00

Table 14: Trig16 – n=10³, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	7.62e-09	4/200	yes	-	2.970	0.07	2.49e-09	22/500	yes	-	1.490	0.00
rand1	4	5.52e-11	14/200	yes	-	1.500	0.24	4.31e-06	500/500	no	maxit	NaN	0.46
rand2	4	1.82e-11	13/200	yes	-	1.600	0.21	1.01e-08	21/500	yes	-	1.540	0.00
rand3	4	1.60e-11	15/200	yes	-	1.870	0.26	5.62e-08	25/500	yes	-	1.490	0.01
rand4	4	6.02e-11	16/200	yes	-	2.980	0.24	6.34e-07	23/500	yes	-	1.320	0.00
rand5	4	1.69e-08	16/200	yes	-	3.080	0.27	1.07e-07	21/500	yes	-	1.490	0.00
Avg (successes)	4	4.11e-09	13/200	-	-	2.333	0.21	1.62e-07	22/500	-	-	1.466	0.00
xbar	8	4.82e-09	4/200	yes	-	3.020	0.06	3.29e-09	20/500	yes	-	1.520	0.00
rand1	8	2.58e-09	16/200	yes	-	3.160	0.28	5.87e-06	500/500	no	maxit	NaN	0.49
rand2	8	1.82e-11	13/200	yes	-	1.600	0.21	1.01e-08	21/500	yes	-	1.540	0.00
rand3	8	6.53e-07	13/200	yes	-	3.130	0.23	2.69e-07	25/500	yes	-	1.500	0.00
rand4	8	6.26e-11	16/200	yes	-	1.420	0.26	1.61e-06	500/500	no	maxit	NaN	0.48
rand5	8	1.35e-08	16/200	yes	-	3.070	0.28	9.60e-08	21/500	yes	-	1.480	0.00
Avg (successes)	8	1.12e-07	13/200	-	-	2.567	0.22	9.46e-08	22/500	-	-	1.510	0.00
xbar	12	4.36e-07	4/200	yes	-	2.460	0.07	9.29e-08	22/500	yes	-	1.440	0.00
rand1	12	8.87e-08	15/200	yes	-	1.230	0.25	1.94e-09	22/500	yes	-	1.500	0.00
rand2	12	7.04e-07	18/200	yes	-	1.000	0.31	2.37e-08	21/500	yes	-	1.520	0.00
rand3	12	5.50e-09	15/200	yes	-	1.610	0.26	2.48e-08	25/500	yes	-	1.460	0.00
rand4	12	6.96e-07	20/200	yes	-	1.000	0.35	8.03e-08	21/500	yes	-	1.500	0.00
rand5	12	2.83e-08	14/200	yes	-	1.170	0.25	1.13e-08	21/500	yes	-	1.510	0.00
Avg (successes)	12	3.26e-07	14/200	-	-	1.412	0.25	3.92e-08	22/500	-	-	1.488	0.00

Table 15: Trig16 – n=10³, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	4.81e-09	4/200	yes	-	3.030	0.08	7.49e-09	22/500	yes	-	1.480	0.00
rand1	4	1.40e-08	19/200	yes	-	2.910	0.34	4.32e-09	22/500	yes	-	1.500	0.00
rand2	4	5.24e-10	14/200	yes	-	2.420	0.25	1.01e-08	21/500	yes	-	1.540	0.00
rand3	4	4.69e-08	14/200	yes	-	2.990	0.24	4.00e-07	25/500	yes	-	1.460	0.01
rand4	4	1.54e-09	15/200	yes	-	2.940	0.27	8.62e-07	21/500	yes	-	0.650	0.00
rand5	4	1.78e-08	16/200	yes	-	3.070	0.26	8.96e-08	21/500	yes	-	1.480	0.00
Avg (successes)	4	1.43e-08	14/200	-	-	2.893	0.24	2.29e-07	22/500	-	-	1.352	0.00
xbar	8	1.38e-08	4/200	yes	-	2.890	0.07	7.93e-07	19/500	yes	-	1.490	0.01
rand1	8	6.34e-07	14/200	yes	-	3.120	0.22	2.96e-06	500/500	no	maxit	NaN	0.51
rand2	8	2.37e-08	12/200	yes	-	2.470	0.22	2.28e-09	24/500	yes	-	17.650	0.01
rand3	8	4.83e-09	18/200	yes	-	2.930	0.33	4.30e-08	25/500	yes	-	1.500	0.01
rand4	8	6.96e-11	16/200	yes	-	1.430	0.29	2.14e-09	21/500	yes	-	1.510	0.00
rand5	8	1.58e-08	16/200	yes	-	3.080	0.27	9.49e-08	21/500	yes	-	1.490	0.00
Avg (successes)	8	1.15e-07	13/200	-	-	2.653	0.23	1.87e-07	22/500	-	-	4.728	0.01
xbar	12	1.65e-09	5/200	yes	-	1.190	0.08	3.00e-07	21/500	yes	-	1.460	0.01
rand1	12	1.61e-08	13/200	yes	-	1.430	0.22	4.07e-09	22/500	yes	-	1.530	0.00
rand2	12	4.82e-07	12/200	yes	-	1.930	0.20	1.26e-07	20/500	yes	-	1.460	0.00
rand3	12	7.22e-08	17/200	yes	-	2.500	0.29	2.06e-08	27/500	yes	-	1.480	0.01
rand4	12	1.06e-07	13/200	yes	-	2.680	0.24	6.08e-08	21/500	yes	-	1.510	0.00
rand5	12	1.47e-10	17/200	yes	-	1.430	0.39	2.77e-07	21/500	yes	-	1.470	0.00
Avg (successes)	12	1.13e-07	13/200	-	-	1.860	0.24	1.31e-07	22/500	-	-	1.485	0.00

Table 16: Trig16 – n=10³, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.45e-08	4/200	yes	-	2.890	0.07	8.46e-08	20/500	yes	-	1.490	0.00
rand1	4	5.82e-09	17/200	yes	-	1.980	0.26	7.15e-09	22/500	yes	-	1.420	0.00
rand2	4	2.15e-06	20/200	no	bt/maxit	NaN	0.31	1.15e-08	21/500	yes	-	1.530	0.01
rand3	4	5.93e-09	15/200	yes	-	1.650	0.29	2.14e-07	25/500	yes	-	1.470	0.01
rand4	4	3.24e-08	17/200	yes	-	3.080	0.28	6.17e-09	22/500	yes	-	0.700	0.01
rand5	4	2.88e-08	16/200	yes	-	3.080	0.24	1.06e-07	21/500	yes	-	1.500	0.01
Avg (successes)	4	1.75e-08	14/200	-	-	2.536	0.23	7.16e-08	22/500	-	-	1.352	0.01
xbar	8	1.98e+00	200/200	no	maxit	NaN	3.36	7.69e-04	500/500	no	maxit	NaN	0.19
rand1	8	8.90e-01	200/200	no	maxit	NaN	3.38	6.01e-04	500/500	no	maxit	NaN	0.19
rand2	8	2.27e+00	200/200	no	maxit	NaN	3.47	3.47e-04	500/500	no	maxit	NaN	0.18
rand3	8	2.47e-01	200/200	no	maxit	NaN	3.41	8.88e-04	500/500	no	maxit	NaN	0.18
rand4	8	5.43e-01	200/200	no	maxit	NaN	3.34	1.96e-04	500/500	no	maxit	NaN	0.18
rand5	8	1.13e+00	200/200	no	maxit	NaN	3.30	8.07e-05	492/500	no	bt/maxit	NaN	0.22
Avg (successes)	8	-	-	-	-	-	-	-	-	-	-	-	-
xbar	12	1.19e+04	200/200	no	maxit	NaN	2.28	8.40e+00	500/500	no	maxit	NaN	0.14
rand1	12	1.05e+04	200/200	no	maxit	NaN	2.25	4.74e+00	500/500	no	maxit	NaN	0.15
rand2	12	9.60e+03	200/200	no	maxit	NaN	2.26	5.97e+00	500/500	no	maxit	NaN	0.14
rand3	12	1.01e+04	200/200	no	maxit	NaN	2.27	1.02e+01	500/500	no	maxit	NaN	0.15
rand4	12	1.02e+04	200/200	no	maxit	NaN	2.26	5.35e+00	500/500	no	maxit	NaN	0.14
rand5	12	1.02e+04	200/200	no	maxit	NaN	2.24	4.49e+00	500/500	no	maxit	NaN	0.14
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 17: Trig16 – n=10³, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	2.30e-06	7/200	no	bt/maxit	NaN	0.12	2.16e-06	500/500	no	maxit	NaN	0.38
rand1	4	2.14e-06	28/200	no	bt/maxit	NaN	0.46	1.48e-06	500/500	no	maxit	NaN	0.43
rand2	4	1.69e-06	23/200	no	bt/maxit	NaN	0.35	2.00e-06	500/500	no	maxit	NaN	0.38
rand3	4	2.08e-06	22/200	no	bt/maxit	NaN	0.40	1.44e-06	500/500	no	maxit	NaN	0.40
rand4	4	1.79e-06	28/200	no	bt/maxit	NaN	0.48	1.87e-06	500/500	no	maxit	NaN	0.49
rand5	4	1.72e-06	23/200	no	bt/maxit	NaN	0.40	4.83e+00	18/500	-	bt/maxit	NaN	0.01
Avg (successes)	4	-	-	-	-	-	-	-	-	-	-	-	-
xbar	8	6.05e+03	200/200	no	maxit	NaN	3.49	1.29e-01	500/500	no	maxit	NaN	0.14
rand1	8	8.28e+03	200/200	no	maxit	NaN	3.39	6.98e-01	500/500	no	maxit	NaN	0.15
rand2	8	8.86e+03	200/200	no	maxit	NaN	3.48	4.43e-01	500/500	no	maxit	NaN	0.14
rand3	8	8.01e+03	200/200	no	maxit	NaN	3.51	3.75e-01	500/500	no	maxit	NaN	0.13
rand4	8	8.29e+03	200/200	no	maxit	NaN	2.38	3.81e-01	500/500	no	maxit	NaN	0.13
rand5	8	8.74e+03	200/200	no	maxit	NaN	2.26	2.13e-01	500/500	no	maxit	NaN	0.14
Avg (successes)	8	-	-	-	-	-	-	-	-	-	-	-	-
xbar	12	1.45e+04	78/200	no	bt/maxit	NaN	0.99	4.95e+01	500/500	no	maxit	NaN	0.23
rand1	12	1.42e+04	150/200	no	bt/maxit	NaN	2.02	4.93e+01	500/500	no	maxit	NaN	0.20
rand2	12	1.41e+04	113/200	no	bt/maxit	NaN	1.67	4.92e+01	500/500	no	maxit	NaN	0.16
rand3	12	1.41e+04	200/200	no	maxit	NaN	2.64	4.85e+01	500/500	no	maxit	NaN	0.18
rand4	12	1.43e+04	107/200	no	bt/maxit	NaN	1.43	4.88e+01	500/500	no	maxit	NaN	0.21
rand5	12	1.39e+04	200/200	no	maxit	NaN	2.63	4.87e+01	500/500	no	maxit	NaN	0.25
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 18: Trig16 – $n=10^4$, exact: Modified Newton vs Truncated Newton with exact gradient and hessian.

start.pt ID	Modified Newton						Truncated Newton					
	grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	$1.67e-07$	4/200	yes	-	3.030	0.01	$1.36e-05$	500/500	no	maxit	NaN	7.99
rand1	$8.98e-09$	19/200	yes	-	1.450	0.03	$1.14e-06$	500/500	no	maxit	NaN	7.79
rand2	$3.08e-09$	28/200	yes	-	1.890	0.05	$8.51e-07$	29/500	yes	-	1.500	0.07
rand3	$5.71e-09$	21/200	yes	-	2.630	0.04	$3.50e-08$	26/500	yes	-	1.470	0.07
rand4	$7.53e-09$	19/200	yes	-	1.980	0.04	$3.88e-07$	26/500	yes	-	1.500	0.06
rand5	$1.55e-08$	23/200	yes	-	3.030	0.04	$2.23e-08$	27/500	yes	-	1.490	0.09
Avg (successes)	$3.46e-08$	19/200	-	-	2.335	0.04	$3.24e-07$	27/500	-	-	1.490	0.07

Table 19: Trig16 – $n=10^4$, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	$2.59e-07$	4/200	yes	-	2.970	0.01	$5.71e-08$	24/500	yes	-	1.490	0.08
rand1	4	$5.96e-07$	20/200	yes	-	3.010	0.04	$2.63e-06$	500/500	no	maxit	NaN	7.51
rand2	4	$2.58e-09$	23/200	yes	-	2.870	0.05	$7.52e-08$	29/500	yes	-	1.490	0.09
rand3	4	$1.98e-08$	20/200	yes	-	3.030	0.04	$2.82e-07$	26/500	yes	-	1.500	0.07
rand4	4	$1.73e-08$	21/200	yes	-	2.900	0.04	$1.52e-07$	27/500	yes	-	0.900	0.09
rand5	4	$3.17e-09$	20/200	yes	-	2.010	0.04	$1.44e-05$	500/500	no	maxit	NaN	7.63
Avg (successes)	4	$1.50e-07$	18/200	-	-	2.798	0.04	$1.42e-07$	26/500	-	-	1.345	0.08
xbar	8	$1.67e-07$	4/200	yes	-	3.030	0.01	$7.09e-06$	500/500	no	maxit	NaN	7.92
rand1	8	$7.49e-09$	20/200	yes	-	1.270	0.04	$1.14e-08$	29/500	yes	-	1.500	0.08
rand2	8	$2.06e-09$	26/200	yes	-	1.210	0.05	$7.31e-06$	500/500	no	maxit	NaN	7.47
rand3	8	$8.48e-08$	18/200	yes	-	3.080	0.04	$3.52e-08$	26/500	yes	-	1.520	0.07
rand4	8	$3.20e-09$	24/200	yes	-	1.870	0.05	$9.83e-08$	27/500	yes	-	1.090	0.08
rand5	8	$3.17e-09$	20/200	yes	-	1.090	0.04	$1.02e-05$	500/500	no	maxit	NaN	7.10
Avg (successes)	8	$4.46e-08$	19/200	-	-	1.925	0.04	$4.83e-08$	27/500	-	-	1.370	0.08
xbar	12	$6.39e-07$	5/200	yes	-	0.610	0.01	$1.95e-07$	24/500	yes	-	1.510	0.07
rand1	12	$4.60e-07$	34/200	yes	-	1.000	0.07	$6.26e-07$	28/500	yes	-	1.470	0.08
rand2	12	$8.89e-07$	35/200	yes	-	1.160	0.07	$4.94e-08$	30/500	yes	-	0.890	0.11
rand3	12	$5.90e-08$	18/200	yes	-	1.640	0.04	$1.43e-06$	500/500	no	maxit	NaN	7.88
rand4	12	$6.01e-07$	37/200	yes	-	0.700	0.08	$5.37e-09$	27/500	yes	-	1.490	0.08
rand5	12	$6.83e-07$	33/200	yes	-	0.840	0.06	$4.97e-08$	34/500	yes	-	1.080	0.10
Avg (successes)	12	$5.55e-07$	27/200	-	-	0.992	0.06	$1.85e-07$	29/500	-	-	1.288	0.09

Table 20: Trig16 – $n=10^4$, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	$1.67e-07$	4/200	yes	-	3.030	0.01	$2.46e-05$	500/500	no	maxit	NaN	8.37
rand1	4	$8.33e-09$	18/200	yes	-	2.520	0.03	$9.06e-08$	29/500	yes	-	1.090	0.11
rand2	4	$6.53e-07$	20/200	yes	-	2.840	0.04	$3.32e-07$	29/500	yes	-	1.500	0.11
rand3	4	$8.96e-08$	17/200	yes	-	3.090	0.03	$2.92e-07$	26/500	yes	-	1.490	0.09
rand4	4	$1.24e-09$	20/200	yes	-	2.260	0.04	$1.12e-07$	27/500	yes	-	1.100	0.11
rand5	4	$9.98e-07$	19/200	yes	-	0.710	0.04	$7.90e-06$	500/500	no	maxit	NaN	8.65
Avg (successes)	4	$3.20e-07$	16/200	-	-	2.408	0.03	$2.07e-07$	28/500	-	-	1.295	0.10
xbar	8	$4.14e-07$	4/200	yes	-	2.910	0.01	$1.37e-05$	500/500	no	maxit	NaN	7.41
rand1	8	$8.22e-09$	26/200	yes	-	1.900	0.06	$1.72e-06$	500/500	no	maxit	NaN	7.19
rand2	8	$7.40e-07$	17/200	yes	-	2.930	0.03	$8.54e-07$	29/500	yes	-	1.500	0.08
rand3	8	$5.70e-09$	20/200	yes	-	1.380	0.04	$1.48e-05$	500/500	no	maxit	NaN	7.08
rand4	8	$7.40e-09$	19/200	yes	-	1.850	0.04	$1.07e-07$	26/500	yes	-	1.510	0.07
rand5	8	$3.18e-09$	19/200	yes	-	2.650	0.04	$5.68e-08$	27/500	yes	-	1.490	0.09
Avg (successes)	8	$1.96e-07$	18/200	-	-	2.270	0.04	$3.39e-07$	27/500	-	-	1.500	0.08
xbar	12	$1.75e-09$	5/200	yes	-	1.210	0.01	$1.72e-05$	500/500	no	maxit	NaN	7.01
rand1	12	$3.18e-08$	23/200	yes	-	1.910	0.05	$1.93e-07$	27/500	yes	-	1.500	0.09
rand2	12	$2.14e-08$	20/200	yes	-	1.590	0.04	$4.31e-08$	29/500	yes	-	1.500	0.09
rand3	12	$4.81e-07$	17/200	yes	-	3.380	0.03	$1.54e-07$	26/500	yes	-	1.500	0.08
rand4	12	$1.25e-08$	21/200	yes	-	1.430	0.04	$1.68e-08$	26/500	yes	-	1.490	0.08
rand5	12	$4.25e-08$	21/200	yes	-	1.960	0.04	$8.49e-07$	29/500	yes	-	0.650	0.09
Avg (successes)	12	$9.85e-08$	18/200	-	-	1.913	0.04	$2.51e-07$	27/500	-	-	1.328	0.09

Table 21: Trig16 – $n=10^4$, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	$4.82e-07$	4/200	yes	-	2.900	0.01	$1.96e-07$	24/500	yes	-	1.360	0.07
rand1	4	$1.88e-07$	22/200	yes	-	2.060	0.05	$1.85e-07$	29/500	yes	-	1.140	0.10
rand2	4	$1.87e-07$	18/200	yes	-	1.300	0.04	$2.18e-07$	30/500	yes	-	1.010	0.09
rand3	4	$1.87e-07$	19/200	yes	-	1.830	0.04	$2.31e-07$	27/500	yes	-	0.850	0.09
rand4	4	$1.87e-07$	22/200	yes	-	2.310	0.05	$7.42e-06$	500/500	no	maxit	NaN	7.11
rand5	4	$1.87e-07$	27/200	yes	-	1.090	0.06	$1.87e-05$	500/500	no	maxit	NaN	6.65
Avg (successes)	4	$2.36e-07$	19/200	-	-	1.915	0.04	$2.07e-07$	28/500	-	-	1.090	0.09
xbar	8	$2.52e+02$	200/200	no	maxit	NaN	0.49	$1.77e+00$	500/500	no	maxit	NaN	1.23
rand1	8	$2.98e+02$	200/200	no	maxit	NaN	0.47	$1.72e+00$	500/500	no	maxit	NaN	1.35
rand2	8	$3.12e+02$	200/200	no	maxit	NaN	0.44	$9.00e+00$	500/500	no	maxit	NaN	1.24
rand3	8	$2.98e+02$	200/200	no	maxit	NaN	0.49	$7.83e-01$	500/500	no	maxit	NaN	1.24
rand4	8	$2.96e+02$	200/200	no	maxit	NaN	0.51	$8.91e-01$	500/500	no	maxit	NaN	1.20
rand5	8	$2.96e+02$	200/200	no	maxit	NaN	0.46	$4.51e+00$	500/500	no	maxit	NaN	1.22
Avg (successes)	8	-	-	-	-	-	-	-	-	-	-	-	-
xbar	12	$4.57e+05$	200/200	no	maxit	NaN	0.98	$1.43e+02$	500/500	no	maxit	NaN	1.13
rand1	12	$4.21e+05$	200/200	no	maxit	NaN	0.59	$1.46e+02$	500/500	no	maxit	NaN	1.22
rand2	12	$4.18e+05$	200/200	no	maxit	NaN	0.59	$8.17e+01$	500/500	no	maxit	NaN	1.11
rand3	12	$4.21e+05$	200/200	no	maxit	NaN	0.59	$8.03e+01$	500/500	no	maxit	NaN	1.18
rand4	12	$4.24e+05$	200/200	no	maxit	NaN	0.58	$8.36e+01$	500/500	no	maxit	NaN	1.30
rand5	12	$4.20e+05$	200/200	no	maxit	NaN	0.61	$1.00e+02$	500/500	no	maxit	NaN	1.17
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 22: Trig16 – n=10⁴, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	5.37e-04	15/200	no	bt/maxit	NaN	0.05	4.09e-04	500/500	no	maxit	NaN	3.35
rand1	4	3.73e+04	200/200	no	maxit	NaN	0.77	5.48e-04	500/500	no	maxit	NaN	2.45
rand2	4	3.25e+04	200/200	no	maxit	NaN	0.74	4.20e-04	500/500	no	maxit	NaN	3.60
rand3	4	1.97e+04	200/200	no	maxit	NaN	0.78	6.87e+00	73/500	no	bt/maxit	NaN	0.19
rand4	4	3.98e+04	200/200	no	maxit	NaN	0.76	8.64e+00	67/500	no	bt/maxit	NaN	0.18
rand5	4	2.97e+04	200/200	no	maxit	NaN	1.08	4.44e-04	500/500	no	maxit	NaN	2.95
Avg (successes)	4	-	-	-	-	-	-	-	-	-	-	-	-
xbar	8	2.56e+05	200/200	no	maxit	NaN	0.50	7.95e+01	500/500	no	maxit	NaN	1.14
rand1	8	4.45e+05	200/200	no	maxit	NaN	0.91	1.26e+02	500/500	no	maxit	NaN	1.26
rand2	8	4.39e+05	200/200	no	maxit	NaN	0.80	1.01e+02	500/500	no	maxit	NaN	1.13
rand3	8	4.35e+05	200/200	no	maxit	NaN	0.66	7.85e+01	500/500	no	maxit	NaN	1.14
rand4	8	4.45e+05	200/200	no	maxit	NaN	0.95	1.03e+02	500/500	no	maxit	NaN	1.23
rand5	8	4.43e+05	200/200	no	maxit	NaN	0.94	1.61e+02	500/500	no	maxit	NaN	1.17
Avg (successes)	8	-	-	-	-	-	-	-	-	-	-	-	-
xbar	12	4.78e+05	119/200	no	bt/maxit	NaN	0.63	7.21e+02	500/500	no	maxit	NaN	0.99
rand1	12	4.46e+05	200/200	no	maxit	NaN	1.60	6.69e+02	500/500	no	maxit	NaN	1.14
rand2	12	4.43e+05	200/200	no	maxit	NaN	1.49	6.80e+02	500/500	no	maxit	NaN	1.14
rand3	12	4.43e+05	200/200	no	maxit	NaN	1.51	7.93e+02	500/500	no	maxit	NaN	1.10
rand4	12	4.48e+05	7/200	no	bt/maxit	NaN	0.05	7.60e+02	500/500	no	maxit	NaN	1.03
rand5	12	4.45e+05	200/200	no	maxit	NaN	1.60	8.07e+02	500/500	no	maxit	NaN	1.25
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 23: Trig16 – n=10⁵, exact: Modified Newton vs Truncated Newton with exact gradient and hessian.

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.09e-13	8/200	yes	-	NaN	0.35	1.04e-05	500/500	no	maxit	NaN	225.23
rand1	4	1.74e-05	200/200	no	maxit	NaN	4.57	1.95e-06	500/500	no	maxit	NaN	156.54
rand2	4	3.90e-05	200/200	no	maxit	NaN	4.38	1.52e-06	500/500	no	maxit	NaN	258.56
rand3	4	3.42e-05	200/200	no	maxit	NaN	4.43	1.55e-04	500/500	no	maxit	NaN	160.43
rand4	4	6.82e-05	200/200	no	maxit	NaN	4.52	3.89e-04	500/500	no	maxit	NaN	136.00
rand5	4	9.65e-05	200/200	no	maxit	NaN	4.44	8.98e-08	38/500	yes	-	1.500	2.11
Avg (successes)	4	1.09e-13	8/200	-	-	NaN	0.35	8.98e-08	38/500	-	-	1.500	2.11

Table 24: Trig16 – n=10⁵, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.73e-08	9/200	yes	-	3.080	0.25	2.63e-05	500/500	no	maxit	NaN	178.14
rand1	4	1.84e-05	200/200	no	maxit	NaN	4.22	9.04e-05	500/500	no	maxit	NaN	169.59
rand2	4	3.16e-05	200/200	no	maxit	NaN	4.47	1.16e-05	500/500	no	maxit	NaN	114.08
rand3	4	3.65e-05	200/200	no	maxit	NaN	4.62	2.26e-05	500/500	no	maxit	NaN	141.94
rand4	4	6.84e-05	200/200	no	maxit	NaN	4.94	1.93e-05	500/500	no	maxit	NaN	133.47
rand5	4	8.84e-05	200/200	no	maxit	NaN	4.93	8.21e-07	41/500	yes	-	0.740	3.96
Avg (successes)	4	1.73e-08	9/200	-	-	3.080	0.25	8.21e-07	41/500	-	-	0.740	3.96
xbar	8	5.19e-12	10/200	yes	-	1.840	0.20	1.47e-05	500/500	no	maxit	NaN	156.93
rand1	8	2.81e-05	200/200	no	maxit	NaN	4.56	6.69e-06	500/500	no	maxit	NaN	153.10
rand2	8	3.55e-05	200/200	no	maxit	NaN	4.57	1.21e-09	38/500	yes	-	1.490	2.83
rand3	8	3.43e-05	200/200	no	maxit	NaN	5.06	2.18e-07	34/500	yes	-	1.500	2.28
rand4	8	6.80e-05	200/200	no	maxit	NaN	7.97	1.00e-04	500/500	no	maxit	NaN	179.51
rand5	8	8.78e-05	200/200	no	maxit	NaN	5.22	1.24e-04	500/500	no	maxit	NaN	130.85
Avg (successes)	8	5.19e-12	10/200	-	-	1.840	0.20	1.10e-07	36/500	-	-	1.495	2.55
xbar	12	3.77e-07	6/200	yes	-	0.720	0.34	3.10e-08	28/500	yes	-	1.500	1.43
rand1	12	3.44e+04	22/200	no	bt/maxit	NaN	1.71	5.32e-05	500/500	no	maxit	NaN	123.44
rand2	12	1.62e+05	18/200	no	bt/maxit	NaN	0.99	6.45e-08	42/500	yes	-	1.490	2.18
rand3	12	1.48e+05	18/200	no	bt/maxit	NaN	1.07	2.08e-05	500/500	no	maxit	NaN	199.25
rand4	12	6.18e+04	25/200	no	bt/maxit	NaN	2.02	2.36e-09	40/500	yes	-	1.490	3.82
rand5	12	3.22e+04	25/200	no	bt/maxit	NaN	1.99	3.11e-09	38/500	yes	-	1.490	1.64
Avg (successes)	12	3.77e-07	6/200	-	-	0.720	0.34	2.52e-08	37/500	-	-	1.493	2.27

Table 25: Trig16 – n=10⁵, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	3.70e-09	9/200	yes	-	1.350	0.28	1.80e-05	500/500	no	maxit	NaN	225.55
rand1	4	1.52e+05	21/200	no	bt/maxit	NaN	0.97	2.21e-05	500/500	no	maxit	NaN	148.55
rand2	4	2.39e+04	24/200	no	bt/maxit	NaN	0.79	6.06e-06	500/500	no	maxit	NaN	150.54
rand3	4	2.50e+04	19/200	no	bt/maxit	NaN	0.64	1.96e-06	500/500	no	maxit	NaN	230.05
rand4	4	3.01e+04	20/200	no	bt/maxit	NaN	0.67	3.04e-09	40/500	yes	-	1.490	1.74
rand5	4	2.51e+04	29/200	no	bt/maxit	NaN	0.99	1.16e-05	500/500	no	maxit	NaN	138.17
Avg (successes)	4	3.70e-09	9/200	-	-	1.350	0.28	3.04e-09	40/500	-	-	1.490	1.74
xbar	8	6.69e-07	8/200	yes	-	3.010	0.22	2.76e-06	500/500	no	maxit	NaN	168.28
rand1	8	1.89e-05	200/200	no	maxit	NaN	4.58	9.16e-05	500/500	no	maxit	NaN	129.66
rand2	8	3.20e-05	200/200	no	maxit	NaN	4.85	2.63e-08	41/500	yes	-	1.490	3.57
rand3	8	3.48e-05	200/200	no	maxit	NaN	5.54	6.21e-06	500/500	no	maxit	NaN	204.54
rand4	8	6.66e-05	200/200	no	maxit	NaN	9.33	5.73e-07	41/500	yes	-	0.650	1.93
rand5	8	8.76e-05	200/200	no	maxit	NaN	8.85	1.99e-06	500/500	no	maxit	NaN	176.28
Avg (successes)	8	6.69e-07	8/200	-	-	3.010	0.22	3.00e-07	41/500	-	-	1.070	2.75
xbar	12	1.05e-10	6/200	yes	-	1.050	0.33	8.21e-04	500/500	no	maxit	NaN	97.23
rand1	12	4.32e-05	200/200	no	maxit	NaN	9.35	4.95e-05	500/500	no	maxit	NaN	143.13
rand2	12	3.17e-05	200/200	no	maxit	NaN	7.54	9.77e-07	40/500	yes	-	1.510	1.56
rand3	12	3.45e-05	200/200	no	maxit	NaN	9.22	7.99e-05	500/500	no	maxit	NaN	114.95
rand4	12	7.38e-05	200/200	no	maxit	NaN	9.39	8.19e-07	37/500	yes	-	1.500	1.55
rand5	12	8.98e-05	200/200	no	maxit	NaN	10.26	2.66e-05	500/500	no	maxit	NaN	169.55
Avg (successes)	12	1.05e-10	6/200	-	-	1.050	0.33	8.98e-07	38/500	-	-	1.505	1.56

Table 26: Trig16 – n=10⁵, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	5.81e-06	16/200	no	bt/maxit	NaN	1.72	2.85e-05	500/500	no	maxit	NaN	170.77
rand1	4	2.37e-05	44/200	no	bt/maxit	NaN	3.31	4.81e-06	500/500	no	maxit	NaN	129.50
rand2	4	3.16e-05	54/200	no	bt/maxit	NaN	3.85	4.14e-06	500/500	no	maxit	NaN	111.02
rand3	4	4.37e-05	40/200	no	bt/maxit	NaN	3.97	4.69e-06	500/500	no	maxit	NaN	109.89
rand4	4	7.07e-05	42/200	no	bt/maxit	NaN	2.76	1.02e-04	500/500	no	maxit	NaN	178.39
rand5	4	9.11e-05	42/200	no	bt/maxit	NaN	3.13	4.43e-06	500/500	no	maxit	NaN	16.21
Avg (successes)	4	-	-	-	-	-	-	-	-	-	-	-	-
xbar	8	2.33e+06	27/200	no	bt/maxit	NaN	3.08	4.34e+01	500/500	no	maxit	NaN	7.27
rand1	8	1.28e+07	7/200	no	bt/maxit	NaN	1.12	6.65e+01	500/500	no	maxit	NaN	7.21
rand2	8	1.21e+07	9/200	no	bt/maxit	NaN	1.32	3.24e+01	500/500	no	maxit	NaN	8.26
rand3	8	1.20e+07	11/200	no	bt/maxit	NaN	1.57	4.68e+01	500/500	no	maxit	NaN	8.59
rand4	8	1.22e+07	7/200	no	bt/maxit	NaN	0.88	2.20e+02	500/500	no	maxit	NaN	11.28
rand5	8	1.24e+07	7/200	no	bt/maxit	NaN	0.93	3.04e+01	500/500	no	maxit	NaN	7.04
Avg (successes)	8	-	-	-	-	-	-	-	-	-	-	-	-
xbar	12	1.54e+07	1/200	no	bt/maxit	NaN	0.27	3.75e+03	500/500	no	maxit	NaN	7.83
rand1	12	1.40e+07	200/200	no	maxit	NaN	41.70	3.60e+03	500/500	no	maxit	NaN	7.99
rand2	12	1.40e+07	115/200	no	bt/maxit	NaN	23.78	3.89e+03	500/500	no	maxit	NaN	7.95
rand3	12	1.40e+07	200/200	no	maxit	NaN	20.89	3.81e+03	500/500	no	maxit	NaN	11.83
rand4	12	1.39e+07	200/200	no	maxit	NaN	18.57	3.41e+03	500/500	no	maxit	NaN	10.26
rand5	12	1.40e+07	200/200	no	maxit	NaN	18.92	3.67e+03	500/500	no	maxit	NaN	7.69
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 27: Trig16 – n=10⁵, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.75e+03	4/200	no	bt/maxit	NaN	0.36	7.22e+01	500/500	no	maxit	NaN	8.10
rand1	4	1.81e+06	49/200	no	bt/maxit	NaN	3.18	7.59e+01	500/500	no	maxit	NaN	5.79
rand2	4	2.10e+06	44/200	no	bt/maxit	NaN	3.00	6.38e+01	500/500	no	maxit	NaN	7.12
rand3	4	1.70e+06	105/200	no	bt/maxit	NaN	6.42	3.15e+02	500/500	no	maxit	NaN	5.89
rand4	4	1.69e+06	124/200	no	bt/maxit	NaN	7.28	8.19e+01	500/500	no	maxit	NaN	8.28
rand5	4	2.09e+06	70/200	no	bt/maxit	NaN	4.33	8.99e+01	500/500	no	maxit	NaN	8.47
Avg (successes)	4	-	-	-	-	-	-	-	-	-	-	-	-
xbar	8	1.00e+07	2/200	no	bt/maxit	NaN	0.28	3.56e+03	500/500	no	maxit	NaN	11.52
rand1	8	1.41e+07	1/200	no	bt/maxit	NaN	0.25	4.72e+03	500/500	no	maxit	NaN	13.05
rand2	8	1.41e+07	1/200	no	bt/maxit	NaN	0.24	4.54e+03	500/500	no	maxit	NaN	9.66
rand3	8	1.41e+07	1/200	no	bt/maxit	NaN	0.25	2.53e+03	500/500	no	maxit	NaN	8.30
rand4	8	1.40e+07	2/200	no	bt/maxit	NaN	0.33	5.10e+03	500/500	no	maxit	NaN	8.47
rand5	8	1.41e+07	1/200	no	bt/maxit	NaN	0.26	6.14e+03	500/500	no	maxit	NaN	14.13
Avg (successes)	8	-	-	-	-	-	-	-	-	-	-	-	-
xbar	12	1.54e+07	1/200	no	bt/maxit	NaN	0.10	3.22e+04	500/500	no	maxit	NaN	6.63
rand1	12	1.41e+07	200/200	no	maxit	NaN	8.76	4.34e+04	500/500	no	maxit	NaN	8.93
rand2	12	1.41e+07	200/200	no	maxit	NaN	11.69	1.98e+04	500/500	no	maxit	NaN	11.91
rand3	12	1.41e+07	200/200	no	maxit	NaN	8.37	2.76e+04	500/500	no	maxit	NaN	8.22
rand4	12	1.40e+07	200/200	no	maxit	NaN	8.84	4.41e+04	500/500	no	maxit	NaN	8.43
rand5	12	1.41e+07	200/200	no	maxit	NaN	8.36	4.44e+04	500/500	no	maxit	NaN	8.79
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

D.2 Broy31

Table 28: Broyden31 – n=2, exact: Modified Newton vs Truncated Newton with exact gradient and hessian.

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.32e-07	5/50	yes	-	1.880	0.13	6.17e-11	6/50	yes	-	1.940	0.13
rand1	4	1.78e-13	8/50	yes	-	NaN	0.02	9.93e-07	7/50	yes	-	2.140	0.00
rand2	4	1.69e-07	5/50	yes	-	1.870	0.01	1.30e-09	6/50	yes	-	1.920	0.00
rand3	4	5.75e-07	7/50	yes	-	1.850	0.01	3.99e-08	7/50	yes	-	2.190	0.00
rand4	4	8.25e-11	6/50	yes	-	1.940	0.02	9.30e-08	6/50	yes	-	1.880	0.00
rand5	4	5.34e-08	6/50	yes	-	1.890	0.00	1.69e-10	7/50	yes	-	1.930	0.00
Avg (successes)	4	1.55e-07	6/50	-	-	1.886	0.09	1.88e-07	6/50	-	-	2.000	0.02

Table 29: Broyden31 – n=2, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.32e-07	5/50	yes	-	1.880	0.13	6.18e-11	6/50	yes	-	1.940	0.05
rand1	4	1.87e-13	8/50	yes	-	NaN	0.01	9.93e-07	7/50	yes	-	2.140	0.00
rand2	4	1.69e-07	5/50	yes	-	1.870	0.00	1.30e-09	6/50	yes	-	1.920	0.00
rand3	4	5.75e-07	7/50	yes	-	1.850	0.00	3.99e-08	7/50	yes	-	2.190	0.00
rand4	4	8.27e-11	6/50	yes	-	1.940	0.01	9.30e-08	6/50	yes	-	1.880	0.00
rand5	4	5.34e-08	6/50	yes	-	1.890	0.00	1.69e-10	7/50	yes	-	1.930	0.00
Avg (successes)	4	1.55e-07	6/50	-	-	1.886	0.03	1.88e-07	6/50	-	-	2.000	0.01
xbar	8	1.32e-07	5/50	yes	-	1.880	0.00	6.17e-11	6/50	yes	-	1.940	0.00
rand1	8	1.79e-13	8/50	yes	-	NaN	0.01	9.93e-07	7/50	yes	-	2.140	0.00
rand2	8	1.69e-07	5/50	yes	-	1.870	0.00	1.30e-09	6/50	yes	-	1.920	0.00
rand3	8	5.75e-07	7/50	yes	-	1.850	0.01	3.99e-08	7/50	yes	-	2.190	0.00
rand4	8	8.25e-11	6/50	yes	-	1.940	0.01	9.30e-08	6/50	yes	-	1.880	0.00
rand5	8	5.34e-08	6/50	yes	-	1.890	0.01	1.69e-10	7/50	yes	-	1.930	0.00
Avg (successes)	8	1.55e-07	6/50	-	-	1.886	0.01	1.88e-07	6/50	-	-	2.000	0.00
xbar	12	1.05e-07	5/50	yes	-	1.910	0.01	1.16e-09	6/50	yes	-	1.680	0.00
rand1	12	3.65e-11	8/50	yes	-	1.560	0.01	9.45e-07	7/50	yes	-	2.140	0.00
rand2	12	1.40e-07	5/50	yes	-	1.890	0.00	3.44e-09	6/50	yes	-	1.820	0.00
rand3	12	5.16e-07	7/50	yes	-	1.870	0.01	2.21e-08	7/50	yes	-	2.280	0.00
rand4	12	8.02e-10	6/50	yes	-	1.740	0.00	1.08e-07	6/50	yes	-	1.860	0.00
rand5	12	5.64e-08	6/50	yes	-	1.880	0.01	8.93e-10	7/50	yes	-	1.780	0.00
Avg (successes)	12	1.36e-07	6/50	-	-	1.808	0.01	1.80e-07	6/50	-	-	1.927	0.00

Table 30: Broyden31 – n=2, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.32e-07	5/50	yes	-	1.880	0.01	6.17e-11	6/50	yes	-	1.940	0.00
rand1	4	1.80e-13	8/50	yes	-	NaN	0.01	9.93e-07	7/50	yes	-	2.140	0.00
rand2	4	1.69e-07	5/50	yes	-	1.870	0.00	1.30e-09	6/50	yes	-	1.920	0.00
rand3	4	5.75e-07	7/50	yes	-	1.850	0.01	3.99e-08	7/50	yes	-	2.190	0.00
rand4	4	8.26e-11	6/50	yes	-	1.940	0.00	9.30e-08	6/50	yes	-	1.880	0.00
rand5	4	5.34e-08	6/50	yes	-	1.890	0.01	1.69e-10	7/50	yes	-	1.930	0.00
Avg (successes)	4	1.55e-07	6/50	-	-	1.886	0.01	1.88e-07	6/50	-	-	2.000	0.00
xbar	8	1.32e-07	5/50	yes	-	1.880	0.00	6.17e-11	6/50	yes	-	1.940	0.00
rand1	8	1.90e-13	8/50	yes	-	NaN	0.01	9.93e-07	7/50	yes	-	2.140	0.00
rand2	8	1.69e-07	5/50	yes	-	1.870	0.01	1.30e-09	6/50	yes	-	1.920	0.00
rand3	8	5.75e-07	7/50	yes	-	1.850	0.01	3.99e-08	7/50	yes	-	2.190	0.00
rand4	8	8.26e-11	6/50	yes	-	1.940	0.01	9.30e-08	6/50	yes	-	1.880	0.00
rand5	8	5.34e-08	6/50	yes	-	1.890	0.01	1.69e-10	7/50	yes	-	1.930	0.00
Avg (successes)	8	1.55e-07	6/50	-	-	1.886	0.01	1.88e-07	6/50	-	-	2.000	0.00
xbar	12	1.83e-07	5/50	yes	-	1.840	0.00	3.00e-10	6/50	yes	-	1.800	0.00
rand1	12	9.14e-10	8/50	yes	-	1.630	0.01	1.23e-10	8/50	yes	-	1.570	0.00
rand2	12	1.37e-07	5/50	yes	-	1.890	0.01	4.34e-09	6/50	yes	-	1.800	0.00
rand3	12	4.73e-07	7/50	yes	-	1.860	0.01	1.49e-07	7/50	yes	-	1.990	0.00
rand4	12	1.94e-09	6/50	yes	-	1.660	0.00	1.29e-07	6/50	yes	-	1.840	0.00
rand5	12	6.22e-08	6/50	yes	-	1.870	0.01	3.13e-09	7/50	yes	-	1.670	0.00
Avg (successes)	12	1.43e-07	6/50	-	-	1.792	0.01	4.76e-08	7/50	-	-	1.778	0.00

Table 31: Broyden31 – n=2, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.32e-07	5/50	yes	-	1.880	0.12	6.18e-11	6/50	yes	-	1.940	0.06
rand1	4	1.86e-13	8/50	yes	-	NaN	0.02	9.93e-07	7/50	yes	-	2.140	0.01
rand2	4	1.69e-07	5/50	yes	-	1.870	0.01	1.30e-09	6/50	yes	-	1.920	0.00
rand3	4	5.75e-07	7/50	yes	-	1.850	0.01	3.99e-08	7/50	yes	-	2.190	0.01
rand4	4	8.27e-11	6/50	yes	-	1.940	0.01	9.30e-08	6/50	yes	-	1.880	0.02
rand5	4	5.34e-08	6/50	yes	-	1.890	0.01	1.69e-10	7/50	yes	-	1.930	0.00
Avg (successes)	4	1.55e-07	6/50	-	-	1.886	0.03	1.88e-07	6/50	-	-	2.000	0.02
xbar	8	6.69e-08	5/50	yes	-	1.870	0.01	2.55e-08	5/50	yes	-	1.870	0.00
rand1	8	3.19e-07	12/50	yes	-	1.350	0.02	1.74e-12	7/50	yes	-	1.970	0.00
rand2	8	4.10e-12	5/50	yes	-	1.960	0.01	1.40e-10	6/50	yes	-	1.910	0.00
rand3	8	8.09e-12	8/50	yes	-	1.970	0.01	5.97e-08	6/50	yes	-	1.820	0.00
rand4	8	4.92e-11	6/50	yes	-	1.940	0.01	1.66e-07	6/50	yes	-	1.900	0.00
rand5	8	9.56e-09	6/50	yes	-	1.810	0.01	7.63e-08	7/50	yes	-	1.990	0.01
Avg (successes)	8	6.59e-08	7/50	-	-	1.817	0.01	5.46e-08	6/50	-	-	1.910	0.00
xbar	12	6.52e-07	39/50	yes	-	1.460	0.06	3.42e-07	19/50	yes	-	6.770	0.01
rand1	12	4.69e-11	36/50	yes	-	8.000	0.05	1.72e-07	9/50	yes	-	2.880	0.00
rand2	12	3.89e-09	27/50	yes	-	4.260	0.04	4.85e-09	24/50	yes	-	5.130	0.01
rand3	12	1.93e-07	37/50	yes	-	0.970	0.05	9.37e-09	9/50	yes	-	1.310	0.00
rand4	12	2.43e-09	11/50	yes	-	0.610	0.02	9.47e-07	15/50	yes	-	0.850	0.01
rand5	12	7.61e-11	32/50	yes	-	13.380	0.05	7.95e-07	19/50	yes	-	0.960	0.01
Avg (successes)	12	1.42e-07	30/50	-	-	4.780	0.05	3.78e-07	16/50	-	-	2.983	0.01

Table 32: Broyden31 – n=2, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.32e-07	5/50	yes	-	1.880	0.01	6.17e-11	6/50	yes	-	1.940	0.00
rand1	4	1.80e-13	8/50	yes	-	NaN	0.01	9.93e-07	7/50	yes	-	2.140	0.00
rand2	4	1.69e-07	5/50	yes	-	1.870	0.01	1.30e-09	6/50	yes	-	1.920	0.00
rand3	4	5.75e-07	7/50	yes	-	1.850	0.01	3.99e-08	7/50	yes	-	2.190	0.00
rand4	4	8.26e-11	6/50	yes	-	1.940	0.01	9.30e-08	6/50	yes	-	1.880	0.00
rand5	4	5.34e-08	6/50	yes	-	1.890	0.01	1.69e-10	7/50	yes	-	1.930	0.00
Avg (successes)	4	1.55e-07	6/50	-	-	1.886	0.01	1.88e-07	6/50	-	-	2.000	0.00
xbar	8	3.00e-09	5/50	yes	-	2.030	0.01	8.41e-07	5/50	yes	-	1.780	0.00
rand1	8	4.39e-07	8/50	yes	-	1.860	0.01	4.04e-12	7/50	yes	-	2.060	0.01
rand2	8	1.08e-12	6/50	yes	-	2.030	0.01	7.68e-09	6/50	yes	-	1.960	0.00
rand3	8	1.73e-10	11/50	yes	-	1.980	0.02	2.48e-07	12/50	yes	-	0.960	0.01
rand4	8	7.07e-13	6/50	yes	-	NaN	0.01	6.10e-09	6/50	yes	-	1.920	0.00
rand5	8	1.11e-07	6/50	yes	-	1.890	0.01	4.49e-10	7/50	yes	-	2.020	0.00
Avg (successes)	8	9.22e-08	7/50	-	-	1.958	0.01	1.84e-07	7/50	-	-	1.783	0.00
xbar	12	2.77e-08	31/50	yes	-	2.380	0.05	4.20e-09	26/50	yes	-	4.930	0.01
rand1	12	3.64e-04	50/50	no	maxit	NaN	0.06	1.77e-04	28/50	no	bt/maxit	NaN	0.01
rand2	12	4.37e-10	17/50	yes	-	0.800	0.03	9.88e-08	35/50	yes	-	7.000	0.01
rand3	12	1.96e-04	33/50	no	bt/maxit	NaN	0.04	1.83e-07	21/50	yes	-	1.460	0.01
rand4	12	9.81e-11	15/50	yes	-	2.340	0.02	1.13e-09	32/50	yes	-	7.740	0.01
rand5	12	2.45e-10	21/50	yes	-	-41.750	0.02	1.66e-10	20/50	yes	-	1.930	0.02
Avg (successes)	12	7.12e-09	21/50	-	-	-9.058	0.03	5.75e-08	27/50	-	-	4.612	0.01

Table 33: Broyden31 – n=10³, exact: Modified Newton vs Truncated Newton with exact gradient and hessian.

start.pt ID	Modified Newton						Truncated Newton					
	grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	1.79e-08	5/50	yes	-	1.710	0.19	1.61e-07	11/50	yes	-	1.590	0.01
rand1	1.25e-11	10/50	yes	-	1.950	0.40	2.59e-07	12/50	yes	-	1.480	0.00
rand2	5.76e-11	10/50	yes	-	1.940	0.40	4.66e-08	13/50	yes	-	1.500	0.01
rand3	1.97e-09	10/50	yes	-	1.900	0.47	2.40e-07	12/50	yes	-	1.480	0.00
rand4	1.65e-07	8/50	yes	-	1.830	0.33	6.72e-08	14/50	yes	-	1.490	0.01
rand5	4.23e-11	10/50	yes	-	1.910	0.32	3.48e-07	13/50	yes	-	1.430	0.00
Avg (successes)	3.08e-08	9/50	-	-	1.873	0.35	1.87e-07	12/50	-	-	1.495	0.01

Table 34: Broyden31 – $n=10^3$, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.710	0.18	1.61e-07	11/50	yes	-	1.590	0.06
rand1	4	1.26e-11	10/50	yes	-	1.950	0.54	2.59e-07	12/50	yes	-	1.480	0.05
rand2	4	5.77e-11	10/50	yes	-	1.940	0.55	4.66e-08	13/50	yes	-	1.500	0.05
rand3	4	1.97e-09	10/50	yes	-	1.900	0.57	2.40e-07	12/50	yes	-	1.480	0.05
rand4	4	1.65e-07	8/50	yes	-	1.830	0.35	6.72e-08	14/50	yes	-	1.490	0.07
rand5	4	4.25e-11	10/50	yes	-	1.910	0.39	3.48e-07	13/50	yes	-	1.430	0.05
Avg (successes)	4	3.08e-08	9/50	-	-	1.873	0.43	1.87e-07	12/50	-	-	1.495	0.06
xbar	8	1.79e-08	5/50	yes	-	1.710	0.31	1.61e-07	11/50	yes	-	1.590	0.05
rand1	8	1.25e-11	10/50	yes	-	1.950	0.91	2.59e-07	12/50	yes	-	1.480	0.06
rand2	8	5.79e-11	10/50	yes	-	1.940	0.98	4.66e-08	13/50	yes	-	1.500	0.05
rand3	8	1.97e-09	10/50	yes	-	1.900	1.31	2.40e-07	12/50	yes	-	1.480	0.04
rand4	8	1.65e-07	8/50	yes	-	1.830	0.77	6.72e-08	14/50	yes	-	1.490	0.07
rand5	8	4.26e-11	10/50	yes	-	1.910	0.88	3.48e-07	13/50	yes	-	1.430	0.07
Avg (successes)	8	3.08e-08	9/50	-	-	1.873	0.86	1.87e-07	12/50	-	-	1.495	0.06
xbar	12	2.36e-08	5/50	yes	-	1.680	0.42	1.61e-07	11/50	yes	-	1.590	0.06
rand1	12	1.50e-09	10/50	yes	-	1.560	1.17	2.59e-07	12/50	yes	-	1.480	0.06
rand2	12	4.60e-10	10/50	yes	-	1.770	1.67	4.67e-08	13/50	yes	-	1.500	0.05
rand3	12	7.22e-08	10/50	yes	-	1.860	1.27	2.41e-07	12/50	yes	-	1.480	0.05
rand4	12	1.09e-07	8/50	yes	-	1.880	0.47	6.71e-08	14/50	yes	-	1.490	0.06
rand5	12	2.91e-09	10/50	yes	-	1.560	0.42	3.49e-07	13/50	yes	-	1.430	0.06
Avg (successes)	12	3.49e-08	9/50	-	-	1.718	0.90	1.87e-07	12/50	-	-	1.495	0.06

Table 35: Broyden31 – $n=10^3$, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.710	0.18	1.61e-07	11/50	yes	-	1.590	0.04
rand1	4	1.25e-11	10/50	yes	-	1.950	0.46	2.59e-07	12/50	yes	-	1.480	0.05
rand2	4	5.77e-11	10/50	yes	-	1.940	0.76	4.66e-08	13/50	yes	-	1.500	0.05
rand3	4	1.97e-09	10/50	yes	-	1.900	1.11	2.40e-07	12/50	yes	-	1.480	0.05
rand4	4	1.65e-07	8/50	yes	-	1.830	0.81	6.72e-08	14/50	yes	-	1.490	0.07
rand5	4	4.24e-11	10/50	yes	-	1.910	0.84	3.48e-07	13/50	yes	-	1.430	0.05
Avg (successes)	4	3.08e-08	9/50	-	-	1.873	0.69	1.87e-07	12/50	-	-	1.495	0.05
xbar	8	1.79e-08	5/50	yes	-	1.710	0.33	1.61e-07	11/50	yes	-	1.590	0.06
rand1	8	1.25e-11	10/50	yes	-	1.950	0.81	2.59e-07	12/50	yes	-	1.480	0.06
rand2	8	5.77e-11	10/50	yes	-	1.940	1.01	4.66e-08	13/50	yes	-	1.500	0.08
rand3	8	1.97e-09	10/50	yes	-	1.900	1.33	2.40e-07	12/50	yes	-	1.480	0.06
rand4	8	1.65e-07	8/50	yes	-	1.830	0.88	6.72e-08	14/50	yes	-	1.490	0.06
rand5	8	4.23e-11	10/50	yes	-	1.910	1.17	3.48e-07	13/50	yes	-	1.430	0.06
Avg (successes)	8	3.08e-08	9/50	-	-	1.873	0.92	1.87e-07	12/50	-	-	1.495	0.06
xbar	12	3.02e-08	5/50	yes	-	1.660	0.21	1.59e-07	11/50	yes	-	1.590	0.05
rand1	12	3.17e-10	10/50	yes	-	1.560	0.54	2.59e-07	12/50	yes	-	1.480	0.05
rand2	12	1.29e-09	10/50	yes	-	1.540	0.66	4.66e-08	13/50	yes	-	1.500	0.06
rand3	12	1.20e-10	11/50	yes	-	1.470	0.73	2.40e-07	12/50	yes	-	1.480	0.05
rand4	12	1.79e-07	8/50	yes	-	1.710	0.50	6.74e-08	14/50	yes	-	1.490	0.06
rand5	12	2.67e-09	10/50	yes	-	1.560	0.50	3.49e-07	13/50	yes	-	1.430	0.05
Avg (successes)	12	3.56e-08	9/50	-	-	1.583	0.52	1.87e-07	12/50	-	-	1.495	0.05

Table 36: Broyden31 – $n=10^3$, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.710	0.66	1.61e-07	11/50	yes	-	1.590	0.42
rand1	4	1.26e-11	10/50	yes	-	1.950	1.25	2.59e-07	12/50	yes	-	1.480	0.45
rand2	4	5.77e-11	10/50	yes	-	1.940	1.44	4.66e-08	13/50	yes	-	1.500	0.49
rand3	4	1.97e-09	10/50	yes	-	1.900	2.33	2.40e-07	12/50	yes	-	1.480	0.46
rand4	4	1.65e-07	8/50	yes	-	1.830	6.38	6.72e-08	14/50	yes	-	1.490	0.59
rand5	4	4.25e-11	10/50	yes	-	1.910	3.21	3.48e-07	13/50	yes	-	1.430	0.42
Avg (successes)	4	3.08e-08	9/50	-	-	1.873	2.54	1.87e-07	12/50	-	-	1.495	0.47
xbar	8	4.71e-08	5/50	yes	-	1.770	0.60	6.05e-07	10/50	yes	-	1.440	0.34
rand1	8	2.05e-07	14/50	yes	-	0.830	2.00	3.86e-07	14/50	yes	-	1.530	0.43
rand2	8	8.56e-08	16/50	yes	-	1.190	2.52	2.01e-07	12/50	yes	-	1.500	0.38
rand3	8	6.01e-07	19/50	yes	-	0.780	2.82	7.18e-08	14/50	yes	-	1.500	0.44
rand4	8	1.06e-08	15/50	yes	-	1.860	2.11	6.46e-08	14/50	yes	-	1.470	0.46
rand5	8	2.60e-07	15/50	yes	-	1.180	2.17	3.32e-07	13/50	yes	-	1.430	0.41
Avg (successes)	8	2.02e-07	14/50	-	-	1.268	2.04	2.77e-07	13/50	-	-	1.478	0.41
xbar	12	1.27e+02	50/50	no	maxit	NaN	4.24	5.72e+00	50/50	no	maxit	NaN	1.67
rand1	12	1.31e+03	50/50	no	maxit	NaN	4.13	5.19e+00	50/50	no	maxit	NaN	1.45
rand2	12	1.38e+03	50/50	no	maxit	NaN	3.86	4.81e+00	50/50	no	maxit	NaN	1.49
rand3	12	1.27e+03	50/50	no	maxit	NaN	4.54	3.13e+00	50/50	no	maxit	NaN	1.48
rand4	12	1.33e+03	50/50	no	maxit	NaN	4.49	5.32e+00	50/50	no	maxit	NaN	1.97
rand5	12	1.36e+03	50/50	no	maxit	NaN	3.99	3.14e+00	50/50	no	maxit	NaN	2.14
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 37: Broyden31 – $n=10^3$, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.710	1.56	1.61e-07	11/50	yes	-	1.590	0.36
rand1	4	1.89e-11	10/50	yes	-	1.940	3.73	2.59e-07	12/50	yes	-	1.480	0.39
rand2	4	5.67e-11	10/50	yes	-	1.940	3.13	4.66e-08	13/50	yes	-	1.500	0.41
rand3	4	1.97e-09	10/50	yes	-	1.900	3.35	2.40e-07	12/50	yes	-	1.480	0.39
rand4	4	1.63e-07	8/50	yes	-	1.830	2.38	6.72e-08	14/50	yes	-	1.490	0.43
rand5	4	4.25e-11	10/50	yes	-	1.910	2.72	3.48e-07	13/50	yes	-	1.430	0.41
Avg (successes)	4	3.05e-08	9/50	-	-	1.872	2.81	1.87e-07	12/50	-	-	1.495	0.40
xbar	8	8.06e-09	5/50	yes	-	1.730	1.27	1.47e-07	12/50	yes	-	1.470	0.39
rand1	8	1.34e+03	50/50	no	maxit	NaN	9.19	5.84e-05	50/50	no	maxit	NaN	1.61
rand2	8	1.16e+03	50/50	no	maxit	NaN	10.43	1.46e-05	50/50	no	maxit	NaN	1.53
rand3	8	1.34e+03	50/50	no	maxit	NaN	8.07	4.91e-07	50/50	yes	-	1.550	1.57
rand4	8	1.34e+03	50/50	no	maxit	NaN	5.61	1.38e-09	15/50	yes	-	1.540	0.49
rand5	8	1.15e+03	50/50	no	maxit	NaN	5.15	3.35e-07	13/50	yes	-	1.490	0.43
Avg (successes)	8	8.06e-09	5/50	-	-	1.730	1.27	2.44e-07	22/50	-	-	1.513	0.72
xbar	12	1.27e+02	50/50	no	maxit	NaN	4.39	2.79e+00	50/50	no	maxit	NaN	2.23
rand1	12	1.37e+03	50/50	no	maxit	NaN	4.64	2.68e+01	50/50	no	maxit	NaN	2.25
rand2	12	1.44e+03	50/50	no	maxit	NaN	4.82	5.13e+00	50/50	no	maxit	NaN	2.34
rand3	12	1.35e+03	50/50	no	maxit	NaN	4.33	6.61e-01	50/50	no	maxit	NaN	2.27
rand4	12	1.39e+03	50/50	no	maxit	NaN	3.98	5.32e+00	50/50	no	maxit	NaN	2.37
rand5	12	1.43e+03	50/50	no	maxit	NaN	5.00	3.34e+00	50/50	no	maxit	NaN	2.20
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 38: Broyden31 – n=10⁴, exact: Modified Newton vs Truncated Newton with exact gradient and hessian.

start.pt ID	Modified Newton						Truncated Newton					
	grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	1.79e – 08	5/50	yes	-	1.670	0.12	1.65e – 08	12/50	yes	-	1.470	0.05
rand1	1.61e – 07	8/50	yes	-	1.800	0.09	1.25e – 07	14/50	yes	-	1.500	0.04
rand2	1.35e – 09	8/50	yes	-	1.820	0.09	5.53e – 09	15/50	yes	-	1.520	0.05
rand3	1.13e – 08	8/50	yes	-	1.800	0.08	1.42e – 09	16/50	yes	-	1.570	0.06
rand4	2.39e – 07	8/50	yes	-	1.770	0.09	1.43e – 07	14/50	yes	-	1.490	0.05
rand5	5.94e – 10	9/50	yes	-	1.900	0.09	1.42e – 07	14/50	yes	-	1.490	0.05
Avg (successes)	7.19e – 08	8/50	-	-	1.793	0.09	7.22e – 08	14/50	-	-	1.507	0.05

Table 39: Broyden31 – n=10⁴, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e – 08	5/50	yes	-	1.670	1.08	1.65e – 08	12/50	yes	-	1.470	1.28
rand1	4	1.61e – 07	8/50	yes	-	1.800	1.61	1.25e – 07	14/50	yes	-	1.500	1.23
rand2	4	1.35e – 09	8/50	yes	-	1.820	1.66	5.53e – 09	15/50	yes	-	1.520	1.30
rand3	4	1.13e – 08	8/50	yes	-	1.800	1.70	1.42e – 09	16/50	yes	-	1.570	1.36
rand4	4	2.39e – 07	8/50	yes	-	1.770	1.72	1.43e – 07	14/50	yes	-	1.490	1.25
rand5	4	5.95e – 10	9/50	yes	-	1.900	1.86	1.42e – 07	14/50	yes	-	1.490	1.48
Avg (successes)	4	7.19e – 08	8/50	-	-	1.793	1.61	7.22e – 08	14/50	-	-	1.507	1.32
xbar	8	1.79e – 08	5/50	yes	-	1.670	1.15	1.65e – 08	12/50	yes	-	1.470	1.63
rand1	8	1.61e – 07	8/50	yes	-	1.800	1.71	1.25e – 07	14/50	yes	-	1.500	1.23
rand2	8	1.35e – 09	8/50	yes	-	1.820	1.76	5.53e – 09	15/50	yes	-	1.520	1.21
rand3	8	1.13e – 08	8/50	yes	-	1.800	1.40	1.42e – 09	16/50	yes	-	1.570	1.41
rand4	8	2.39e – 07	8/50	yes	-	1.770	1.43	1.43e – 07	14/50	yes	-	1.490	1.29
rand5	8	5.95e – 10	9/50	yes	-	1.900	1.54	1.42e – 07	14/50	yes	-	1.490	1.10
Avg (successes)	8	7.19e – 08	8/50	-	-	1.793	1.50	7.22e – 08	14/50	-	-	1.507	1.31
xbar	12	2.63e – 08	5/50	yes	-	1.640	0.86	1.65e – 08	12/50	yes	-	1.470	0.92
rand1	12	1.52e – 07	8/50	yes	-	1.800	1.34	1.26e – 07	14/50	yes	-	1.500	0.99
rand2	12	6.45e – 09	8/50	yes	-	1.680	1.33	5.54e – 09	15/50	yes	-	1.520	1.08
rand3	12	2.02e – 08	8/50	yes	-	1.750	1.32	1.43e – 09	16/50	yes	-	1.570	1.13
rand4	12	1.99e – 07	8/50	yes	-	1.790	1.49	1.43e – 07	14/50	yes	-	1.490	1.01
rand5	12	4.14e – 09	9/50	yes	-	1.720	1.49	1.42e – 07	14/50	yes	-	1.490	0.98
Avg (successes)	12	6.80e – 08	8/50	-	-	1.730	1.31	7.24e – 08	14/50	-	-	1.507	1.02

Table 40: Broyden31 – n=10⁴, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e – 08	5/50	yes	-	1.670	1.13	1.65e – 08	12/50	yes	-	1.470	1.15
rand1	4	1.61e – 07	8/50	yes	-	1.800	1.76	1.25e – 07	14/50	yes	-	1.500	1.35
rand2	4	1.35e – 09	8/50	yes	-	1.820	1.73	5.53e – 09	15/50	yes	-	1.520	1.52
rand3	4	1.13e – 08	8/50	yes	-	1.800	1.76	1.42e – 09	16/50	yes	-	1.570	1.66
rand4	4	2.39e – 07	8/50	yes	-	1.770	1.75	1.43e – 07	14/50	yes	-	1.490	1.43
rand5	4	5.95e – 10	9/50	yes	-	1.900	1.92	1.42e – 07	14/50	yes	-	1.490	1.39
Avg (successes)	4	7.19e – 08	8/50	-	-	1.793	1.67	7.22e – 08	14/50	-	-	1.507	1.42
xbar	8	1.79e – 08	5/50	yes	-	1.670	0.80	1.65e – 08	12/50	yes	-	1.470	1.04
rand1	8	1.61e – 07	8/50	yes	-	1.800	1.44	1.25e – 07	14/50	yes	-	1.500	1.10
rand2	8	1.35e – 09	8/50	yes	-	1.820	1.73	5.53e – 09	15/50	yes	-	1.520	1.19
rand3	8	1.13e – 08	8/50	yes	-	1.800	1.49	1.42e – 09	16/50	yes	-	1.570	1.25
rand4	8	2.39e – 07	8/50	yes	-	1.770	1.46	1.43e – 07	14/50	yes	-	1.490	1.21
rand5	8	5.93e – 10	9/50	yes	-	1.900	1.55	1.42e – 07	14/50	yes	-	1.490	1.15
Avg (successes)	8	7.19e – 08	8/50	-	-	1.793	1.41	7.22e – 08	14/50	-	-	1.507	1.16
xbar	12	3.95e – 08	5/50	yes	-	1.610	0.86	1.66e – 08	12/50	yes	-	1.460	0.85
rand1	12	6.01e – 07	8/50	yes	-	1.660	1.14	1.26e – 07	14/50	yes	-	1.500	1.00
rand2	12	6.42e – 08	8/50	yes	-	1.650	1.16	5.54e – 09	15/50	yes	-	1.520	1.10
rand3	12	6.30e – 08	8/50	yes	-	1.660	1.14	1.43e – 09	16/50	yes	-	1.570	1.13
rand4	12	4.74e – 07	8/50	yes	-	1.690	1.23	1.43e – 07	14/50	yes	-	1.490	1.04
rand5	12	1.13e – 08	9/50	yes	-	1.700	1.50	1.42e – 07	14/50	yes	-	1.490	1.00
Avg (successes)	12	2.09e – 07	8/50	-	-	1.662	1.17	7.24e – 08	14/50	-	-	1.505	1.02

Table 41: Broyden31 – n=10⁴, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e – 08	5/50	yes	-	1.670	5.93	1.65e – 08	12/50	yes	-	1.470	7.35
rand1	4	1.61e – 07	8/50	yes	-	1.800	9.05	1.25e – 07	14/50	yes	-	1.500	8.46
rand2	4	1.35e – 09	8/50	yes	-	1.820	9.73	5.53e – 09	15/50	yes	-	1.520	9.13
rand3	4	1.13e – 08	8/50	yes	-	1.800	9.86	1.42e – 09	16/50	yes	-	1.570	9.68
rand4	4	2.39e – 07	8/50	yes	-	1.770	10.26	1.43e – 07	14/50	yes	-	1.490	8.68
rand5	4	5.95e – 10	9/50	yes	-	1.900	10.27	1.42e – 07	14/50	yes	-	1.490	8.64
Avg (successes)	4	7.19e – 08	8/50	-	-	1.793	9.18	7.22e – 08	14/50	-	-	1.507	8.66
xbar	8	1.76e – 08	5/50	yes	-	1.940	14.65	5.97e – 09	12/50	yes	-	1.530	7.47
rand1	8	7.05e – 07	19/50	yes	-	1.130	43.50	1.39e – 07	14/50	yes	-	1.490	8.68
rand2	8	1.24e – 07	16/50	yes	-	0.930	31.31	5.39e – 09	15/50	yes	-	1.520	9.24
rand3	8	1.93e – 07	16/50	yes	-	0.930	16.67	1.47e – 09	16/50	yes	-	1.570	9.80
rand4	8	2.88e – 10	16/50	yes	-	1.940	9.79	1.55e – 07	14/50	yes	-	1.490	8.60
rand5	8	5.82e – 09	12/50	yes	-	1.920	7.36	1.56e – 07	14/50	yes	-	1.490	8.59
Avg (successes)	8	1.74e – 07	14/50	-	-	1.465	20.55	7.71e – 08	14/50	-	-	1.515	8.73
xbar	12	4.00e + 02	50/50	no	maxit	NaN	29.81	5.63e – 01	50/50	no	maxit	NaN	29.95
rand1	12	4.54e + 03	50/50	no	maxit	NaN	37.34	3.43e + 01	50/50	no	maxit	NaN	29.40
rand2	12	4.56e + 03	50/50	no	maxit	NaN	56.68	7.83e + 01	50/50	no	maxit	NaN	30.67
rand3	12	4.57e + 03	50/50	no	maxit	NaN	43.65	4.66e + 02	50/50	no	maxit	NaN	31.11
rand4	12	4.45e + 03	50/50	no	maxit	NaN	64.11	3.31e + 01	50/50	no	maxit	NaN	29.39
rand5	12	4.55e + 03	50/50	no	maxit	NaN	34.18	3.46e + 01	50/50	no	maxit	NaN	30.10
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 42: Broyden31 – n=10⁴, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.670	6.47	1.65e-08	12/50	yes	-	1.470	7.54
rand1	4	3.44e-07	9/50	yes	-	1.760	9.36	1.25e-07	14/50	yes	-	1.500	8.67
rand2	4	1.38e-09	8/50	yes	-	1.820	8.40	5.53e-09	15/50	yes	-	1.520	9.28
rand3	4	1.15e-08	8/50	yes	-	1.800	8.03	1.42e-09	16/50	yes	-	1.570	9.84
rand4	4	2.39e-07	8/50	yes	-	1.770	8.99	1.43e-07	14/50	yes	-	1.490	8.66
rand5	4	3.58e-12	11/50	yes	-	1.960	25.31	1.42e-07	14/50	yes	-	1.490	8.66
Avg (successes)	4	1.02e-07	8/50	-	-	1.797	11.09	7.22e-08	14/50	-	-	1.507	8.77
xbar	8	1.24e-08	5/50	yes	-	1.620	3.34	6.51e-07	20/50	yes	-	1.430	12.08
rand1	8	4.53e+03	50/50	no	maxit	NaN	29.18	5.68e-08	16/50	yes	-	1.490	9.83
rand2	8	4.55e+03	50/50	no	maxit	NaN	28.98	9.92e+00	50/50	no	maxit	NaN	29.83
rand3	8	4.55e+03	50/50	no	maxit	NaN	28.81	5.40e-08	17/50	yes	-	1.410	10.65
rand4	8	4.45e+03	50/50	no	maxit	NaN	29.11	1.18e-06	50/50	no	maxit	NaN	29.72
rand5	8	4.53e+03	50/50	no	maxit	NaN	29.00	3.08e-07	17/50	yes	-	1.400	10.35
Avg (successes)	8	1.24e-08	5/50	-	-	1.620	3.34	2.67e-07	18/50	-	-	1.433	10.73
xbar	12	4.00e+02	50/50	no	maxit	NaN	31.36	5.10e+00	50/50	no	maxit	NaN	29.57
rand1	12	4.54e+03	50/50	no	maxit	NaN	31.45	3.29e+01	50/50	no	maxit	NaN	29.67
rand2	12	4.54e+03	50/50	no	maxit	NaN	47.57	3.53e+01	50/50	no	maxit	NaN	29.40
rand3	12	4.55e+03	50/50	no	maxit	NaN	40.72	1.49e+01	50/50	no	maxit	NaN	29.40
rand4	12	4.44e+03	50/50	no	maxit	NaN	49.72	3.72e+01	50/50	no	maxit	NaN	30.05
rand5	12	4.53e+03	50/50	no	maxit	NaN	40.13	3.47e+01	50/50	no	maxit	NaN	29.63
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 43: Broyden31 – n=10⁵, exact: Modified Newton vs Truncated Newton with exact gradient and hessian.

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.700	0.25	1.72e-07	12/50	yes	-	1.520	0.48
rand1	4	1.07e-10	10/50	yes	-	1.940	0.63	8.85e-07	16/50	yes	-	1.510	0.55
rand2	4	4.68e-07	9/50	yes	-	1.800	0.58	5.99e-08	16/50	yes	-	1.460	0.56
rand3	4	2.15e-09	9/50	yes	-	1.870	0.58	1.18e-09	16/50	yes	-	1.530	0.59
rand4	4	3.17e-13	9/50	yes	-	NaN	0.58	5.11e-08	15/50	yes	-	1.470	0.53
rand5	4	4.07e-08	12/50	yes	-	1.900	0.71	1.10e-09	16/50	yes	-	1.530	0.60
Avg (successes)	4	8.81e-08	9/50	-	-	1.842	0.56	1.95e-07	15/50	-	-	1.503	0.55

Table 44: Broyden31 – n=10⁵, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.700	37.09	1.72e-07	12/50	yes	-	1.520	88.84
rand1	4	1.07e-10	10/50	yes	-	1.940	70.29	8.85e-07	16/50	yes	-	1.510	118.76
rand2	4	4.68e-07	9/50	yes	-	1.800	62.81	5.99e-08	16/50	yes	-	1.460	115.71
rand3	4	2.15e-09	9/50	yes	-	1.870	65.10	1.18e-09	16/50	yes	-	1.530	116.03
rand4	4	3.27e-13	9/50	yes	-	NaN	165.52	5.11e-08	15/50	yes	-	1.470	113.36
rand5	4	4.07e-08	12/50	yes	-	1.900	128.26	1.10e-09	16/50	yes	-	1.520	115.89
Avg (successes)	4	8.81e-08	9/50	-	-	1.842	88.18	1.95e-07	15/50	-	-	1.502	111.43
xbar	8	1.79e-08	5/50	yes	-	1.700	82.06	1.72e-07	12/50	yes	-	1.520	87.53
rand1	8	1.08e-10	10/50	yes	-	1.940	107.97	8.85e-07	16/50	yes	-	1.510	114.89
rand2	8	4.68e-07	9/50	yes	-	1.800	95.83	5.99e-08	16/50	yes	-	1.460	115.57
rand3	8	2.15e-09	9/50	yes	-	1.870	98.64	1.18e-09	16/50	yes	-	1.530	113.57
rand4	8	3.26e-13	9/50	yes	-	NaN	100.62	5.11e-08	15/50	yes	-	1.470	108.03
rand5	8	4.07e-08	12/50	yes	-	1.900	141.85	1.10e-09	16/50	yes	-	1.520	121.05
Avg (successes)	8	8.81e-08	9/50	-	-	1.842	104.50	1.95e-07	15/50	-	-	1.502	110.11
xbar	12	2.72e-08	5/50	yes	-	1.670	61.16	1.72e-07	12/50	yes	-	1.520	86.81
rand1	12	2.36e-09	10/50	yes	-	1.670	123.76	8.85e-07	16/50	yes	-	1.510	114.25
rand2	12	3.45e-07	9/50	yes	-	1.830	168.48	6.00e-08	16/50	yes	-	1.460	115.52
rand3	12	1.39e-08	9/50	yes	-	1.690	118.71	1.18e-09	16/50	yes	-	1.530	113.02
rand4	12	2.36e-10	9/50	yes	-	1.440	146.96	5.11e-08	15/50	yes	-	1.470	107.65
rand5	12	5.98e-08	12/50	yes	-	1.850	379.12	1.11e-09	16/50	yes	-	1.520	117.30
Avg (successes)	12	7.47e-08	9/50	-	-	1.692	166.36	1.95e-07	15/50	-	-	1.502	109.09

Table 45: Broyden31 – n=10⁵, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 1).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.700	45.93	1.72e-07	12/50	yes	-	1.520	87.79
rand1	4	1.07e-10	10/50	yes	-	1.940	83.25	8.85e-07	16/50	yes	-	1.510	113.26
rand2	4	4.68e-07	9/50	yes	-	1.800	138.67	5.99e-08	16/50	yes	-	1.460	115.30
rand3	4	2.15e-09	9/50	yes	-	1.870	239.01	1.18e-09	16/50	yes	-	1.530	116.45
rand4	4	3.23e-13	9/50	yes	-	NaN	211.22	5.11e-08	15/50	yes	-	1.470	107.09
rand5	4	4.07e-08	12/50	yes	-	1.900	319.30	1.10e-09	16/50	yes	-	1.520	113.82
Avg (successes)	4	8.81e-08	9/50	-	-	1.842	172.90	1.95e-07	15/50	-	-	1.502	108.95
xbar	8	1.79e-08	5/50	yes	-	1.700	60.20	1.72e-07	12/50	yes	-	1.520	87.19
rand1	8	1.07e-10	10/50	yes	-	1.940	194.00	8.85e-07	16/50	yes	-	1.510	115.05
rand2	8	4.68e-07	9/50	yes	-	1.800	114.49	5.99e-08	16/50	yes	-	1.460	114.38
rand3	8	2.15e-09	9/50	yes	-	1.870	165.29	1.18e-09	16/50	yes	-	1.530	114.50
rand4	8	3.12e-13	9/50	yes	-	NaN	116.56	5.11e-08	15/50	yes	-	1.470	109.86
rand5	8	4.06e-08	12/50	yes	-	1.900	193.41	1.10e-09	16/50	yes	-	1.520	115.41
Avg (successes)	8	8.81e-08	9/50	-	-	1.842	140.66	1.95e-07	15/50	-	-	1.502	109.40
xbar	12	8.68e-08	5/50	yes	-	1.580	61.46	1.71e-07	12/50	yes	-	1.520	87.26
rand1	12	4.17e-09	10/50	yes	-	1.640	83.18	8.86e-07	16/50	yes	-	1.510	116.21
rand2	12	4.61e-08	9/50	yes	-	1.670	121.32	6.00e-08	16/50	yes	-	1.460	116.06
rand3	12	9.83e-09	9/50	yes	-	1.650	116.99	1.19e-09	16/50	yes	-	1.530	114.94
rand4	12	1.18e-10	9/50	yes	-	1.440	144.29	5.13e-08	15/50	yes	-	1.470	109.19
rand5	12	3.00e-07	12/50	yes	-	1.800	111.69	1.11e-09	16/50	yes	-	1.520	112.65
Avg (successes)	12	7.45e-08	9/50	-	-	1.630	106.49	1.95e-07	15/50	-	-	1.502	109.38

Table 46: Broyden31 – n=10⁵, h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.700	520.11	1.72e-07	12/50	yes	-	1.520	569.03
rand1	4	1.07e-10	10/50	yes	-	1.940	667.96	8.85e-07	16/50	yes	-	1.510	746.01
rand2	4	4.68e-07	9/50	yes	-	1.800	693.86	5.99e-08	16/50	yes	-	1.460	741.88
rand3	4	2.15e-09	9/50	yes	-	1.870	749.96	1.18e-09	16/50	yes	-	1.530	744.52
rand4	4	3.27e-13	9/50	yes	-	NaN	716.82	5.11e-08	15/50	yes	-	1.470	707.69
rand5	4	4.07e-08	12/50	yes	-	1.900	695.80	1.10e-09	16/50	yes	-	1.520	792.16
Avg (successes)	4	8.81e-08	9/50	-	-	1.842	674.09	1.95e-07	15/50	-	-	1.502	716.88
xbar	8	5.28e-08	5/50	yes	-	1.940	280.18	4.84e-08	13/50	yes	-	1.440	635.27
rand1	8	3.01e-07	36/50	yes	-	0.940	1874.27	9.90e-07	16/50	yes	-	1.500	761.96
rand2	8	1.32e-07	14/50	yes	-	1.870	1226.45	4.26e-08	15/50	yes	-	1.490	717.84
rand3	8	1.67e-07	33/50	yes	-	0.820	1891.84	7.34e-10	17/50	yes	-	1.520	793.75
rand4	8	3.68e-10	15/50	yes	-	1.920	751.12	9.32e-07	16/50	yes	-	1.480	2320.34
rand5	8	8.00e-07	16/50	yes	-	1.090	796.59	9.85e-07	16/50	yes	-	1.470	1172.26
Avg (successes)	8	2.42e-07	20/50	-	-	1.430	1136.74	5.00e-07	16/50	-	-	1.483	1066.90
xbar	12	1.26e+03	50/50	no	maxit	NaN	2597.30	2.39e+00	50/50	no	maxit	NaN	2266.59
rand1	12	1.42e+04	50/50	no	maxit	NaN	3684.03	1.05e+02	50/50	no	maxit	NaN	2479.77
rand2	12	1.42e+04	50/50	no	maxit	NaN	3773.22	2.44e+02	50/50	no	maxit	NaN	2375.28
rand3	12	1.43e+04	50/50	no	maxit	NaN	2377.56	1.14e+02	50/50	no	maxit	NaN	3291.74
rand4	12	1.43e+04	50/50	no	maxit	NaN	2325.63	1.07e+02	50/50	no	maxit	NaN	2065.59
rand5	12	1.43e+04	50/50	no	maxit	NaN	2287.00	1.05e+02	50/50	no	maxit	NaN	2362.31
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

Table 47: Broyden31 – n=10⁵, hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

start.pt ID	k	Modified Newton						Truncated Newton					
		grad.norm	iters/max.iters	success	flag	rate	time (s)	grad.norm	iters/max.iters	success	flag	rate	time (s)
xbar	4	1.79e-08	5/50	yes	-	1.700	283.01	1.72e-07	12/50	yes	-	1.520	871.83
rand1	4	9.15e-08	10/50	yes	-	1.760	562.37	8.85e-07	16/50	yes	-	1.510	1004.22
rand2	4	6.75e-11	10/50	yes	-	1.860	1604.42	5.99e-08	16/50	yes	-	1.460	751.38
rand3	4	2.68e-08	11/50	yes	-	1.840	633.06	1.18e-09	16/50	yes	-	1.530	801.86
rand4	4	1.86e-07	10/50	yes	-	1.780	555.25	5.11e-08	15/50	yes	-	1.470	700.32
rand5	4	1.32e-07	12/50	yes	-	1.870	654.14	1.10e-09	16/50	yes	-	1.530	746.06
Avg (successes)	4	7.57e-08	10/50	-	-	1.802	715.38	1.95e-07	15/50	-	-	1.503	812.61
xbar	8	1.24e-08	5/50	yes	-	1.590	279.52	5.84e-09	17/50	yes	-	1.520	809.17
rand1	8	1.42e+04	50/50	no	maxit	NaN	2401.41	1.76e-01	50/50	no	maxit	NaN	3779.63
rand2	8	1.42e+04	50/50	no	maxit	NaN	2430.76	5.00e-05	50/50	no	maxit	NaN	2238.40
rand3	8	1.43e+04	50/50	no	maxit	NaN	3037.78	7.29e-07	21/50	yes	-	1.520	968.25
rand4	8	1.43e+04	50/50	no	maxit	NaN	2401.59	1.41e-07	25/50	yes	-	1.460	1171.16
rand5	8	1.43e+04	50/50	no	maxit	NaN	3021.91	3.91e-07	22/50	yes	-	1.550	1011.68
Avg (successes)	8	1.24e-08	5/50	-	-	1.590	279.52	3.17e-07	21/50	-	-	1.512	990.06
xbar	12	1.26e+03	50/50	no	maxit	NaN	2248.63	4.35e+00	50/50	no	maxit	NaN	6505.44
rand1	12	1.42e+04	50/50	no	maxit	NaN	2378.86	4.62e+01	50/50	no	maxit	NaN	2858.16
rand2	12	1.42e+04	50/50	no	maxit	NaN	2297.65	5.00e+01	50/50	no	maxit	NaN	2575.26
rand3	12	1.43e+04	50/50	no	maxit	NaN	2276.58	2.28e+01	50/50	no	maxit	NaN	2576.97
rand4	12	1.43e+04	50/50	no	maxit	NaN	2205.26	5.01e+01	50/50	no	maxit	NaN	2568.40
rand5	12	1.43e+04	50/50	no	maxit	NaN	2188.60	5.12e+01	50/50	no	maxit	NaN	2565.28
Avg (successes)	12	-	-	-	-	-	-	-	-	-	-	-	-

D.3 Finite Difference Error Estimation

n	k	type	err_gradfd	err_hessfd_gradex	err_hessfd_gradfd
2	4	const	1.67e-09	1.67e-09	1.53e-08
2	4	rel	1.77e-09	1.63e-09	2.03e-08
2	8	const	6.34e-09	6.66e-09	1.37e+00
2	8	rel	6.53e-09	8.07e-09	1.92e+00
2	12	const	6.65e-05	6.61e-05	1.50e+08
2	12	rel	5.98e-05	6.86e-05	1.97e+08
1000	4	const	1.67e-09	1.67e-09	2.21e-08
1000	4	rel	2.37e-09	1.34e-09	3.25e-06
1000	8	const	7.21e-09	9.16e-09	2.20e+00
1000	8	rel	8.98e-09	9.38e-09	3.55e+02
1000	12	const	8.52e-05	8.78e-05	2.19e+08
1000	12	rel	7.98e-05	9.42e-05	1.13e+10
10000	4	const	1.67e-09	1.67e-09	2.23e-08
10000	4	rel	2.37e-09	1.33e-09	1.49e-03
10000	8	const	7.25e-09	9.32e-09	2.23e+00
10000	8	rel	4.56e-08	9.40e-09	3.24e+04
10000	12	const	8.55e-05	8.83e-05	2.19e+08
10000	12	rel	8.15e-05	9.34e-05	1.02e+10
100000	4	const	1.67e-09	1.67e-09	2.23e-08
100000	4	rel	2.36e-09	1.33e-09	6.79e-04
100000	8	const	7.25e-09	9.45e-09	2.23e+00
100000	8	rel	2.55e-08	9.54e-09	1.69e+04
100000	12	const	8.44e-05	9.46e-05	2.06e+08
100000	12	rel	7.71e-05	9.72e-05	1.62e+10

Table 48: Finite-difference accuracy for Problem 16 for different values of n . Reported values are mean relative errors over 30 random test points in the domain. Gradient errors are measured using the Euclidean norm, while Hessian errors are measured using the Frobenius norm.

n	k	type	err_gradfd	err_hessfd_gradex	err_hessfd_gradfd
2	4	const	6.88e-13	1.06e-09	2.13e-09
2	4	rel	6.08e-13	1.40e-09	2.02e-09
2	8	const	6.54e-09	6.18e-09	1.02e-01
2	8	rel	6.55e-09	6.96e-09	1.52e-01
2	12	const	7.85e-05	7.61e-05	9.79e+06
2	12	rel	5.13e-05	5.79e-05	1.59e+07
1000	4	const	8.65e-13	1.11e-09	2.44e-09
1000	4	rel	1.55e-12	1.39e-09	9.30e-07
1000	8	const	7.78e-09	6.99e-09	1.58e-01
1000	8	rel	1.89e-08	1.39e-08	2.03e+02
1000	12	const	9.38e-05	8.62e-05	1.33e+07
1000	12	rel	1.11e-04	9.43e-05	1.31e+09
10000	4	const	8.31e-13	1.11e-09	2.43e-09
10000	4	rel	7.82e-12	1.39e-09	7.42e-05
10000	8	const	7.75e-09	6.99e-09	1.55e-01
10000	8	rel	7.46e-08	3.82e-08	6.40e+03
10000	12	const	9.45e-05	8.62e-05	1.32e+07
10000	12	rel	1.17e-04	8.88e-05	1.37e+09
100000	4	const	8.29e-13	1.11e-09	2.43e-09
100000	4	rel	7.36e-11	1.40e-09	2.53e-01
100000	8	const	7.74e-09	6.98e-09	1.54e-01
100000	8	rel	2.17e-06	6.21e-07	4.48e+06
100000	12	const	9.47e-05	8.63e-05	1.31e+07
100000	12	rel	1.33e-04	9.28e-05	2.03e+09

Table 49: Finite-difference accuracy for Problem 31 for different values of n . Reported values are mean relative errors over 30 random test points in the domain. Gradient errors are measured using the Euclidean norm, while Hessian errors are measured using the Frobenius norm.

D.4 Diagnostic Trig16

Start	DM	k	Type	$\lambda_{\min}(H_0)$	τ_0	$\lambda_{\min}(B_k)$	$\ p_0\ $	$\ \nabla f(x_0)\ $	Ampl.
1	Exact	–	–	–1.1426	1.1526	0.0100	192.21	2.2361	85.96
2	Exact	–	–	0.7124	0.0000	0.7124	3.1890	2.7084	1.18
3	Exact	–	–	–2.2137	2.2237	0.0100	31.540	0.9508	33.17
4	Exact	–	–	–2.2097	2.2197	0.0100	34.214	1.0463	32.70
5	Exact	–	–	–1.9160	1.9260	0.0100	115.28	2.0582	56.01
6	Exact	–	–	–2.2260	2.2360	0.0100	21.205	2.2227	9.54
1	Case 1	4	h	–1.1426	1.1526	0.0100	192.21	2.2361	85.96
2	Case 1	4	h	0.7124	0.0000	0.7124	3.1890	2.7084	1.18
3	Case 1	4	h	–2.2137	2.2237	0.0100	31.540	0.9508	33.17
4	Case 1	4	h	–2.2097	2.2197	0.0100	34.214	1.0463	32.70
5	Case 1	4	h	–1.9160	1.9260	0.0100	115.28	2.0582	56.01
6	Case 1	4	h	–2.2260	2.2360	0.0100	21.205	2.2227	9.54
1	Case 2	4	h	–1.1426	1.1526	0.0100	192.21	2.2361	85.96
2	Case 2	4	h	0.7124	0.0000	0.7124	3.1890	2.7084	1.18
3	Case 2	4	h	–2.2137	2.2237	0.0100	31.540	0.9508	33.17
4	Case 2	4	h	–2.2097	2.2197	0.0100	34.214	1.0463	32.70
5	Case 2	4	h	–1.9160	1.9260	0.0100	115.28	2.0582	56.01
6	Case 2	4	h	–2.2260	2.2360	0.0100	21.205	2.2227	9.54
1	Case 2	12	h	0.0000	0.0100	0.0100	192.20	2.2360	85.96
2	Case 2	12	h	-4.4409×10^8	4.4409×10^8	0.0100	168.61	2.7084	62.25
3	Case 2	12	h	-3.3307×10^8	3.3307×10^8	0.0100	89.701	0.9508	94.34
4	Case 2	12	h	0.0000	0.0100	0.0100	98.866	1.0462	94.50
5	Case 2	12	h	0.0000	0.0100	0.0100	115.29	2.0584	56.01
6	Case 2	12	h	-8.8818×10^8	8.8818×10^8	0.0100	221.29	2.2230	99.55
1	Case 2	12	h_i	0.0000	0.0100	0.0100	192.20	2.2360	85.96
2	Case 2	12	h_i	-1.2786×10^8	1.2786×10^8	0.0100	168.61	2.7086	62.25
3	Case 2	12	h_i	-4.3363×10^8	4.3363×10^8	0.0100	89.708	0.95088	94.34
4	Case 2	12	h_i	-2.2204×10^{12}	2.2204×10^{12}	0.0098	101.26	1.0464	96.77
5	Case 2	12	h_i	-1.2536×10^8	1.2536×10^8	0.0100	170.51	2.0582	82.84
6	Case 2	12	h_i	-1.2439×10^8	1.2439×10^8	0.0100	221.25	2.2226	99.54

Table 50: Modified Newton, first step, Trig16 ($n = 2$, $\beta = 10^{-2}$). $\lambda_{\min}(H_0)$ denotes the smallest eigenvalue of the Hessian at x_0 , τ_0 the correction applied ($B_k = H_0 + \tau_0 I$), $\lambda_{\min}(B_k)$ the smallest eigenvalue after correction, and $\text{Ampl.} = \|p_0\|/\|\nabla f(x_0)\|$. With a minimal β -correction, $\lambda_{\min}(B_k)$ stays pinned at β , and the first step can be strongly amplified relative to the gradient norm. For the FD case, the large spurious negative eigenvalues are absorbed by the correction, but the resulting step amplification remains severe.

Start	DM	k	Type	$\lambda_{\min}(H_0)$	Inner iters	η_k	$\ p_0\ $	$\ \nabla f(x_0)\ $	Ampl.
1	Exact	–	–	–1.1426	0	1.1180	2.2361	2.2361	1.00
2	Exact	–	–	0.7124	0	1.3542	2.6935	2.7084	0.99
3	Exact	–	–	–2.2137	1	0.4754	0.6020	0.9508	0.63
4	Exact	–	–	–2.2097	1	0.5231	0.6729	1.0463	0.64
5	Exact	–	–	–1.9160	1	1.0291	5.2543	2.0582	2.55
6	Exact	–	–	–2.2260	1	1.1114	7.4125	2.2227	3.33
1	Case 1	4	h	–1.1426	0	1.1180	2.2361	2.2361	1.00
2	Case 1	4	h	0.7124	0	1.3542	2.6935	2.7084	0.99
3	Case 1	4	h	–2.2137	1	0.4754	0.6020	0.9508	0.63
4	Case 1	4	h	–2.2097	1	0.5231	0.6729	1.0463	0.64
5	Case 1	4	h	–1.9160	1	1.0291	5.2543	2.0582	2.55
6	Case 1	4	h	–2.2260	1	1.1114	7.4125	2.2227	3.33
1	Case 2	4	h	–1.1426	0	1.1180	2.2361	2.2361	1.00
2	Case 2	4	h	0.7124	0	1.3542	2.6935	2.7084	0.99
3	Case 2	4	h	–2.2137	1	0.4754	0.6020	0.9508	0.63
4	Case 2	4	h	–2.2097	1	0.5231	0.6729	1.0463	0.64
5	Case 2	4	h	–1.9160	1	1.0291	5.2543	2.0582	2.55
6	Case 2	4	h	–2.2260	1	1.1114	7.4125	2.2227	3.33
1	Case 2	12	h	0.0000	1	1.1180	7.7121×10^{-8}	2.2360	0.00
2	Case 2	12	h	-4.4409×10^8	0	1.3542	2.7084	2.7084	1.00
3	Case 2	12	h	-3.3307×10^8	0	0.4754	0.9508	0.9508	1.00
4	Case 2	12	h	0.0000	1	0.5231	2.2023×10^{-8}	1.0462	0.00
5	Case 2	12	h	0.0000	1	1.0292	2.7014×10^{-8}	2.0584	0.00
6	Case 2	12	h	-8.8818×10^8	0	1.1115	2.2230	2.2230	1.00
1	Case 2	12	h_i	0.0000	1	1.1180	7.7121×10^{-8}	2.2360	0.00
2	Case 2	12	h_i	-1.2786×10^8	1	1.3543	1.5933×10^{-9}	2.7086	0.00
3	Case 2	12	h_i	-4.3363×10^8	0	0.4754	0.9509	0.9509	1.00
4	Case 2	12	h_i	-2.2204×10^{12}	0	0.5232	1.0464	1.0464	1.00
5	Case 2	12	h_i	-1.2536×10^8	0	1.0291	2.0582	2.0582	1.00
6	Case 2	12	h_i	-1.2439×10^8	0	1.1113	2.2226	2.2226	1.00

Table 51: Truncated Newton, first step, Trig16 ($n = 2$, $\beta = 10^{-2}$). CG it. denotes the number of inner CG iterations performed. A zero iteration count does not necessarily indicate a pure steepest-descent fallback, as the inexact-Newton tolerance may already be satisfied. Even when negative curvature is present, the resulting amplification remains $O(1)$ – $O(3)$, never approaching the large amplification observed for Modified Newton. This behaviour is preserved in the FD case despite the extreme spurious eigenvalues introduced by the finite-difference approximation.

References

- [1] L. Luksan and J. Vlček, *Test Problems for Unconstrained Optimization*, Technical report, Nov. 2003. https://www.researchgate.net/publication/325314497_Test_Problems_for_Unconstrained_Optimization
- [2] C. W. Royer, M. O’Neill and S. J. Wright, *A Newton-CG Algorithm with Complexity Guarantees for Smooth Unconstrained Optimization*, arXiv preprint, Mar. 2018. <https://arxiv.org/pdf/1803.02924>
- [3] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer Series in Operations Research and Financial Engineering, Springer, New York, 2006.